

Contents

pyqula user guide	3
Contents	4
Setting up a Hamiltonian	5
Including second and third neighbor hopping	6
Including an onsite energy	6
Including an external Zeeman field	7
Including an external orbital field	8
Setting a filling	9
Observables	10
Electronic band structures	10
Density of states	11
Local density of states	13
Momentum resolved spectral functions	14
Fermi surfaces	15
Quasiparticle interference	16
Spin splitting of an altermagnet	18
Operators	20
Nonlinear spin current as a measurement of altermagnetic order	20
Spin operators	22
Listing the accepted names	23
Location operator	24
Bulk-edge operator	24
Valley operator	25
In-plane valley operators	25
Nambu operators	26
Berry curvature operator	27
Inverse participation ratio operator	27
Non-Hermitian Hamiltonians	28
Choosing which part is the energy: eigmode	29
What is and is not available	30
Superconductivity	30
s-wave superconductivity	31
Spin-triplet d-vector and non-unitary superconductivity	32
Superfluid weight and BKT temperature	34
Interactions at the mean-field level	36
The collinear Hubbard model	36
Non-collinear Hubbard model	38
Superconducting mean-field	39
Long range interactions	40

Spin-spin exchange interactions	42
Abrikosov-pseudofermion (spinon) mean field for Heisenberg models . .	45
Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice	46
Spatially resolved density of states	48
Electronic structure folding and unfolding	49
Which k-path to unfold along	51
Surface spectral functions	52
Twisted bilayer graphene structural relaxation	53
Topological insulators	54
Topological invariants	54
Quantum geometric tensor (multiorbital/multiband)	57
Berry curvature density in frequency space	59
Berry curvature density in real-space	60
Chern number in real-space	60
Topological surface states	61
Topological markers	62
Entanglement	62
Entanglement entropy and entanglement spectrum	62
Response functions	64
Optical conductivity	64
Charge-charge response function	66
Generic operator-operator response function	67
RKKY response function	67
Spin susceptibility and RPA	68
Excitons and the Bethe-Salpeter equation	76
Quantum transport	83
Magnetoresistance in metal-metal transport	83
Superconductor-metal transport	84
Transport through an arbitrary finite region	85
Landauer transmission through a real-space device	85
Multiple Andreev reflection and AC-Josephson current	86
Single defects in infinite systems	88
Wannierization	89
Symmetry-enforced Wannierization	90
Disentanglement	91
Chebyshev kernel polynomial (KPM) methods	93

Classical spin models	94
Lattice gas models	95
Ising models	97
Parallelism and reproducibility	98
Running on a GPU	98
Errors and unsupported inputs	99
Main functions and methods	101
Geometry functions and methods	101
Hamiltonian functions and methods	102
Heterostructure functions and methods	134
SpinModel functions and methods	136
LatticeGas functions and methods	139
LatticeIsing functions and methods	143

pyqula user guide

pyqula computes the electronic structure of tight-binding models on lattices: band structures, densities of states and spectral functions, self-consistent mean-field Hamiltonians for interacting electrons, superconductivity, topological invariants, response functions, quantum transport, and classical spin and lattice-gas models. This guide goes through them in that order, starting from a single chain and adding one ingredient at a time.

Almost everything in it follows the same four steps: we build a geometry, we generate its Hamiltonian, we add terms to that Hamiltonian, and we ask it for an observable

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # 1. a lattice
h = g.get_hamiltonian()          # 2. its tight-binding Hamiltonian
h.add_zeeman([0.,0.,0.3])       # 3. add terms (in place)
(k,e) = h.get_bands()            # 4. compute an observable
```

The geometry is the lattice, the Hamiltonian is the tight-binding model generated from it (first-neighbor hopping, spinful by default), the terms are added in place, and the observable comes back as arrays. Every chapter of the guide is a set of terms to add or a set of observables to ask for, and the same `h` runs through all of them.

Install with `pip install pyqula`, or from a clone of the repository with `pip install -e .` from its root. Note that `import pyqula` on its own exposes

nothing; always import the submodule you need, as in `from pyqula import geometry`.

Each code block below carries its own imports and runs on its own, except where a block picks up the variables of the one before it inside the same section (a second call on an `h` that was just built, say). The snippets stop at the computed arrays and leave the plotting to you. Two other places in the repository take it further:

- `examples/` holds several hundred runnable scripts organized by dimensionality (0d/ 1d/ 2d/ 3d/, plus `transport/`, `embedding/`, `wannier/`, `classicalspin/`, `latticegas/`, `latticeising/`, `spinon/`, `kondolattice/`, `minimal/` and `readme_examples/`), most of them ending in a figure. Most sections below point at the relevant ones
- `jupyter-notebooks/functionalities/` holds 53 executed notebooks, one per feature, grouped the same way as the README's functionality list (single-particle Hamiltonians, mean field, topology, spectral functions, KPM, Wannierization, transport). Each carries its physics discussion and its output plots inline, so they are the place to look for what a result should actually look like

The last chapter, Main functions and methods, is a reference of the `Geometry` and `Hamiltonian` methods and their arguments, followed by the `Heterostructure`, `SpinModel`, `LatticeGas` and `LatticeIsing` classes. It is not exhaustive: it covers the methods the sections above use, not every public method on those classes.

Contents

- Setting up a Hamiltonian
- Observables
- Operators
- Non-Hermitian Hamiltonians
- Superconductivity
- Interactions at the mean-field level
- Spatially resolved density of states
- Electronic structure folding and unfolding
- Surface spectral functions
- Twisted bilayer graphene structural relaxation
- Topological insulators
- Entanglement
- Response functions
- Quantum transport
- Single defects in infinite systems
- Wannierization
- Chebyshev kernel polynomial (KPM) methods
- Classical spin models

- Lattice gas models
- Ising models
- Parallelism and reproducibility
- Errors and unsupported inputs
- Main functions and methods

Setting up a Hamiltonian

Let us start with the simplest tight-binding model there is, a one-dimensional chain with hopping between first neighbors, and see how its band structure is computed. Everything else in this guide is built the same way: a geometry, the Hamiltonian generated from it, terms added to that Hamiltonian, and a quantity computed from the result. In this chapter we generate the Hamiltonian and add the first few terms to it, longer-range hoppings, an onsite energy, a Zeeman field, an orbital magnetic field and a filling; the observables come in the next chapter.

The Hamiltonian of a one-dimensional tight-binding chain takes the form

$$H = \sum_n c_n^\dagger c_{n+1} + h.c.$$

where $c_n^\dagger$ creates an electron at site n , so that each term moves an electron from site $n+1$ to site n and its hermitian conjugate moves it back. The hopping is the unit of energy, $t = 1$, and the distance between sites is the unit of length. This model can be diagonalized analytically, giving rise to a diagonal Hamiltonian of the form

$$H = \sum_k \epsilon_k \Psi_k^\dagger \Psi_k$$

where the energy-momentum dispersion takes the form

$$\epsilon_k = 2 \cos k$$

With the pyqula library, the previous band structure can be computed as

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
(k,e) = h.get_bands() # compute band structure
```

The geometry holds the positions of the sites and the lattice vector, the Hamiltonian is generated from it with first-neighbor hopping, and `h.get_bands()` returns the k-points along the path and the energies at each of them, in this case the cosine band above. The Hamiltonian is spinful by default, meaning that every energy appears twice, once per spin; a spinless one is generated

with `g.get_hamiltonian(has_spin=False)`. The methods that add a term to a Hamiltonian, all of them named `add_something`, modify `h` in place, so a calculation is a sequence of calls on the same object.

See `examples/1d/linear_chain/main.py` for a runnable version ending in a plot (with a second-neighbor hopping added, so the band is not the pure cosine above), and the notebooks in `jupyter-notebooks/functionalities/single_particle_hamiltonians/` for the spinless, spinful and Nambu bases and for models from zero to three dimensions.

Including second and third neighbor hopping

By default, the Hamiltonian generated includes only first-neighbor hopping, $t_1 = 1$. We will now see how to include hopping to sites further away, considering a generalized Hamiltonian of the form

$$H = \sum_n c_n^\dagger c_{n+1} + t_2 \sum_n c_n^\dagger c_{n+2} + t_3 \sum_n c_n^\dagger c_{n+3} + h.c.$$

where t_2 and t_3 are the second- and third-neighbor hoppings, in units of t_1 . Passing the list of hoppings to `g.get_hamiltonian()`, first neighbor first, generates this Hamiltonian; to compute the eigenvalues taking $t_2 = 0.2$ and $t_3 = 0.3$, we write

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian(tij=[1.0,0.2,0.3]) # Hamiltonian with t1,t2,t3
(k,e) = h.get_bands() # compute band structure
```

The dispersion is now $\epsilon_k = 2 \cos k + 2t_2 \cos 2k + 2t_3 \cos 3k$, and what you see in the band structure is that the band is no longer symmetric between positive and negative energies: the second-neighbor hopping is what breaks that symmetry, while the first- and third-neighbor ones preserve it. The same list works for any geometry, and a function of the two positions or a hopping generator can be passed instead of the list; see `g.get_hamiltonian()` in the reference chapter.

See `examples/1d/NNN_chain/main.py` for a runnable version.

Including an onsite energy

An onsite energy changes the energy of an electron that sits on a site, without moving it. With the same value μ on every site it is a chemical potential,

$$H = \mu \sum_n c_n^\dagger c_n$$

meaning that the whole band structure is shifted rigidly by μ . Adding a uniform onsite energy is therefore how the Fermi energy of a Hamiltonian is moved, since

the observables that depend on it, the filling, the density of states at the Fermi level, the expectation values, take zero energy as the Fermi energy. This can be added to the Hamiltonian with `h.add_onsite()` as

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
mu = 0.3 # value of the onsite
h.add_onsite(mu) # add onsite energy
```

Possible inputs

- Float: the same onsite energy is added to all the sites
- Iterable (list or array): adds a different onsite energy to each site in the geometry, one value per site
- Callable (function): adds a different onsite energy to each site according to its location $\mathbf{r}$

A site-dependent onsite energy is how a potential landscape is built: a sublattice imbalance in the honeycomb lattice, a single impurity (an onsite energy much larger than the bandwidth on one site, as in the quasiparticle interference section), or a smooth electrostatic potential given as a function of the position.

Including an external Zeeman field

We will now add an external magnetic field acting on the spin of the electrons, a Zeeman field, with `h.add_zeeman()`. We now include the existence of a spin degree of freedom, considering the Hamiltonian

$$H = H_0 + H_Z$$

where H_0 is the original tight-binding Hamiltonian

$$H_0 = \sum_{n,s} c_{n,s}^\dagger c_{n+1,s} + h.c.$$

and

$$H_Z = \sum_{n,s,s'} \vec{B} \cdot \vec{\sigma}^{s,s'} c_{n,s}^\dagger c_{n,s'}$$

with n running over the sites and s, s' running over the spin degree of freedom. The magnetic field takes the form $\vec{B} = (B_x, B_y, B_z)$, and σ_α are the spin Pauli matrices, so that the field splits the two spin bands by $2|\vec{B}|$ and sets the direction along which the spin is quantized. To add a magnetic field of the form $\vec{B} = (0.1, 0.2, 0.3)$ to our chain we write

```

from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
h.add_zeeman([0.1,0.2,0.3]) # add the Zeeman field (modifies h in place)
(k,e) = h.get_bands() # compute band structure

```

The Hamiltonian is spinful by default, so the field acts on a spin degree of freedom that is already there; on a spinless Hamiltonian this call adds the spin degree of freedom first. The field can also differ from site to site, given as one vector per site or as a function of the position. The same kind of term, written as the magnetization of the material rather than as an external field, is `h.add_exchange()`: the two add the same matrix, and the name follows the physics, an exchange field being what a magnet has and what we will find self-consistently in the mean-field chapter.

Including an external orbital field

A magnetic field also acts on the orbital motion of the electrons, through the phase that a hopping picks up when an electron goes from one site to another. This is the Peierls substitution

$$t_{\alpha\beta} \rightarrow t_{\alpha\beta} e^{i \int_{r_\alpha}^{r_\beta} \vec{A} \cdot d\vec{l}}$$

where $\vec{A}$ is the vector potential, so that $\vec{B} = \nabla \times \vec{A}$, and the integral runs along the bond. Let us apply it to a ribbon, a system finite in one direction and periodic in the other, with `h.add_orbital_magnetic_field()`

```

from pyqula import geometry
N = 20 # number of unit cells as the width
g = geometry.square_ribbon(N) # ribbon
h = g.get_hamiltonian() # generate the Hamiltonian
B = 0.02 # magnetic field in quantum flux unit
h.add_orbital_magnetic_field(B) # add an out-of plane magnetic field
(k,e) = h.get_bands() # compute the Landau-level band structure

```

B is the magnetic flux through a plaquette of unit area, in units of the flux quantum, so that B = 0.02 means one flux quantum every fifty plaquettes. What you see in the band structure is the quantum Hall effect: flat bands, the Landau levels, the lowest of them close to $-4 + 2\pi B$ for the square lattice, and between them dispersive bands that cross the gaps, the chiral states at the two edges of the ribbon, propagating in opposite directions on opposite edges. For a honeycomb ribbon the same call gives the Landau levels of the Dirac equation, with a level pinned at zero energy.

See `examples/1d/landau_levels_zigzag_ribbon/main.py` for a runnable version on a honeycomb ribbon.

Setting a filling

Up to now the Fermi energy of the chain has been at zero energy, which for the cosine band is half filling. If you want to enforce a certain filling ν in a Hamiltonian, the fraction of the states that are occupied, so that

$$\langle c_n^\dagger c_n \rangle = \nu$$

on average over the sites and, in a spinful system, over the two spins, use

```
from pyqula import geometry
g = geometry.chain() # chain
h = g.get_hamiltonian()
h.set_filling(0.7) # enforce a filling
```

`h.set_filling()` computes the Fermi energy that gives this filling on a k-mesh of `nk` points per direction, and adds the onsite energy that brings it to zero, so that afterwards every observable that takes zero as the Fermi energy is at the filling you asked for. The filling is a single number, enforced on average over the whole system, with $\nu = 0.5$ being half filling. For a metal the k-mesh matters, since the filling changes continuously with the Fermi energy; for an insulator any Fermi energy inside the gap gives the same filling. The same keyword goes into `h.get_mean_field_hamiltonian(filling=...)` in the mean-field chapter, where the filling is kept fixed along the self-consistent calculation.

What if the filling should differ from site to site, for example to model a charge imbalance between two sublattices imposed by a gate or a substrate? A single Fermi energy cannot do that, so the way we do this is by giving `h.set_filling()` one filling per site, and it then solves for one onsite energy per site such that every site reaches its own filling

```
import numpy as np
from pyqula import geometry
g = geometry.honeycomb_lattice() # honeycomb lattice, two sites per unit cell
h = g.get_hamiltonian() # first-neighbor Hamiltonian
h.set_filling(np.array([0.4,0.6])) # filling 0.4 on sublattice A, 0.6 on sublattice B
occ = h.get_vev() # occupation of each site
```

Each entry keeps the convention of the scalar filling, the fraction of the states of that site that are occupied, so in this spinful example `occ` comes out as roughly 0.8 and 1.2 electrons, and the average filling is the mean of the array, here half filling. The same per-site solve is reached with a scalar and `average=False`, which enforces the same filling on every site rather than on average. Note that on a finite island the levels are discrete and the total number of electrons is an integer, so fillings that add up to a fractional number of electrons are only reached through the broadening $\delta = 0.01$ the solver uses, and the occupations should be read with `h.get_vev(delta=1e-2)`.

Observables

With a Hamiltonian in hand, let us now ask it for the quantities that describe its electronic structure: the band structure, the density of states and its local and momentum-resolved versions, the Fermi surface, the quasiparticle interference pattern of a defect, and the spin splitting of an altermagnet. Each of them is one method on the Hamiltonian, and each returns the arrays a plot is made of.

Electronic band structures

We will start with the band structure, with `h.get_bands()`. For any system that is periodic in space, we can compute the electronic band structure as given by

$$H = \sum_{k,\alpha} \epsilon_{k,\alpha} \Psi_{k,\alpha}^\dagger \Psi_{k,\alpha}$$

where α is the band index, meaning that at each momentum k the Hamiltonian is diagonal with one energy per band. The previous calculation can be performed as

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # geometry of the 2D model
h = g.get_hamiltonian() # generate the Hamiltonian
(k,e) = h.get_bands() # compute band structure
```

`h.get_bands()` returns two arrays of the same length, the position along the k-path and one energy per band at each k-point, so the two together are the scatter plot of the band structure. For the honeycomb lattice what you see is the two bands touching at zero energy at the K point, the Dirac point, with a bandwidth of three times the hopping on each side.

Optional arguments - **kpath**: k-path to use, either a list of high-symmetry labels (e.g. ["G", "K", "M"]) or explicit k-vectors; defaults to the geometry's standard path - **nk**: number of k-points along the path - **operator**: color/weight each band by the expectation value of an operator (or a list of operators), returning (k,e,c) instead of (k,e) - **num_bands**: for large sparse Hamiltonians, only compute this many bands around **central_energy**, instead of the full spectrum

Passing an operator computes its expectation value in every eigenstate and returns it as a third array, which is what colors the bands in a plot: the group velocity below tells apart flat bands from dispersive ones, and the spin operator "sz" tells apart the two spin channels of a magnet, which we will use in the mean-field chapter

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
```

```
h = g.get_hamiltonian(has_spin=True)
(k,e,c) = h.get_bands(operator="velocity") # bands colored by group velocity
```

Passing a list of operators computes the expectation value of each of them for every eigenstate, returning one extra column per operator ((k,e,c1,c2,...) instead of (k,e,c))

```
(k,e,c_sz,c_site) = h.get_bands(operator=["sz","site"]) # two operators at once
```

For a very large system, a moire supercell say, diagonalizing the full Hamiltonian at every k-point is wasteful if only a handful of bands around the Fermi level are of interest. Passing num_bands switches to a sparse solver that computes only those bands

```
(k,e) = h.get_bands(num_bands=20) # only the 20 bands closest to central_energy
```

See `examples/2d/velocity_bands/main.py` and `examples/2d/strain_TBG/main.py` for runnable versions, and `jupyter-notebooks/functionalities/single_particle_hamiltonians/04_states` for an executed notebook with operator-colored bands.

Density of states

We will now count the states rather than follow them along a path, with `h.get_dos()`. The density of states counts how many states are in a certain energy window. It is defined as

$$D(\omega) = \int \delta(\omega - \epsilon_k) dk$$

where ϵ_k are the eigenenergies of the Hamiltonian, meaning that every band contributes a plateau where it disperses and a peak where it is flat, so that the density of states is where the van Hove singularities of a band structure show up. It can be used as shown below

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(es,ds) = h.get_dos()
```

`h.get_dos()` returns the energies and the density of states at each of them, by default on a window from -4 to 4 in units of the hopping, sampled on a k-mesh and broadened with a Lorentzian of width `delta`. For the triangular lattice what you see is a density of states that is not symmetric in energy, with a van Hove singularity at $E = -2$, the saddle point of the dispersion at the M point; the top of its band, at $E = 6$, lies outside the default window, so pass `energies=` when the whole band is needed.

Optional arguments - `energies`: array with the energies for which the DOS is computed - `delta`: smearing of the DOS - `operator`: operator to which the DOS is projected - `mode`: how the DOS is computed, "ED" (default), "Green"/"RG",

"KPM" or "adaptive" - nk: number of k-points per direction of the mesh, for "ED" and "KPM"

The four modes are four ways of doing the same integral. "ED" diagonalizes the Hamiltonian on a k-mesh and broadens the eigenvalues; "Green" and "RG" sum a Green's function at each energy, which pays off when only a handful of energies are needed; "KPM" expands the density of states in Chebyshev polynomials and is the choice for large sparse systems (see the section on Chebyshev kernel polynomial methods); "adaptive" does not sample a mesh at all: it integrates over the Brillouin zone with an error-controlled quadrature, tuned by `error=1e-1`, and uses `nk` only as a limit on how far it subdivides. "Green" and "RG" pass `nk` to the Brillouin-zone sum behind the self-energy, where it matters for `gmode="full"` and, in two dimensions, for `gmode="renormalization"`, but not for the default `gmode="adaptive"`.

Whatever the mode, the result is normalized as a density of states, meaning that integrating it over the energies gives the number of states per unit cell: two for a spinful chain, and all four modes reproduce that number to better than 1% on the same window, so they can be compared with each other directly, and so can the density of states that `h.get_multidos()` writes next to an LDOS map. On the spinful chain below, with longer-range hopping and Rashba coupling, the two curves fall on top of each other

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian(tij=[0.5,0.,0.,0.5],has_spin=True)
h.add_rashba(0.7)
energies = np.linspace(-4.,4.0,60)
(e1,d1) = h.get_dos(energies=energies,delta=1e-2,mode="ED",nk=1000)
(e2,d2) = h.get_dos(energies=energies,delta=1e-2,mode="Green")
```

An operator can be passed to project the DOS onto a subspace, e.g. the sublattice-resolved DOS of a gapped honeycomb lattice, where the sublattice imbalance puts the states below the gap on one sublattice and the states above it on the other, so that the projected density of states comes out with opposite signs on the two sides of the gap

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=True)
h.add_sublattice_imbalance(.4)
(es,ds) = h.get_dos(operator="sublattice",nk=40,delta=5e-2)
```

See `examples/1d/dos_GF/main.py` and `examples/2d/operator_dos/main.py` for runnable versions, and `jupyter-notebooks/functionalities/single_particle_hamiltonians/06_operator_dos` for an executed notebook.

Local density of states

We will now resolve the density of states in space, with `h.get_ldos()`, which gives the quantity that STM measures. The local density of states resolves the density of states by site: it counts how many states are in a certain energy window, weighted by how much of each state sits on site n . It is defined as

$$D(\omega, n) = \int \delta(\omega - \epsilon_k) |\langle \Psi_k | n \rangle|^2 dk$$

where ϵ_k are the eigenenergies of the Hamiltonian and $\langle \Psi_k | n \rangle$ is the amplitude of each eigenstate on site n , so that summing over the sites gives back the density of states. It can be used as shown below

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(x,y,d) = h.get_ldos()
```

`h.get_ldos()` returns three arrays, the x and y coordinates of the sites and the local density of states on each of them, at the energy `e` (zero by default). For the zigzag ribbon what you see at zero energy is that three quarters of the weight sits on the outermost sites of the two edges: these are the zigzag edge states, and this profile is what one measures with STM at a zigzag edge of graphene.

Optional arguments - `e`: energy at which the LDOS is evaluated - `delta`: smearing of the LDOS - `operator`: operator to which the LDOS is projected (a name such as `"sz"`, a matrix, or an `Operator`), giving e.g. a spin-resolved real-space map instead of the charge one - `mode`: `"arpack"` (default, diagonalization on a k-mesh) or `"green"` (Green's function, 2D Hamiltonians only) - `projection`: `"TB"` (default, one value per lattice site), `"TBRs"` (same, but interpolated onto a continuous real-space map for smoother plotting), or `"atomic"` (projected onto atomic orbitals rather than tight-binding sites) - `num_bands`: for large sparse Hamiltonians, only compute this many states around the target energy

The two evaluation modes weight an operator differently, and both integrate over sites to the same operator-resolved density of states: `mode="arpack"` uses the expectation value $\langle \Psi | A | \Psi \rangle$ of each eigenstate times its local density $|\Psi(i)|^2$, while `mode="green"` uses the local matrix element $\text{Re} \Psi^*(i) (A\Psi)(i)$, which is the genuinely local quantity for states that are not eigenstates of the operator. A momentum-dependent operator (`"valley"`) only works in `mode="arpack"`, since `mode="green"` has already integrated over the Brillouin zone.

For a smoother map than one value per site, `projection="TBRs"` interpolates the same weights onto a continuous real-space grid

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
```

```
h = g.get_hamiltonian() # get the Hamiltonian
(x,y,d) = h.get_ldos(e=0.0,projection="TBRs") # interpolated real-space map
```

To follow how the map evolves with energy, `h.get_multildos()` computes the LDOS at many energies at once, writing one file per energy to a `MULTILDOS/` folder (the frames of an `LDOS(x,y,E)` stack), plus a `MULTILDOS/DOS.OUT` holding the corresponding total DOS

```
import numpy as np
h.get_multildos(energies=np.linspace(-2.0,2.0,100),projection="atomic")
```

The maps and the `DOS.OUT` beside them carry the same normalization as `h.get_ldos()` and `h.get_dos()` on the same system, so a map can be read against a single-energy LDOS and the `DOS.OUT` against `h.get_dos()` without rescaling. An `operator=` is honoured here as it is by `get_ldos` (the older spelling `op=` still works, but passing both is a `TypeError`), and the weight it applies is the expectation value $\langle \Psi | A | \Psi \rangle$ of the eigenstate, a gauge-invariant number, times the local density, the same convention `get_ldos(mode="arnpack")` uses. `projection="atomic"` does not accept an operator and says so

See `examples/0d/island/main.py` (single-energy, `projection="TBRs"`, superconducting island) and `examples/readme_examples/ldos_island/main.py` (`get_multildos`, `projection="atomic"`) for runnable versions.

Momentum resolved spectral functions

Apart from the band structure, in certain cases it is interesting to compute the momentum-resolved spectral function, with `h.get_kdos_bands()`, that takes the form

$$A(k, \omega) = \delta(\omega - \epsilon_k) |\langle \Psi_k | A | \Psi_k \rangle|^2$$

where A is a certain operator. The previous quantity allows defining a heatmap of the momentum-resolved spectral function, the band structure seen the way ARPES sees it, with each band drawn as a line of finite width and weighted by the operator. For example, in a superconducting state, if the operator is chosen to be the projection onto the electron sector, the previous quantity shows the electronic spectral function, with the hole copy of the bands removed

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
h.get_kdos_bands()
```

`h.get_kdos_bands()` returns three arrays, the position along the k -path, the energy, and the spectral weight at each pair, on a window from -3 to 3 by

default, and writes the same three columns to `KDOS_BANDS.OUT`; the broadening `delta` sets the width of each line in the heatmap.

Optional arguments - `energies`: array with the energies for which the spectral function is computed - `delta`: smearing of the spectral function - `operator`: operator to which the spectral function is projected - `kpath`, `nk`: the k-path and the number of k-points along it, as in `h.get_bands()` - `mode`: "ED" (default, diagonalization), "green" (Green's function) or "KPM" (Chebyshev expansion, for large sparse systems)

See `examples/1d/kdos_armchair/main.py` for a runnable version on a wide ribbon with `mode="KPM"`, and `jupyter-notebooks/functionalities/single_particle_hamiltonians/05_mom` and `jupyter-notebooks/functionalities/spectral_functions/06_operator_momentum_resolved.ipynb` for executed notebooks.

Fermi surfaces

Let us now cut the band structure at a single energy rather than follow it along a path. The Fermi surface is the set of momenta where a band crosses the Fermi energy, and it is what a metal looks like in reciprocal space. For a 2D periodic Hamiltonian, `h.get_fermi_surface()` computes the spectral weight on a (k_x, k_y) mesh at a single energy (by default the Fermi level, `e=0.0`), i.e. a single constant-energy cut

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(kx,ky,fs) = h.get_fermi_surface(e=0.0,nk=50,delta=1e-1)
```

It returns the two momentum coordinates of the mesh and the spectral weight at each point, so a scatter plot colored by the weight is the Fermi surface; the broadening `delta` sets how thick the contour is drawn, and a value below the level spacing of the mesh shows the mesh rather than the contour.

Optional arguments - `e`: energy of the cut - `nk`: number of k-points per direction - `delta`: broadening - `operator`: project/weight the Fermi surface by an operator, giving e.g. a spin- or valley-textured Fermi surface

An operator weights each point of the Fermi surface by its expectation value, which is how the spin texture of a spin-orbit coupled material is seen: the NbSe2 Hamiltonian below is a multi-orbital model with spin-orbit coupling, and coloring its Fermi surface by the spin along z tells the two spin channels apart on each pocket

```
from pyqula.specialhamiltonian import NbSe2
h = NbSe2(soc=0.9) # multi-orbital spin-orbit-coupled Hamiltonian
(kx,ky,fs) = h.get_fermi_surface(e=0.,nk=100,delta=3e-1,operator="sz")
```

`h.get_multi_fermi_surface()` computes the same kind of map at many energies at once, writing one file per energy to a `MULTIFERMISURFACE/` folder, which

is the way to scan how the Fermi surface evolves away from the Fermi level

```
import numpy as np
h.get_multi_fermi_surface(energies=np.linspace(-4,4,100),delta=1e-1)
```

Passing `operator="unfold"` together with `nsuper` unfolds the Fermi surface of a defective or disordered supercell back onto the primitive Brillouin zone (see the section on electronic structure folding and unfolding); as with QPI unfolding below, the supercell must be built with `store_primal=True`

```
import numpy as np
from pyqula import geometry
g0 = geometry.triangular_lattice()
n = 3 # size of the supercell
g = g0.get_supercell(n,store_primal=True)
h = g.get_hamiltonian()
h.add_onsite(lambda r: 100.0 if np.linalg.norm(r-g.r[0])<1e-1 else 0.0) # a point defect

out = h.get_multi_fermi_surface(nk=50,energies=np.linspace(-4,4,100),delta=0.1,
                                nsuper=n,operator="unfold")
```

See `examples/readme_examples/fermi_surface/main.py`, `examples/2d/operator_fermi_surface/main.py` and `examples/readme_examples/unfolding_FS/main.py` for runnable versions.

Quasiparticle interference

Let us now see what a single defect does to the local density of states of a two-dimensional system, with `h.get_qpi()`. A defect scatters an electron from a state at momentum $\vec{k}$ to a state at $\vec{k}'$ on the same constant-energy contour, and the two interfere, so that the local density of states around the defect is modulated with wavevector $\vec{q} = \vec{k} - \vec{k}'$. The Fourier transform of that modulation at each energy is what is called the quasiparticle interference pattern, and it is what an STM measurement obtains by Fourier transforming a conductance map: a picture of the constant-energy contours of the band structure, and of which pairs of states the defect connects. `h.get_qpi()` computes it for 2D Hamiltonians; unlike the other observables here it does not return arrays but writes one file per energy to a folder (`MULTIQPI/` by default) and the density of states next to it in `DOS.OUT`

```
import numpy as np
from pyqula import geometry
g = geometry.triangular_lattice()
h = g.get_hamiltonian(has_spin=False)
h.get_qpi(mode="pm",nk=50,delta=1e-1,energies=np.linspace(-6.,6.,100))
```

Optional arguments - `energies`: array of energies to compute - `nk`: number of k-points per direction - `delta`: broadening - `mode`: "pm" (autoconvolution of the k-resolved spectral weight) or "response" (default, joint density of states of the

clean band structure) - `nunfold`: for a defect embedded in an `nunfoldxnunfold` supercell, unfold the QPI signal back onto the primitive Brillouin zone

The two modes differ in what they scatter. `"pm"` (“poor man’s”) autoconvolves the actual k -resolved spectral weight of the system, defect included, in q -space, which is the physically meaningful QPI signal for a real scatterer; `"response"` instead computes a cheaper Lindhard-like convolution of the joint density of states from the clean band structure only, ignoring the form factors of the wavefunctions.

A single point defect embedded in a supercell, with the resulting QPI unfolded back onto the primitive cell, is the realistic case. The supercell must be built with `store_primal=True` so that `pyqula` remembers the primitive cell needed to unfold; `operator="unfold"` then resolves to the corresponding unfolding operator

```
import numpy as np
from pyqula import geometry
g0 = geometry.honeycomb_lattice()
ns = 2
g = g0.get_supercell(ns, store_primal=True)
h = g.get_hamiltonian(has_spin=False)
h.add_onsite(lambda r: 100.0 if np.linalg.norm(r-g.r[0])<1e-1 else 0.0) # a strong point de
h.get_qpi(mode="pm", delta=1e-2, operator="unfold", nsuper=2, nk=140, nunfold=ns)
```

This is the most expensive snippet in the guide: `mode="pm"` diagonalizes on an `nkxnk` mesh and then autoconvolves the result, so the cost grows quadratically with `nk` and the `nk=140` above takes minutes. Drop to `nk=60` (about 40 seconds) while setting a calculation up, and raise `nk` only for the final figure, since the q -space resolution of the QPI pattern is what it buys.

See `examples/2d/multiqpi/main.py` (clean system, `mode="pm"`) and `examples/2d/multiqpi_unfold/main.py` (defect in a supercell, unfolded) for runnable versions.

Real-space-impurity QPI

The modes of `h.get_qpi()` all work in reciprocal space: they convolve or scatter k -resolved spectral weight and never touch a real-space impurity. `h.get_qpi_impurity()` takes the direct route instead, the one an experiment takes: it builds a supercell of `h`, puts one or more actual impurities in it, computes the real-space LDOS map around them, and Fourier transforms that map to obtain the QPI signal as a function of $\vec{q}$. Unlike `get_qpi()`, it returns arrays rather than only writing to disk

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)
```

```

r,ldos_r,q,qpi_q = h.get_qpi_impurity(nsuper=10,
    impurities=[{"position": [0.,0.,0.], "onsite": 3.0}],
    energies=0.3,num_waves=60,nk=2,delta=0.2)

```

The four arrays are the positions of the sites and the LDOS on them, and the q-points and the QPI weight on them, so the first pair is the real-space map and the second its Fourier transform.

Optional arguments - `nsuper`: supercell size (scalar or (n1,n2)) - `impurities`: list of dicts, each an onsite potential (`{"position": [x,y,z], "onsite": v}`), or `{"index": i, "onsite": v}` for a specific supercell site index) or a vacancy (`{"position": [x,y,z], "vacancy": True}`) - `energies`: a single energy or an array - `num_waves`: a starting guess for how many eigenstates nearest the requested energies to compute - `nk`, `delta`: as in `get_ldos`

A vacancy is modeled as a strong onsite potential rather than as a removed site, since deleting sites from a large sparse supercell Hamiltonian would require densifying it. The number of eigenstates `num_waves` is only a starting guess: it is grown automatically until the window covers `margin*delta` past every requested energy and never cuts a degenerate manifold in half, because summing over half a degenerate manifold is not basis independent and would leak spurious QPI weight even for a clean supercell.

No unfolding step is needed here: this route never diagonalizes supercell bands and projects them onto primitive Bloch states (which is what `get_qpi`'s `unfold` and `store_primal` are for). It only Fourier transforms a real-space scalar density, evaluated directly at `q` spanning the full primitive Brillouin zone. `q` is fixed at exactly the `nsuper1*nsuper2` points commensurate with the supercell, meaning that `nsuper` sets the achievable `q` resolution, not the range of the Brillouin zone; evaluating the direct-sum Fourier transform at any other `q` would show finite-size leakage even for a perfectly clean system, since only the commensurate points are free of it.

See `examples/2d/qpi_realspace_impurity/main.py` for a runnable version that plots both the real-space LDOS and QPI(`q`).

Spin splitting of an altermagnet

Let us now look at a magnet with no net magnetization whose two spin channels nevertheless have different bands. An altermagnet is a collinear magnet whose spin-up and spin-down bands are split even though it carries no net magnetization, and the natural measure of that order is how far apart the two spin channels are pushed, and at which energy. Both quantities below diagonalize the two spin blocks separately and pair the bands by index, so that every k-point and band index n has a splitting $\Delta_n(k) = E_n^\uparrow(k) - E_n^\downarrow(k)$ sitting at the mean energy $\bar{E}_n(k) = (E_n^\uparrow(k) + E_n^\downarrow(k))/2$.

The two differ in how they reduce that set. `h.get_spin_splitting_density()` broadens every pair into a smooth weighted density, giving the typical splitting

at each energy. `h.get_spin_splitting_vs_energy()` instead keeps the largest $|\Delta|$ found in each energy bin, so that its global maximum is a bound on the spin splitting anywhere in the Brillouin zone. The distinction matters when reporting a single number for a material: a maximum taken along one cut through reciprocal space (a circle of fixed radius, say) depends on choosing where to look, while the binned maximum over a full mesh does not

```
import numpy as np
from pyqula import specialhamiltonian

h = specialhamiltonian.square_altermagnet(am=1.)

# largest spin splitting in each energy bin, over the whole BZ
(es, ds) = h.get_spin_splitting_vs_energy(nk=100, nbins=400)
print("largest spin splitting anywhere in the BZ:", ds.max())

# the smooth counterpart, on the same axes
(xs, ys) = h.get_spin_splitting_density(nk=100, delta=1e-1, energies=es)
```

Both return (energies, values) with the same shape convention, so they can be plotted together. Bins containing no states come back as 0.0 rather than NaN, and states falling outside an explicitly requested energy window are dropped rather than piled onto the end bins.

Two things about that index pairing decide whether the number you report is the number you meant.

The first is reassuring. In a collinear altermagnet the two spin channels are related by a point-group operation acting on k rather than by a state-by-state correspondence at fixed k : the sorted spectra satisfy $E_n^\uparrow(k) = E_n^\downarrow(Rk)$ exactly, so that $\Delta_n(k) = E_n^\downarrow(Rk) - E_n^\downarrow(k)$ compares the same sorted index within one channel at two related momenta. No band-identification ambiguity survives, and the usual worry, that a splitting exceeding the band spacing makes the n -th up and n -th down band different bands, does not arise, because the symmetry supplies the correspondence. For a new system it pays to identify R : compare `sorted(E_up(k))` with `sorted(E_dn(Rk))` over the point group and look for the operation giving $\sim 1e-14$. For the square altermagnet it is the $k_1 \leftrightarrow k_2$ mirror.

The second is a real dependence that symmetry does not remove. The band index n labels whatever set of bands your unit cell produces, and folding changes that set: on a supercell, bands at one k come from several primitive k -points, and pairing them by index compares across those. The reported maximum then comes out too small. Give this the true magnetic unit cell: if the converged order repeats with a shorter period than the cell it was solved in, the result is a lower bound rather than the maximum.

Both require spin to be a good quantum number, since they keep one spin block and discard the off-diagonal one. `h.get_spin_splitting_vs_energy()` checks

this and raises if the spin off-diagonal block of the Bloch Hamiltonian is not negligible: with Rashba coupling, any other spin-orbit term, or non-collinear magnetic order the splitting defined above is not a meaningful quantity, and a silently wrong number would be worse than an error. Diagonalization is dense throughout, deliberately: a sparse solver returns only the eigenvalues nearest $E = 0$, and the splitting commonly peaks far away from there.

See `examples/2d/spin_splitting_vs_energy/main.py` for a runnable version plotting both curves (and showing the redundant-cell trap), and `examples/2d/altermagnetism_density/main.py` for the density alone.

Operators

When computing band structures, densities of states or expectation values we can pass an operator to filter the result, so that every state carries the expectation value of that operator next to its energy. Let us now go through the operators that are available, and comment on their physical meaning.

Operators in `pyqula` have some important properties. First, for periodic Hamiltonians they can have an intrinsic momentum dependence. Second, `pyqula` allows for native algebra between them, namely they can be summed or multiplied, automatically accounting for intrinsic momentum dependences. Third, they can be non-linear, providing a generalization of matrix operators.

Every operator below is obtained with `h.get_operator(name)`, and goes into the `operator=` argument of `h.get_bands()`, `h.get_dos()`, `h.get_ldos()`, `h.get_kdos_bands()` or `h.get_vev()`: the band structure comes back with one extra column, the expectation value of the operator in each eigenstate, the density of states becomes the density of states weighted by that expectation value, and so on. The executed notebooks `jupyter-notebooks/functionalities/single_particle_hamiltonians/04_state_resolved_bands.ipynb` and `06_operator_resolved_dos.ipynb` show what an operator-resolved band structure and density of states look like. Before the operators themselves, the first section takes up a quantity that is not a matrix operator at all, the nonlinear spin current, which continues the altermagnet section of the previous chapter.

Nonlinear spin current as a measurement of altermagnetic order

The spin splitting of the previous chapter tells you how big the altermagnetic order is. A harder question is which kind it is, d-wave, g-wave or i-wave, and that question turns out to have a purely electrical answer, one that needs no spin-orbit coupling at all. We will now see how to read the wave index off the nonlinear spin current, with `h.get_nonlinear_drude_orders()`.

The idea is the following. The spin-splitting form factor of an X-wave collinear

magnet is a harmonic of order $l+1$ in reciprocal space: $k_x k_y$ for d-wave, $k_x k_y (k_x^2 - k_y^2)$ for g-wave, $k_x k_y (3k_x^2 - k_y^2) (k_x^2 - 3k_y^2)$ for i-wave. In the semiclassical Boltzmann treatment, the l -th order nonlinear Drude conductivity is a Brillouin-zone integral of the $(l+1)$ -th derivative of the band energy,

$$\sigma_{\mathbf{s}}^{x^{l1} y^{l2}; \mathbf{b}} = (-e/\hbar)^{(l+1)} / (i\omega + 1/\tau)^l \int d^2k / (2\pi)^2 f_s^{(0)} d^{(l+1)} \epsilon_{\mathbf{s}} / (dk_x^{l1} dk_y^{l2} dk_b)$$

with $\sigma_{\text{spin}} = (\sigma_{\text{up}} - \sigma_{\text{dn}})/2$ and $\sigma_{\text{charge}} = \sigma_{\text{up}} + \sigma_{\text{dn}}$. A harmonic of order $l+1$ has a nonzero constant $(l+1)$ -th derivative, and nothing below it survives the zone integral, so the spin response switches on at exactly one order and is silent below it:

wave	p	d	f	g	i
lowest order	1	0	1	2	3
					5

Reading off the lowest order at which a nonlinear spin current appears therefore identifies the wave index. This is the content of Ezawa, *Phys. Rev. B* **111**, 125420 (arXiv:2411.16036). For an i-wave altermagnet we write

```
from pyqula import specialhamiltonian

h = specialhamiltonian.iwave_altermagnet(J=0.3)  # or dwave_/fwave_/gwave_

# largest |sigma_spin| over every component of each order l
orders = h.get_nonlinear_drude_orders(lmax=6, nk=48, T=0.02, mu=-5.5)
# -> zero to machine precision for l = 0..4, nonzero at l = 5

# one component, and every component of a given order
s = h.get_nonlinear_drude_conductivity(field="yyyyy", current="x", mu=-5.5)
c = h.get_nonlinear_drude_components(5, nk=48, mu=-5.5)  # {"x^l1 y^l2;b": value}
```

What comes back from `h.get_nonlinear_drude_orders()` is, for each order l up to l_{max} , the largest spin conductivity over every component of that order: zero to machine precision for l from 0 to 4 and nonzero at $l = 5$, which is the i-wave fingerprint. `field` carries one character per power of the electric field, so `field="yyyyy"` is the fifth-order response to E_y ; `current` is the direction the current is measured in, and `channel` picks "spin", "charge", "up" or "dn". Units are $e = \hbar = 1$, with the zone integral normalized as a density, the same convention `fermi_volume` uses.

The X-wave models themselves are `specialhamiltonian.xwave_magnet(wave=...)`, with `pwave_magnet`, `dwave_altermagnet`, `fwave_magnet`, `gwave_altermagnet` and `iwave_altermagnet` as named aliases. These are Ezawa's models, each on the lattice its symmetry requires: square for p, d and g, triangular for f and i.

Before quoting a result, there are a few things to keep in mind. First, the orders above the threshold do not vanish: on a lattice the form factor carries higher harmonics beyond the leading one, so a d-wave altermagnet responds at $l = 1$ but also at $l = 3$ and $l = 5$, and it is the absence of everything below the threshold that carries the information, not the presence of a single isolated order. Second, an insulator gives zero at every order, which is a trap: with the chemical potential in a gap every valence band is full and every conduction band empty, the zone integral vanishes identically at every order including the fifth, and a gapped system reproduces the “nothing below fifth order” pattern trivially. The fingerprint only identifies i-wave order if the fifth order is also shown to be present, and that needs a metal. Third, a spin-degenerate state is flagged rather than answered: the spin channel only means something if the two spin channels have different bands, and a compensated Neel state on a bipartite lattice is PT symmetric and exactly spin degenerate. That is an antiferromagnet rather than an altermagnet, and what comes back there is noise that does not look small, so a warning is raised whenever the splitting falls below 10^{-10} of the bandwidth; a real altermagnet stays far above that floor.

Reality is itself a selection rule, and a useful one when building a model. With real hoppings and no spin-orbit coupling $\epsilon_s(k) = \epsilon_s(-k)$, so the splitting is even under $\phi \rightarrow \phi + 180^\circ$ and every odd harmonic is forbidden. A C3 axis then removes everything not divisible by three, leaving only the harmonics of order 6, 12, ..., that is, $l = 5, 11, \dots$, meaning that a real, spin-orbit-free, C3-symmetric cell is necessarily i-wave, and no vacancy pattern or supercell can make it f-wave. The models here bear this out: p-wave and f-wave need imaginary hoppings, while d-, g- and i-wave are real.

Finally, spin must be a good quantum number. The formula treats each spin channel on its own, so a Hamiltonian with spin-orbit coupling is rejected rather than mistreated: with Rashba or Kane-Mele terms the nonlinear response picks up quantum-metric and Berry-curvature-dipole contributions (arXiv:2409.09241) that this formula does not contain.

Multiorbital cells (superlattices, antidot lattices, multilayers) are supported, and are the interesting case. The band derivatives there are taken analytically, including at the degeneracies that a C3 or C6 axis forces at high-symmetry points, so an order the selection rule forbids comes out as an exact zero rather than as small numerical noise.

See `examples/2d/xwave_nonlinear_spin_current/main.py` for a runnable version printing the whole selection-rule table.

Spin operators

The simplest operators are the spin operators

$$S_\alpha = \sum_n \sigma_\alpha^{\mu\nu} c_{n,\mu}^\dagger c_{n,\nu}$$

with σ_α the Pauli matrices, that can be obtained as

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
sx = h.get_operator("sx") # Spin x component
sy = h.get_operator("sy") # Spin y component
sz = h.get_operator("sz") # Spin z component
```

Passing one of them to `h.get_bands(operator=sz)` colors each band by its spin projection, which is how a spin-split band structure, the one of the Zeeman field in the first chapter, is told apart from a spin-degenerate one; the three names also work directly as strings, `operator="sz"`. The same operators resolve the density of states, the Fermi surface and the momentum-resolved spectral function by spin, see `examples/2d/operator_fermi_surface/main.py` for a spin-resolved Fermi surface and the executed notebook `jupyter-notebooks/functionalities/spectral_functions/06_operator` for a spin-resolved spectral function.

Listing the accepted names

Every routine that takes an `operator` argument (`h.get_bands()`, `h.get_dos()`, `h.get_ldos()`, `h.get_kdos_bands()`, `h.get_vev()`, the topological invariants) accepts an `Operator`, a `Hamiltonian`, a `Potential`, a callable of the position, a bare matrix, or the name of one of the operators pyqula knows how to build. Named operators are not the only thing selected by a string: so are the mean-field initialization (`mf=`), the superconducting pairing symmetry (`mode=`), the high-symmetry k-point labels of a band path, and the quantities `h.extract()` pulls out of a `Hamiltonian`. If you do not remember a name, each accepted set can be listed

```
from pyqula import operatorlist, meanfield, extract
from pyqula.sctk import pairing
from pyqula.kpointstk import labels

print(operatorlist.get_operator_names()) # every name h.get_operator accepts
print(meanfield.get_guess_names())      # every mf= mean-field initialization
print(pairing.get_pairing_modes())      # every h.add_pairing(mode=...) symmetry
print(labels.get_label_names())         # every high-symmetry kpoint label
print(extract.get_extractable_names()) # every h.extract(...) quantity
```

A name outside those lists raises a `ValueError` quoting both the offending name and the accepted ones, rather than failing somewhere downstream.

Location operator

To understand the spatial location of the states we can use the spatial operators, that denote where wavefunctions are located in real space

$$R_\alpha = \sum_{r,s} r_\alpha c_{r,s}^\dagger c_{r,s}$$

with r_α the component of the position of site r

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
x = h.get_operator("xposition") # x component
y = h.get_operator("yposition") # y component
z = h.get_operator("zposition") # z component
```

The expectation value of y in each state of a ribbon says where across the width that state lives, so `h.get_bands(operator=y)` separates the states at the two edges from the bulk ones by their position, the continuous version of the bulk-edge operator that follows.

Bulk-edge operator

In order to know if a state is located at the edge or in the bulk of the system you can use the bulk-edge location operators. The edge operator takes value 1 for sites on the edge, and 0 for sites in the bulk.

$$\hat{E} = \sum_{r \in \text{Edge}, s} c_{r,s}^\dagger c_{r,s}$$

The bulk operator takes value 1 for sites in the bulk, and 0 for sites on the edge.

$$\hat{B} = \sum_{r \in \text{Bulk}, s} c_{r,s}^\dagger c_{r,s}$$

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
b = h.get_operator("bulk") # bulk operator
e = h.get_operator("edge") # edge operator
```

The two are defined by position rather than by connectivity: for a ribbon the sites in the outer tenth of the width on each side count as edge, for a finite island the sites at the outer part of the radius, and for a two-dimensional multilayer the outermost layers along z . In the zigzag ribbon above, `h.get_bands(operator=e)` gives an edge weight close to 1 for the flat band at zero energy, the zigzag edge states, and a small one for the dispersive

bulk bands, so the operator is a quick way to tell in-gap edge states from bulk states in a band structure.

Valley operator

For honeycomb-like systems, including aligned and twisted multilayers, we can extract the valley degree of freedom of each state with the valley operator, `h.get_operator("valley")`. This operator takes the form

$$V = i \sum_{\langle\langle ij \rangle\rangle, s} \nu_{ij} \sigma_{ij} c_{r_i, s}^\dagger c_{r_j, s}$$

where $\nu = \pm 1$ and $\sigma = \pm 1$ for clockwise/anticlockwise, sublattice A/B. This is the so-called anti-Haldane hopping, and takes opposite values in opposite valleys. It can be obtained for honeycomb systems as

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
vall = h.get_operator("valley") # valley operator
```

Its expectation value has opposite signs for the states around K and around K' , so the band structure computed with `h.get_bands(operator=vall)` is colored by valley, with the two valleys at opposite ends of the color scale. This is most useful in a supercell, where the folded bands of the two valleys land on top of each other and the valley operator is what tells them apart. See `examples/1d/valley_operator/main.py` for the valley-resolved bands of a zigzag ribbon, `examples/2d/valley_bands/main.py` for those of a honeycomb supercell, and the executed notebook `jupyter-notebooks/functionalities/single_particle_hamiltonian`

In-plane valley operators

The operator above is the out-of-plane valley pseudospin τ_z . We will now see how to obtain the two remaining components, τ_x and τ_y , with `h.get_operator("valley_x")` and `h.get_operator("valley_y")`, giving access to the full valley vector (τ_x, τ_y, τ_z) , the valley-space analogue of (S_x, S_y, S_z) for real spin. They are built from a chiral Kekule coupling, symmetrized over the three inequivalent Kekule registries so that the result is exactly C_3 -covariant about every atom and not only about special high-symmetry points, rather than from the second-neighbor coupling behind τ_z

```
from pyqula import geometry
g = geometry.honeycomb_lattice().supercell(3) # Kekule-commensurate cell
h = g.get_hamiltonian(has_spin=False) # get the Hamiltonian
taux = h.get_operator("valley_x") # tau_x
tauy = h.get_operator("valley_y") # tau_y
```

Both require a honeycomb-like geometry with a sublattice index. For a periodic Hamiltonian they additionally require a Kekule-commensurate cell (a 3x3, or any other multiple-of-3, supercell of the primitive honeycomb cell) to be well defined, which is the reason for the `supercell(3)` above; a finite (0d) flake needs no such commensurability.

A single vacancy in an otherwise pristine honeycomb flake is an atomically sharp, intervalley-scattering defect, and induces a vortex in the in-plane valley pseudospin around it, a nice way to see τ_x and τ_y in action

```
from pyqula import islands
from pyqula import spectrum
g = islands.get_geometry(name="honeycomb",n=8,nedges=6)
gv = g.remove(g.get_central()[0]) # flake with a single vacancy
hv = gv.get_hamiltonian(has_spin=False)
dvx = spectrum.real_space_vev(hv,operator=hv.get_operator("valley_x"))
dvy = spectrum.real_space_vev(hv,operator=hv.get_operator("valley_y"))
```

What comes back is the expectation value of each component on every site of the flake, and plotting (τ_x, τ_y) as an arrow on each site shows the in-plane valley pseudospin winding around the vacancy.

The valley vector can also be coupled to, not only measured: `h.add_valley_exchange(v)`, with `v=(vx,vy,vz)`, adds a valley-space exchange term $\vec{v} \cdot (\tau_x, \tau_y, \tau_z)$ to the Hamiltonian, the valley-pseudospin analogue of `h.add_exchange()` for real spin

```
from pyqula import geometry
g = geometry.honeycomb_lattice().supercell(3) # Kekule-commensurate cell
h = g.get_hamiltonian(has_spin=False)
h.add_valley_exchange([0.1,0.05,0.2]) # (vx,vy,vz)
```

See `examples/0d/valley_vortex_vacancy/main.py` and `examples/2d/valley_vortex/main.py` for runnable versions of the vacancy-vortex example.

Nambu operators

In the presence of superconductivity, you can project onto the electron or hole component of the Nambu spinor using the electron-hole operators. The Hamiltonian must already be in the Nambu (BdG) basis, meaning that some pairing has been added or `h.setup_nambu_spinor()` has been called; otherwise there is no hole sector to project onto, and the hole projector raises an error saying so

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
h.add_swave(0.2) # add pairing, doubling the basis into Nambu space
electron = h.get_operator("electron") # electron component
```

```
hole = h.get_operator("hole") # hole component
(k,e,c) = h.get_bands(operator=electron) # electron weight of each state
```

The electron weight is what separates the physical copy of the band structure from its spurious one: far from the Fermi energy a state is either purely electron, weight 1, or purely hole, weight 0, and only close to the superconducting gap do the two mix, with a weight of one half exactly at the gap edge. This is the same weight that turns the full BdG density of states into the electron spectral function of the superconductivity chapter.

Berry curvature operator

The Berry curvature operator is a first example of an operator that is intrinsically momentum dependent. The Berry curvature operator is defined as

$$O|\Psi_k\rangle = \Omega(k, \epsilon_k)|\Psi_k\rangle$$

where $\Omega(k, \omega)$ is the Berry curvature evaluated at the momentum k and energy ω of the eigenstate $|\Psi\rangle$. In particular, this operator allows to directly see the contribution to the Berry curvature of different states in the band structure. It is obtained as `h.get_operator("berry")`, it is defined for two-dimensional Hamiltonians, and it is evaluated through the Green's function at the energy of each state, with a broadening `delta` that defaults to 0.1; passing it to `h.get_bands(operator="berry")` colors the bands of a Chern insulator by where their Berry curvature sits, and the Brillouin-zone integral of that curvature is the Chern number of the topological insulators chapter.

Inverse participation ratio operator

So far we have considered operators that are linear, namely that fulfill the condition

$$A(|\Psi_1\rangle + |\Psi_2\rangle) = A|\Psi_1\rangle + A|\Psi_2\rangle$$

There is however one operator that it is interesting to consider that does not fulfill such condition. The operator is the so-called inverse participation ratio, which we define as

$$O|\Psi\rangle = \sum_i |\langle i|\Psi\rangle|^4 |\Psi\rangle$$

In particular, the previous operator allows to identify states that are highly localized in a few lattice sites, becoming useful to highlight impurity states and localized modes.

```

from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
h.add_onsite(0.3) # add a sublattice imbalance
ipr = h.get_operator("IPR") # IPR operator

```

Its expectation value is $\sum_i |\langle i | \Psi \rangle|^4$, which is 1 for a state sitting on a single site and $1/N$ for a state spread evenly over N sites, so a large value flags a localized state: in the zigzag ribbon above, `h.get_bands(operator=ipr)` gives the edge states at zero energy an inverse participation ratio more than ten times larger than the one of the bulk bands.

Non-Hermitian Hamiltonians

Up to now every Hamiltonian in this guide has been Hermitian, meaning that its eigenvalues are real energies and its eigenvectors are orthogonal. Let us now see what happens when the system is open, exchanging particles or energy with an environment. Such a system is often described by an effective Hamiltonian that is no longer Hermitian: gain and loss enter as imaginary onsite energies, and a non-reciprocal hopping, with $t_{ij} \neq t_{ji}^*$, as an asymmetric hopping matrix. The eigenvalues are then complex, and what their imaginary parts measure is an amplification or a decay rate rather than an energy, while the eigenvectors are no longer orthogonal. This is the setting of photonic and acoustic lattices with gain and loss, of the non-Hermitian skin effect, and of $\mathcal{PT}$ -symmetric models.

pyqula generates such a Hamiltonian with the `non_hermitian=True` flag of `g.get_hamiltonian()`, after which the usual observables are computed with their non-Hermitian counterparts. Nothing else about building the model changes: an `h.add_onsite()` with a complex-valued function is what puts the gain and loss in. As an example, let us take a chain of twenty sites with a purely imaginary modulation of the onsite energy, gain on the sites where the cosine is positive and loss where it is negative

```

import numpy as np
from pyqula import geometry

n = 20
g = geometry.chain().get_supercell(n)
h = g.get_hamiltonian(has_spin=False, non_hermitian=True, tij=[-1.])
# a purely imaginary Aubry-Andre modulation, with no Hermitian part at
# all: gain where the cosine is positive, loss where it is negative
h.add_onsite(lambda r: 0.6j*np.cos(2.*np.pi*r[0]/n))

(ks, es) = h.get_bands(kpath=[[0., 0., 0.]], write=False)
print("eigenvalues are complex:", es.dtype)
print("largest gain rate Im(E):", np.round(np.max(es.imag), 4))

```

The eigenvalues come back complex rather than real: twenty of them, one per site of the supercell, with real parts inside the band of the chain and imaginary parts between -0.48 and $+0.48$, smaller than the 0.6 of the modulation because the hopping mixes the gain and the loss sites. Anything that plots them has to choose a part: `es.real` for the energy axis, `es.imag` for the gain and decay axis, or both as a scatter in the complex plane.

Choosing which part is the energy: eigmode

`eigmode` is the argument that exists only on this path. It says which part of the complex eigenvalue plays the role of the energy, the quantity a broadening is centred on or a band structure is written out as, and it takes `"complex"` (the default, keep the whole eigenvalue), `"real"` or `"imag"`; anything else raises `ValueError` listing the three.

In `h.get_bands()` it selects what the returned energy row and the written `BANDS.OUT` carry. With the default `"complex"` the file gains one extra column, so that its layout is `k, Re E, Im E`, then one column per operator; the imaginary part is kept rather than silently dropped, since it is the physics the calculation was done for.

In `h.get_ldos()` it decides which axis the requested energy `e` lives on. With `eigmode="imag"` the states are selected by their amplification rate instead of by their energy, which is how one asks where the most amplified mode of the model above actually sits

```
import numpy as np
from pyqula import geometry
n = 20
g = geometry.chain().get_supercell(n)
h = g.get_hamiltonian(has_spin=False,non_hermitian=True,tij=[-1.])
h.add_onsite(lambda r: 0.6j*np.cos(2.*np.pi*r[0]/n))
(ks,es) = h.get_bands(kpath=[[0.,0.,0.]],write=False)

imax = np.argmax(es.imag) # the most amplified state
(x,y,d) = h.get_ldos(e=es[imax].imag,delta=1e-2,eigmode="imag",nrep=1)
print("it lives at x =",x[np.argmax(d)])
```

which returns the site where the gain is largest, $x = -0.5$ here, the maximum of the modulation above: the most amplified state lives where the imaginary onsite energy is largest, as one would expect.

`h.get_dos()` broadens the real part whatever `eigmode` says, so that `"complex"` and `"real"` give the same density of states there and `"imag"` gives the distribution of decay rates instead.

What is and is not available

The band structure, the density of states, the LDOS and the Berry curvature have genuine non-Hermitian versions. Everything else on a `non_hermitian=True` Hamiltonian falls back to the Hermitian formula, which may or may not be meaningful for a complex spectrum, so check before relying on it. The self-consistent mean field is one of those: it runs the ordinary self-consistent loop, and the density matrix it builds is the Hermitian one (see [jupyter-notebooks/functionalities/interacting_mean_field_hamiltonians/08_hermitian_nonhermit](#)).

One restriction is enforced rather than left to the caller: `h.get_dos()` accepts only `mode="ED"`, and refuses `use_kpm=True` and every other mode with a `NotImplementedError`, because the Chebyshev and adaptive expansions both assume a real spectrum. `h.get_ldos()` has the same limitation, only its default diagonalization mode (`mode="arpack"`) is implemented here, but does not enforce it, so pass no other mode.

Operators work as usual, including `operator="unfold"`, so that a supercell calculation can be unfolded back onto the primitive Brillouin zone with a complex spectrum; see `examples/1d/unfolding_non_hermitian/main.py`. `num_bands` also works, selecting the few eigenvalues nearest `central_energy` rather than diagonalizing fully.

See `examples/1d/NH_ldos/main.py` (the model above, resolved mode by mode), `examples/0d/non_hermitian_aah/main.py` and `examples/0d/non_hermitian_aah_dos/main.py` (a non-Hermitian Aubry-Andre chain swept over the modulation phase) for runnable versions.

Superconductivity

Up to now we have focused on Hamiltonians that contain only normal terms, namely that the full Hamiltonian can be written as

$$H_0 = \sum_{ijss'} t_{ijss'} c_{i,s}^\dagger c_{j,s'}$$

where ij run over the sites and ss' over the spins, so that every term destroys one electron and creates one, and the number of electrons is conserved.

In the presence of superconductivity, an anomalous term appears in the Hamiltonian taking the form

$$H_{SC} = \sum_{ijss'} \Delta_{ij}^{ss'} c_{i,s} c_{j,s'} + h.c.$$

meaning that pairs of electrons are destroyed and created, so that the number of electrons is no longer conserved; the pairing amplitude $\Delta_{ij}^{ss'}$ is what is called

the superconducting order parameter, and it is what we will put in by hand in this chapter and obtain self-consistently from an attractive interaction in the mean-field chapter.

To solve the Hamiltonian

$$H = H_0 + H_{SC}$$

we define a Nambu spinor that takes the form

$$\Psi_n = \begin{pmatrix} c_{n,\uparrow} \\ c_{n,\downarrow} \\ c_{n,\downarrow}^\dagger \\ -c_{n,\uparrow}^\dagger \end{pmatrix}$$

You can think of the Nambu spinor as the two electron operators of site n stacked on top of a copy of their conjugates, so that a term creating two electrons becomes a hopping between the upper and the lower half of the spinor, and the problem is again a single-particle one. We then rewrite the Hamiltonian as

$$H = \Psi^\dagger \mathcal{H} \Psi$$

where $\mathcal{H}$ is the Nambu Hamiltonian, also called the Bogoliubov-de Gennes (BdG) Hamiltonian, a matrix twice as large as the normal one. In this new basis, the Hamiltonian can be written in a diagonal form as

$$H = \sum_{\alpha} \epsilon_{\alpha} \Psi_{\alpha}^{\dagger} \Psi_{\alpha}$$

where ϵ_{α} are the Nambu eigenvalues. Because the lower half of the spinor is the conjugate of the upper one, the eigenvalues come in pairs $\pm\epsilon_{\alpha}$, an electron copy and a hole copy of every state, and the gap of the spectrum around zero energy is twice the smallest $|\epsilon_{\alpha}|$. In pyqula the doubled basis is set up by `h.setup_nambu_spinor()`, and the methods that add a pairing, `h.add_swave()` and `h.add_pairing()`, set it up on their own when it is not there yet; every observable of the guide, the band structure, the density of states, the LDOS, is then computed in the Nambu basis. A superconducting mean-field calculation needs the basis before it starts, which is why `h.setup_nambu_spinor()` is called explicitly in the mean-field chapter.

s-wave superconductivity

The simplest form of superconductivity is spin-singlet s-wave superconductivity. A minimal superconducting term of this form can be written as

$$H_{SC} = \Delta_0 \sum_n c_{n,\uparrow} c_{n,\downarrow} + h.c.$$

meaning that an electron with spin up and an electron with spin down on the same site are paired, with the same amplitude Δ_0 on every site; this is the pairing of a conventional superconductor.

In the following, we address the electronic structure of a triangular lattice with s-wave superconductivity, whose Hamiltonian takes the form

$$H = H_0 + H_{SC}$$

with

$$H_0 = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c.$$

The previous Hamiltonian can be computed for $\Delta_0 = 0.2$ with `h.add_swave()` as

```
from pyqula import geometry
g = geometry.triangular_lattice() # geometry of the 2D model
h = g.get_hamiltonian() # generate the Hamiltonian
h.add_swave(0.2) # add s-wave superconductivity
(k,e) = h.get_bands() # compute band structure
```

Note that due to the BdG nature of the Hamiltonian, the band structure shows both the electron and the hole states. What you see is a gap of $2\Delta_0$ opening at zero energy along the whole Fermi surface of the triangular lattice, regardless of where the Fermi surface is: an electron at the Fermi energy pays Δ_0 to enter the superconductor whatever its momentum, which is the hallmark of s-wave pairing. Coloring the bands with `operator="electron"` in `h.get_bands()` tells the electron copy from the hole copy, which is how the spectral function of the electrons alone is read off a BdG band structure.

See `examples/2d/comparison_swave/main.py` for a runnable version (the same gap put in with `h.add_swave()` and with `h.add_pairing(mode="swave")`, bands colored by their electron weight), and the notebooks `jupyter-notebooks/functionalities/single_particle_hamiltonians/03_magnetism_soc_supercon` for the Nambu basis and for superconductivity combined with magnetism and spin-orbit coupling. A pairing that arises from an attractive interaction rather than being put in by hand is the subject of the “Superconducting mean-field” section.

Spin-triplet d-vector and non-unitary superconductivity

In the s-wave case above the two paired electrons form a spin singlet. Let us now address the other possibility, a pair in a spin triplet, and see how pyqula

characterizes it with `h.get_dvector_non_unitarity()`. For a spin-triplet superconductor the pairing is a symmetric 2×2 matrix in spin space, and the way it is parametrized is by a complex three-component d-vector $\vec{d}$ as

$$\Delta = i(\vec{d} \cdot \vec{\sigma})\sigma_y = \begin{pmatrix} -d_x + id_y & d_z \\ d_z & d_x + id_y \end{pmatrix}$$

so that d_z is the opposite-spin component $\Delta_{\uparrow\downarrow}$ and d_x, d_y encode the equal-spin ones, $\Delta_{\uparrow\uparrow}$ and $\Delta_{\downarrow\downarrow}$. Only $\Delta_{\uparrow\downarrow}$ can also host a spin-singlet contribution, which is antisymmetric under exchanging the two sites; pyqula projects it out before reading off d_z , so that a state with both singlet and triplet order still gives the triplet d-vector alone.

What separates two classes of triplet states is the product

$$\Delta\Delta^\dagger = |\vec{d}|^2 \mathbb{1} + \vec{q} \cdot \vec{\sigma}, \quad \vec{q} = i(\vec{d} \times \vec{d}^*)$$

When $\vec{q} = 0$ the state is **unitary**: $\Delta\Delta^\dagger$ is proportional to the identity, the two quasiparticle branches are degenerate, and the Cooper pairs carry no net spin. When $\vec{q} \neq 0$ the state is **non-unitary**: the gap is different for the two spin branches, and the pairs carry a spin moment $\vec{q}$, which is real by construction (the cross product of a vector with its own conjugate is purely imaginary). A $\vec{d}$ whose components share a common phase, a real vector times a global phase, is always unitary; non-unitarity requires a relative phase between components, as in $\vec{d} \propto (1, i, 0)$, which is a pure $\Delta_{\uparrow\uparrow}$ pairing with $\vec{q}$ along $+z$. Non-unitary states therefore require broken time-reversal symmetry, and they appear naturally in ferromagnetic superconductors, where $\vec{q}$ is parallel to the magnetization because the pairs form in the majority band.

`h.get_dvector_non_unitarity()` returns $\vec{q}$ per site, as an array of shape (number of sites, 3), with $\vec{d}$ evaluated on a uniform k-mesh, $\vec{q}$ averaged over that mesh and the pairing partners of each site summed over. As an illustration, let us put in by hand a p-wave pairing with $\vec{d} \propto (1, i, 0)$, which is non-unitary with the pair spin along $+z$

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D model
h = g.get_hamiltonian() # generate the Hamiltonian
h.setup_nambu_spinor() # initialize the Nambu basis
h.add_pairing(delta=0.3, mode="pwave", d=[1., 1j, 0.]) # up-up pairing
q = h.get_dvector_non_unitarity(nk=20) # -> [[0., 0., 0.36]]
```

The array has one row, the single site of the chain, and only its z component is nonzero, as the relative phase of i between d_x and d_y dictates; the same p-wave pairing with a real d-vector, $d=[1., 0., 0.]$, is unitary and returns zero. Non-unitary states also arise on their own from a self-consistent calculation: adding attractive first-neighbor interactions on top of a large Zeeman splitting, as in

the example of the “Long range interactions” section, gives rise to a spin-triplet order whose $\vec{q}$ follows the magnetization

```
from pyqula import geometry
g = geometry.triangular_lattice() # generate the geometry
h = g.get_hamiltonian() # create Hamiltonian of the system
h.add_exchange([0.,0.,3.]) # add exchange field
h.setup_nambu_spinor() # initialize the Nambu basis
# perform a superconducting non-collinear mean-field calculation
h = h.get_mean_field_hamiltonian(V1=-1.0,filling=0.3,mf="random",nk=4)
q = h.get_dvector_non_unitarity() # antiparallel to +z, as is h.get_magnetization()
```

For an inhomogeneous system, `h.write_non_unitarity()` writes the same quantity as a real-space map in `NON_UNITARITY_MAP.OUT`, with columns (x, y, z, q_x, q_y, q_z) . If all that is needed is whether the state is non-unitary at all, and not in which direction, `h.get_average_dvector(non_unitarity=True)` returns the k-averaged squared components of $\vec{q}$, a magnitude that carries no sign; the same method with its default `non_unitarity=False` returns $(|d_x|^2, |d_y|^2, |d_z|^2)$, and is what `superconductivity.identify_superconductivity` uses to label a state.

See `examples/1d/dvector_map/main.py` (a self-consistent triplet state along a spin spiral, with its non-unitarity map) and `examples/2d/SC_symmetry/main.py` (a mixed singlet and triplet pairing split into its two parts) for runnable versions, and `jupyter-notebooks/functionalities/interacting_mean_field_hamiltonians/07_order_parameter` for how a self-consistent order parameter is identified.

Superfluid weight and BKT temperature

A finite pairing amplitude does not by itself make a superconductor: what carries the supercurrent is the rigidity of the state against winding the phase of the order parameter, and we will now see how to compute it with `h.get_superfluid_weight()`. This rigidity is the *superfluid weight*, also called the superfluid stiffness,

$$D_s^{ab} = \frac{1}{V} \frac{\partial^2 \Omega}{\partial Q_a \partial Q_b}$$

the second derivative of the grand potential with respect to the twist wavevector $\mathbf{Q}$ of the phase, at fixed $|\Delta|$. What is computed is the general multiband BdG expression of Liang and coworkers, Phys. Rev. B **95**, 024515.

The superfluid weight can also be split into two parts. The *conventional* part comes from the band velocities, and the *quantum-geometric* part from interband matrix elements. For an isolated flat band the second reduces to the integral of the quantum metric (Peotta and Törmä, Nat. Commun. **6**, 8944): a flat band has no velocity, so all of its stiffness is geometric, which is why a flat band can

superconduct at all. The split is offered on top of the full result rather than used as the definition, and it is refused with a `ValueError` when its assumptions (uniform onsite pairing, time-reversal symmetry, a resolvable normal-state gap) do not hold. Let us compute both on a square lattice away from half filling

```
from pyqula import geometry
g = geometry.square_lattice() # geometry of the 2D model
h = g.get_hamiltonian() # generate the Hamiltonian
h.add_onsite(-0.6) # move away from half filling
h.add_swave(0.3) # BdG Hamiltonian with an s-wave gap

D = h.get_superfluid_weight(nk=20) # Cartesian (dim,dim) tensor
out = h.get_superfluid_weight(nk=20,decompose=True)
print(out["total"],out["conventional"],out["geometric"])
print(h.get_bkt_temperature(nk=20)) # Nelson-Kosterlitz criterion
```

D is the Cartesian tensor, isotropic on the square lattice, and the decomposition returns a dictionary whose "geometric" entry vanishes here: a single-band model has no other band to have matrix elements with, so all of its stiffness is conventional, and the geometric part only appears in a multiband model, a flat band above all.

The twist uses the full bond vector $\mathbf{R} + \mathbf{r}_j - \mathbf{r}_i$ rather than the lattice vector $\mathbf{R}$ alone, and this matters. With the lattice vector alone D_s comes out anisotropic on the honeycomb lattice, which C3 symmetry forbids, and it changes when the same crystal is described with a supercell. The full bond vector gives an isotropic, supercell-invariant answer. The two agree only for a cell with one orbital. `gauge="lattice"` selects the other convention, the one used by Peotta and Törmä and by `h.get_quantum_metric()`: at fixed $|\Delta|$ the superfluid weight genuinely depends on where the orbitals sit, see Huhtinen, Herzog-Arbeitman, Chew, Bernevig and Törmä, Phys. Rev. B **106**, 014518.

In two dimensions `h.get_bkt_temperature()` solves the Nelson-Kosterlitz criterion $T_{\text{BKT}} = (\pi/8)D_s(T_{\text{BKT}})$ self-consistently by bisection, at frozen $|\Delta|$ (there is no $\Delta(T)$ feedback, so it is an upper estimate); for the square lattice above it comes out at 0.15 in units of the hopping. Setting `mode="finite_difference"` differentiates the grand potential numerically instead: much slower, but assumption-free. One caveat to know in advance: at $T = 0$ with a *gapless normal state* and zero or tiny pairing, the paramagnetic and diamagnetic contributions cancel through a $-\partial f/\partial E$ that collapses to a delta function, which a finite k-mesh cannot resolve, so D_s comes out at the normal state's Drude weight rather than zero; use a temperature the mesh resolves when checking that a marginal state has no stiffness. See `examples/2d/superfluid_weight/main.py` for a runnable version.

Interactions at the mean-field level

Up to now every Hamiltonian in this guide has been a single-particle one, meaning that all its terms are bilinear, a creation and an annihilation operator. We will now see how electronic interactions, quartic terms with four fermionic operators, can be treated at the mean-field level: the interaction is replaced by a bilinear term that depends on expectation values, and since those expectation values depend in turn on the Hamiltonian, the problem is solved self-consistently. This is an uncontrolled approximation, but it is the practical workhorse for magnetism, charge order and superconductivity driven by interactions, and everything in this chapter goes through one method, `h.get_mean_field_hamiltonian()`, which returns a new Hamiltonian with the converged mean-field terms in it. We start with the Hubbard model, first collinear and then non-collinear, then let the same interaction produce superconductivity, then add interactions between neighbors, and finally treat spin-spin exchange, spin models and the Kondo lattice with the same machinery.

The collinear Hubbard model

We will start with the simplest interaction term, a local repulsive interaction in a spinful system. Our full Hamiltonian takes the form

$$H = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c. + U \sum_i c_{i,\uparrow}^\dagger c_{i,\uparrow} c_{i,\downarrow}^\dagger c_{i,\downarrow}$$

where the first term is the first-neighbor hopping of the previous chapters and U is the energy cost of putting a spin-up and a spin-down electron on the same site. The interaction term $U \sum_i c_{i,\uparrow}^\dagger c_{i,\uparrow} c_{i,\downarrow}^\dagger c_{i,\downarrow}$ is solved at the mean-field level. The mean-field approximation consists in replacing the previous four-fermion operator by all the terms that arise by taking the expectation value of two of the fermions. In particular, in its simplest collinear form, the mean-field term takes the form

$$H_U^{MF} = U \sum_i \langle c_{i,\uparrow}^\dagger c_{i,\uparrow} \rangle c_{i,\downarrow}^\dagger c_{i,\downarrow} + c_{i,\uparrow}^\dagger c_{i,\uparrow} \langle c_{i,\downarrow}^\dagger c_{i,\downarrow} \rangle$$

where $\langle \rangle$ denotes the ground state expectation value of those operators, meaning that each spin-down electron feels an onsite energy set by the density of spin-up electrons on its site, and the other way around. The full Hamiltonian thus takes the form

$$H^{MF} = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c. + U \sum_i \langle c_{i,\uparrow}^\dagger c_{i,\uparrow} \rangle c_{i,\downarrow}^\dagger c_{i,\downarrow} + c_{i,\uparrow}^\dagger c_{i,\uparrow} \langle c_{i,\downarrow}^\dagger c_{i,\downarrow} \rangle$$

As a result, the mean-field Hamiltonian depends on the specific ground state of the system, and the ground state depends of course on the specific mean-field Hamiltonian. The previous circular dependence between the ground state and the mean-field Hamiltonian gives rise to a self-consistent problem.

This self-consistent condition is solved as follows. We start with an initial guess for the full many-body ground state, that we call $|GS_0\rangle$. With this initial state, we compute the mean-field Hamiltonian H_0^{MF} . This mean-field Hamiltonian allows us to compute a new many-body ground state $|GS_1\rangle$, which in turn allows us to compute a new mean-field Hamiltonian H_1^{MF} . The previous algorithm is represented as

$$|GS_0\rangle \rightarrow H_0^{MF} \rightarrow |GS_1\rangle \rightarrow H_1^{MF} \rightarrow |GS_2\rangle \rightarrow H_2^{MF} \rightarrow \dots$$

This iterative calculation is performed until $H_n^{MF} = H_{n+1}^{MF}$, at which point the algorithm has converged. In practice the loop stops when two successive mean fields differ by less than a tolerance, `maxerror`, and nothing in the Hamiltonian is symmetric anymore: the mean-field terms are what a symmetry-broken state looks like at the single-particle level, an onsite energy that differs between spins, or between sites, that the original Hamiltonian did not have.

Two important notes can be taken from the previous approach. First, the final solution may be sensitive to the initial guess for the ground state. This guess corresponds to the initialization of the Hamiltonian, and it can be important for systems whose energy landscape has several local minima. A second point is that the update procedure from one iteration to the next can be done adiabatically, or very suddenly. This corresponds to the mixing between solutions, and for systems close to the critical point can lead to tricky convergence.

Let us now show an example of a mean-field calculation, with `h.get_mean_field_hamiltonian()`. We will take now a square lattice, make a 2x2 supercell and include local repulsive interactions at half filling. The obtained ground state is an antiferromagnetic Neel state that opens a gap at half filling

```
from pyqula import geometry
g = geometry.square_lattice() # geometry of a square lattice
g = g.get_supercell([2,2]) # generate a 2x2 supercell
h = g.get_hamiltonian() # create hamiltonian of the system
h = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,
                                mf="random") # perform SCF
(k,e) = h.get_bands() # calculate band structure
m = h.get_magnetization() # get the magnetization
```

The result is a new Hamiltonian with the converged mean-field terms in it, and the original `h` is left as it was. What you see in its band structure is a gap at zero energy, of about 0.8 for $U = 2$, where the non-interacting square lattice at half filling is a metal; the magnetization is one vector per site, of equal magnitude and alternating sign between the two sublattices of the supercell, along a

direction that the random initial guess picked, since nothing in the Hamiltonian selects one. The supercell is needed for the same reason: a Neel state has two inequivalent sites, and a single-site cell could only hold a ferromagnet. The guess `mf=` is one of the names listed by the library ("`random`", "`antiferro`", "`ferro`", "`swave`" and others) or a Hamiltonian whose mean-field terms seed the calculation; `filling` is the keyword of "Setting a filling", enforced at every iteration; and the calculation returns `None` instead of a Hamiltonian when it does not converge, which is discussed at the end of the section on spin-spin exchange together with the parameters that decide it. Increasing U makes the moments larger and the gap wider, up to a gap that grows linearly with U in the strongly interacting limit, where each site holds one localized electron.

See `examples/2d/hubbard_honeycomb/main.py` for the gap as a function of U on the honeycomb lattice, where the density of states vanishes at the Dirac point and the gap only opens above a critical interaction, and `jupyter-notebooks/functionalities/interacting_mean_field_hamiltonians/02_collinear_noncollinear` for the executed version.

Non-collinear Hubbard model

Let us now see what happens when the magnetization is not restricted to point along one axis. In the mean-field ansatz considered above, only a single term in the Wick contraction was considered. This term is the collinear term in the z-direction, and allows accounting for solutions that have magnetization in the z-direction. However, in the presence of frustration, external magnetic field or spin-orbit coupling, the magnetization of a system may be non-collinear and pointing in an arbitrary direction. To account for that phenomenology, the mean-field Hamiltonian must include the non-collinear term that takes the form

$$H_U^{ncMF} = -U \sum_i \langle c_{i,\downarrow}^\dagger c_{i,\uparrow} \rangle c_{i,\uparrow}^\dagger c_{i,\downarrow} + h.c.$$

which is a spin-flip term on each site, set by the expectation value of the in-plane magnetization. When including this additional term, the full mean-field Hubbard Hamiltonian is rotationally invariant, meaning that it respects $SO(3)$ spin rotational symmetry. This rotationally symmetric form is the default form implemented in the library, which is also why the direction of the Neel moments in the previous section was left to the initial guess.

With the previous point in mind, we now solve a system that develops a non-collinear magnetic state. We take the square lattice considered in the section above, and we add an external magnetic field. The competition between Zeeman energy and antiferromagnetic correlations gives rise to a canted magnetic state

```
from pyqula import geometry
g = geometry.square_lattice() # geometry of a square lattice
g = g.get_supercell([2,2]) # generate a 2x2 supercell
```

```

h = g.get_hamiltonian() # create hamiltonian of the system
h.add_zeeman([0.,0.,0.1]) # add out-of-plane Zeeman field
h = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,
                                mf="random") # perform SCF
(k,e,c) = h.get_bands(operator="sz") # calculate band structure
m = h.get_magnetization() # get the magnetization

```

The magnetization now has two parts: an antiferromagnetic component, alternating between the sublattices, that lies in the plane perpendicular to the field, and a uniform component along the field, the same on every site. The moments cant toward the field rather than flipping, which is what an antiferromagnet does in a field that is weak compared to its exchange, and a larger field tilts them further. The third array returned by `h.get_bands(operator="sz")` is the spin projection of each state, so that the bands can be colored by it and the uniform component read off from the imbalance between up and down.

See `examples/readme_examples/scf_square/main.py` for this calculation ending in a plot of the bands and of the moments, and the notebook `02_collinear_noncollinear.ipynb` in the folder above.

Superconducting mean-field

In the cases above we focused on local repulsive interactions that promote collinear or non-collinear magnetism. However, local interactions can promote a different type of symmetry breaking, in particular gauge symmetry breaking associated to superconductivity. The emergence of superconductivity is associated to one of the Wick contractions of the mean-field, the anomalous term, that takes the form

$$H_U^{aMF} = U \sum_i \langle c_{i,\uparrow} c_{i,\downarrow} \rangle c_{i,\downarrow}^\dagger c_{i,\uparrow}^\dagger + h.c.$$

where the expectation value of two annihilation operators, the pairing amplitude, is the Δ_0 of the s-wave term in the chapter on superconductivity, now obtained from the interaction rather than put in by hand. The previous term in the mean-field Hamiltonian can become non-zero for $U < 0$, and yields an interaction induced superconducting state. This term in the mean-field Hamiltonian is automatically accounted for in Hamiltonians with a Nambu degree of freedom, of course apart from the collinear and non-collinear terms in the mean-field. We show below how an interaction induced superconducting state can be computed with pyqula, with `h.setup_nambu_spinor()` followed by `h.get_mean_field_hamiltonian()`

```

from pyqula import geometry
import numpy as np
g = geometry.triangular_lattice() # geometry of a triangular lattice
h = g.get_hamiltonian() # get the Hamiltonian

```

```

h.setup_nambu_spinor() # setup the Nambu form of the Hamiltonian
h = h.get_mean_field_hamiltonian(U=-1.0,filling=
                                0.15,mf="swave") # perform SCF
# electron spectral-function
h.get_kdos_bands(operator="electron",nk=400,
                 energies=np.linspace(-1.0,1.0,100))

```

The Nambu spinor has to be set up before the calculation, so that the anomalous contraction has a block of the Hamiltonian to go into; the guess `mf="swave"` seeds a uniform singlet pairing, which the loop then adjusts to its self-consistent value. The electron spectral function, projected with `operator="electron"` on the electron half of the Nambu spinor so that the hole copy of the bands does not appear, shows the normal-state band of the triangular lattice with a gap opened at the Fermi energy, and the gap grows with $|U|$. `h.get_kdos_bands()` is described in the section on momentum-resolved spectral functions.

See `examples/readme_examples/scf_SC/main.py` for a runnable version ending in a plot, and the notebook `04_anomalous_mf_sc.ipynb` in the folder above.

Long range interactions

Up to now we have considered interacting Hamiltonians that only have local (attractive or repulsive) Hubbard interactions. In the following we are going to consider systems that have many-body interactions also to a certain number of neighbors. Long range interactions are crucial to stabilize specific symmetry broken states, and in particular charge density waves, Peierls instabilities and unconventional superconductivity. The full Hamiltonian we will consider takes the form

$$H = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c. + U \sum_i c_{i,\uparrow}^\dagger c_{i,\uparrow} c_{i,\downarrow}^\dagger c_{i,\downarrow} + V_1 \sum_{\langle ij \rangle, s, s'} c_{i,s}^\dagger c_{i,s} c_{j,s'}^\dagger c_{j,s'}$$

where U parametrizes onsite interactions and V_1 interactions between first neighbors; second and third neighbors enter `h.get_mean_field_hamiltonian()` in the same way as V_2 and V_3 , and an interaction with an arbitrary dependence on the distance as $V_{\mathbf{r}}$. The previous Hamiltonian gives rise to a variety of terms when performing a mean-field decoupling. By default, `pyqula` includes all the Wick contractions of the mean-field, and in the presence of Nambu spinors it includes all the anomalous contractions. Let us now briefly elaborate on some of the additional terms that arise due to the first-neighbor interaction V_1 .

The first term is the charge order term, that takes the form

$$H^{MF} \sim \langle c_{i,s}^\dagger c_{i,s} \rangle c_{j,s'}^\dagger c_{j,s'}$$

this term can give rise to a different charge imbalance between different sites, and it leads to charge density wave states.

The second term we consider is the bond order, that takes the form

$$H^{MF} \sim \langle c_{i,s}^\dagger c_{j,s} \rangle c_{j,s}^\dagger c_{i,s} + h.c.$$

which leads to an interaction-enhanced hopping. If this happens in a non-uniform way in the system, the resulting state has a Peierls distortion.

Among the anomalous terms, the mean-field Hamiltonian can generate

$$H^{MF} \sim \langle c_{i,\uparrow}^\dagger c_{j,\uparrow}^\dagger \rangle c_{j,\downarrow} c_{i,\downarrow} + h.c.$$

$$H^{MF} \sim \langle c_{i,\downarrow}^\dagger c_{j,\downarrow}^\dagger \rangle c_{j,\uparrow} c_{i,\uparrow} + h.c.$$

$$H^{MF} \sim \langle c_{i,\uparrow}^\dagger c_{j,\downarrow}^\dagger \rangle c_{j,\downarrow} c_{i,\uparrow} + h.c.$$

where the first two terms correspond to the odd superconducting order, and the third term accounts for both even and odd orders. What this means is that a first-neighbor attraction can pair electrons on neighboring sites, with the same spin or with opposite spins, where the onsite U of the previous section could only pair opposite spins on the same site.

Below, we show an example in which an interaction-induced spin-triplet term is generated. By considering an electronic structure with a large Zeeman splitting and attractive first-neighbor interactions, a state with non-zero $\Delta_{\uparrow\uparrow}$ and $\Delta_{\downarrow\downarrow}$ emerges.

```
import numpy as np
from pyqula import geometry
g = geometry.triangular_lattice() # generate the geometry
h = g.get_hamiltonian() # create Hamiltonian of the system
h.add_exchange([0.,0.,1.]) # add exchange field
h.setup_nambu_spinor() # initialize the Nambu basis
# perform a superconducting non-collinear mean-field calculation
h = h.get_mean_field_hamiltonian(V1=-1.0,
                                filling=0.3,mf="random")
# electron spectral-function
h.get_kdos_bands(operator="electron",nk=400,
                 energies=np.linspace(-2.0,2.0,400))
```

The exchange field splits the two spin bands so far apart that singlet pairing between opposite spins has no states to pair at the Fermi energy, and the attractive V_1 pairs electrons of the same spin on neighboring sites instead: the converged state has an odd, spin-triplet order parameter, and the electron spectral function shows a gap at the Fermi energy in each of the two spin-split bands.

The d-vector of that state, and how far it is from unitary, are the subject of the section on the spin-triplet d-vector in the chapter on superconductivity.

See `examples/readme_examples/scf_SC_triplet/main.py` for this calculation ending in a plot, `examples/2d/meanfield_V1V2V3/main.py` for a charge density wave driven by V_1 , V_2 and V_3 in a spinless system, and the notebook `05_full_wick_noncollinear_sc.ipynb` in the folder above.

Spin-spin exchange interactions

We will now see how a direct coupling between the spins on two sites is treated at the mean-field level, with `h.get_szsz_mean_field_hamiltonian()` and its companions. The interactions considered above are all of density-density type, $c_{i,s}^\dagger c_{i,s} c_{j,s'}^\dagger c_{j,s'}$, meaning that they couple the charge on one site to the charge on another. A magnet made of localized moments is described instead by a coupling $\vec{S}_i \cdot \vec{S}_j$ between the (Pauli) spin operators at two sites,

$$H = J_z \sum_{\langle ij \rangle} S_i^z S_j^z + J_x \sum_{\langle ij \rangle} S_i^x S_j^x + J_y \sum_{\langle ij \rangle} S_i^y S_j^y$$

with $J > 0$ the antiferromagnetic (Heisenberg) sign convention and $J < 0$ favoring a ferromagnetic instability. Writing $S_i^z = (n_{i,\uparrow} - n_{i,\downarrow})/2$,

$$S_i^z S_j^z = \frac{1}{4} (n_{i,\uparrow} n_{j,\uparrow} - n_{i,\uparrow} n_{j,\downarrow} - n_{i,\downarrow} n_{j,\uparrow} + n_{i,\downarrow} n_{j,\downarrow})$$

is already a density-density interaction between spin-orbitals, so $S_i^z S_j^z$ is solved at the same mean-field level as the $U/V_1/V_2/V_3$ interactions above, with `h.get_szsz_mean_field_hamiltonian(J1=...)`. J1/J2/J3 are the first-, second- and third-neighbor couplings and Jr a general distance-dependent one, the same convention as V1/V2/V3/Vr. The other two axes are `h.get_sxsx_mean_field_hamiltonian(...)` and `h.get_sysy_mean_field_hamiltonian(...)`. The bare interaction is SU(2) symmetric, so at the same coupling the three differ only in the axis the moment orders along.

```
from pyqula import geometry
g = geometry.chain() # a chain, prone to ferromagnetic order away from half filling
h = g.get_hamiltonian(has_spin=True)
h = h.get_szsz_mean_field_hamiltonian(J1=-2.0,filling=0.2,
                                     mf="ferroZ", # ferromagnetic Sz-Sz coupling
                                     nk=10,mix=0.3,maxite=300)
m = h.get_magnetization() # uniform moment along z
```

The ferromagnetic coupling on a chain away from half filling orders every site with the same moment along z , which `h.get_magnetization()` returns as one vector per site, and the two spin bands of the converged Hamiltonian are split

by an exchange field that the calculation found rather than one put in with `h.add_exchange()`.

Let us spell out here what decides whether any of the mean-field calculations in this chapter converges, since all of them return `None` instead of a Hamiltonian when they do not, and the result has to be checked before it is used. Four parameters decide it:

- `nk` (8 by default): the number of k-points per direction of the mesh on which the expectation values are computed
- `mix` (0.1): the fraction of the new mean field mixed into the old one at each iteration
- `maxite` (1000, or 2000 with `use_jax=True`): the maximum number of iterations before giving up, `None` for no limit
- `maxerror` (1e-5): how close two successive mean fields must be for the result to count as converged

The k-mesh is usually the real culprit: a mesh that does not resolve the Fermi surface makes the occupied states flip from one iteration to the next, and no amount of mixing helps. The chain above at `filling=0.2` never converges at `nk=8` and converges at once at `nk=10`, so this is the parameter to change first. A lower `mix` is the adiabatic update of the collinear section, and helps when the calculation oscillates around a solution rather than settling on it. A calculation that never converges gives up after `maxite` iterations and returns `None`, and on a large system 1000 iterations can take a long while, so set a lower `maxite` while exploring parameters and the answer comes back in seconds.

The three axes can be given different couplings in a single anisotropic-exchange calculation, with `h.get_exchange_mean_field_hamiltonian(Jx1=...,Jy1=...,Jz1=...)`:

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_exchange_mean_field_hamiltonian(Jz1=-1.0,Jx1=-0.5,
                                          filling=0.2,mf="ferroZ",
                                          nk=10,mix=0.3,maxite=300)
```

Density-density interactions and spin-spin exchange can also be solved together in one self-consistent calculation, with `h.get_combined_mean_field_hamiltonian(U=...,V1=...,J1=...,...` which is what a model with both charge and magnetic correlations needs. Here $J_1/J_2/J_3$ (and J_r) are isotropic Heisenberg couplings $J(S_i^x S_j^x + S_i^y S_j^y + S_i^z S_j^z)$ on the first, second and third neighbor shells, and $J_{1x}/J_{1y}/J_{1z}$ add an anisotropy on top of J_1 for the first shell only, so that the effective first-neighbor J_z is $J_1 + J_{1z}$. All of them default to zero

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,
                                          filling=0.2,mf="ferroZ")
```

For a large, sparse system the diagonalization at every k-point is the expensive step of each iteration, and the same self-consistent loop can get its density matrix

instead from a Chebyshev expansion, the kernel polynomial method of its own chapter, never diagonalizing the Bloch Hamiltonian, with `integration="kpm"`:

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,filling=0.2,
                                           mf="ferroZ",integration="kpm")
```

A loop that refuses to converge under plain mixing can be handed to a nonlinear solver instead, with `use_jax=True`, which treats one iteration $x = f(x)$ of the loop as a root-finding problem for $x - f(x) = 0$. `solver="error_gradient"` is the most robust of these on a generic Hamiltonian and the one to reach for first; `"newton"` is the default. These solvers are for the normal state only and do not accept `constrains`.

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,filling=0.2,
                                           mf="ferroZ",use_jax=True,
                                           solver="newton")
```

The accepted names are `"newton"`, `"newton_krylov"`, `"fsolve"`, `"error_gradient"` (also called `"levenberg_marquardt"`), `"lbfgs"`, `"broyden_mixing"` and `"linear_mixing"` (also called `"fixed_point"`), the same for the spinful and the spinless mean-field functions, and an unknown name raises `ValueError` listing them. Without `use_jax=True` the loop is plain mixing, so `solver=` is read only together with it, and passing it alone raises `NotImplementedError` rather than being ignored.

See the notebooks `03_spin_spin_exchange.ipynb`, `11_kpm_scf.ipynb` and `12_jax_scf_solvers.ipynb` in the folder above for executed versions of the three.

All of the spin-spin exchange functions above also work on Bogoliubov-de Gennes (Nambu) Hamiltonians, set up with `h.turn_nambu()` or `h.setup_nambu_spinor()`, where the exchange channels are decoupled in the normal and in the anomalous channel, exactly as $U/V_1/V_2/V_3$ are. Exchange can therefore induce superconducting pairing on its own: an anti-ferromagnetic isotropic J , with no U or V at all, can decouple spontaneously into a purely superconducting singlet-paired state, the resonating-valence-bond mechanism behind exchange-driven superconductivity, while the ferromagnetic sign has no such tendency and stays magnetic. That instability has to be seeded coherently, with `h.add_swave(0.1)` on top of the Hamiltonian passed as `mf`, since a random guess has little overlap with it and usually relaxes back to zero pairing. A state carrying both magnetic and superconducting order can also emerge from an exchange field combined with an attractive V_1 , as below. The total energy returned with `return_total_energy=True` subtracts the double counting of the pairing mean field as well as of the normal one, so it is the mean-field energy of the paired state, and comparing it with the energy of an unpaired solution of the same interaction tells you which of the two is the ground state at the mean-field level:

```

h = g.get_hamiltonian(has_spin=True)
h.add_exchange([0.,0.,0.3])
h.turn_nambu()
h = h.get_combined_mean_field_hamiltonian(V1=-1.0,J1z=-0.3,
                                           filling=0.3,mf="random")

```

Abrikosov-pseudofermion (spinon) mean field for Heisenberg models

Let us now address a pure spin model, a spin- $\frac{1}{2}$ Heisenberg model $H = J \sum_{\langle ij \rangle} \vec{S}_i \cdot \vec{S}_j$ in which no electron hops at all, which `SpinonHamiltonian` treats with the same mean-field machinery. The idea is to write each spin in terms of an auxiliary fermion, $\vec{S}_i = \frac{1}{2} f_i^\dagger \vec{\sigma} f_i$, the spinon, subject to the hard local constraint $f_i^\dagger f_i = 1$, exactly one fermion per site, since a spin one half has no charge fluctuations; you can think of a spinon as the spin sector of an electron, with the charge frozen. The exchange term is then quartic in the spinons, and its mean-field decoupling produces a bond order parameter $\chi_{ij} = \langle f_i^\dagger f_j \rangle$, the resonating-valence-bond ansatz, that gives the spinons a dispersion: this is the same decoupling that the J channel of `h.get_combined_mean_field_hamiltonian()` performs, on a Hamiltonian with zero bare hopping (a pure spin model has no kinetic term) and with the local constraint enforced at every site individually, not only on lattice average. The representation follows the review of Savary and Balents (arXiv:1601.03742, section 4), and `SpinonHamiltonian` (`pyqula.spinon`) packages exactly this:

```

from pyqula import geometry
from pyqula.spinon import SpinonHamiltonian

g = geometry.triangular_lattice() # a canonical frustrated-Heisenberg lattice
h = SpinonHamiltonian(g) # zero bare hopping, couplings come from J1/J2/...
h2 = h.get_mean_field_hamiltonian(J1=1.0,nk=12,mix=0.1,maxerror=1e-4)

h2.local_occupation # <n_i> per site, exactly 1.0 at convergence
h2.constraint_lambda # converged per-site Lagrange multiplier (local chemical potential)
h2.get_bands() # spinon dispersion

```

The converged object is a spinon Hamiltonian: its local occupation is one on every site, the per-site Lagrange multiplier is the local chemical potential that enforces it, and its band structure is the spinon dispersion, the spectrum of spin- $\frac{1}{2}$ excitations that a quantum spin liquid would show in place of the magnons of an ordered magnet. `filling=` cannot be passed here: the representation is only valid at exactly one fermion per site, so that occupation is imposed site by site through the Lagrange multiplier rather than as a lattice-averaged Fermi level. Only the U(1) ansatz, with the bond order alone, is implemented; a Z2 ansatz, which would allow the pairing channel that J can also induce, is not.

On a frustrated lattice (triangular, kagome and the like) the converged state is

ansatz-dependent, not unique: several distinct self-consistent flux sectors of the bond order can coexist at the same J , and which one an unseeded random `mf` guess lands on is itself part of the physics rather than noise of the loop, since there is no way to search all self-consistent solutions and calculations are carried out by assuming a decoupling scheme, as the review puts it. A chain with one site per unit cell has a unique solution (no frustration), so repeated calls agree to within `maxerror`; on a frustrated lattice, pass an explicit `mf=` to select a definite ansatz deliberately rather than comparing energies across differently seeded runs.

An external Zeeman field couples to $\vec{S}_i = \frac{1}{2}f_i^\dagger \vec{\sigma} f_i$ exactly, not through any mean-field decoupling, since the term is already bilinear in f , so it is added as an ordinary single-particle term with the same `h.add_zeeman()` or `h.add_exchange()` used everywhere else in this guide, called on the `SpinonHamiltonian` instance before `get_mean_field_hamiltonian`:

```
h = SpinonHamiltonian(g)
h.add_zeeman([0., 0., 0.3])           # or h.add_exchange([0.,0.,0.3])
h2 = h.get_mean_field_hamiltonian(J1=1.0, nk=12)
h2.get_magnetization()                # induced <S> per site
```

The argument of `add_zeeman` is the coefficient of $\vec{\sigma}$ (the Pauli matrices), not of $\vec{S} = \vec{\sigma}/2$, so the physical field h in $H = -h \cdot S_i$ is twice the value passed in, the same convention `add_exchange` uses on an ordinary electronic Hamiltonian elsewhere in this guide. The one-fermion-per-site constraint is a constraint on the total occupation, not on the spin, so it stays exactly satisfied under a field while $\langle S_i \rangle$ is free to grow with it, saturating once the field dominates J .

See `examples/spinon/heisenberg_chain/main.py` and `examples/spinon/heisenberg_chain_zeeman_field/main.py` for runnable versions, and the notebook `13_spinon_rvb.ipynb` in the folder above.

Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice

Let us finally address the Kondo lattice, localized moments exchange-coupled to a conduction electron at the same site, the minimal model of heavy-fermion compounds, which `KondoLatticeHamiltonian` solves in the same pseudofermion language. Following Coleman's review (arXiv:cond-mat/0612006, section III.C), the model is written in its Coqblin-Schrieffer form $H = \sum_k \epsilon_k c_k^\dagger c_k + \frac{J}{N} \sum_j S_{ab}(j) c_{jb}^\dagger c_{ja}$, with $N = 2$ for a spin- $\frac{1}{2}$ moment, which is not the coefficient of a bare $J \vec{S}_j \cdot \vec{s}_j$ Heisenberg-form Kondo term (see the caveat below). Each moment is represented by an Abrikosov pseudofermion $\vec{S}_j = \frac{1}{2}f_j^\dagger \vec{\sigma} f_j$ subject to the constraint $f_j^\dagger f_j = 1$, as in the previous section, and the exchange term is decoupled into a self-consistent hybridization field $V_j = -\frac{J}{2} \langle f_j^\dagger c_j \rangle$, plus a Lagrange multiplier λ_j enforcing the local constraint. You can think of V_j as the amplitude for a conduction electron to turn

into a spin flip of the local moment and back, a composite fermion that is half electron and half spin flip, and the resulting mean field is the large- N (here $N = 2$) Read-Newns saddle point of the Kondo-lattice path integral. `KondoLatticeHamiltonian` (`pyqula.kondolattice`) packages this: given a conduction-electron Hamiltonian, it fuses on a second, initially decoupled sublattice of localized f-sites (one per conduction site) with zero bare hopping, and self-consistently solves for V_j and λ_j :

```
from pyqula import geometry
from pyqula.kondolattice import KondoLatticeHamiltonian

gc = geometry.chain()
hc = gc.get_hamiltonian(has_spin=True) # conduction electrons
h = KondoLatticeHamiltonian(hc)

seed = ([0.3+0.0j],[0.0]) # (V,lam), see the caveat below for why
h2 = h.get_mean_field_hamiltonian(J=1.5,filling=0.15,nk=200,mf=seed)

h2.local_occupation # <n_f> per localized site, exactly 1.0 at convergence
h2.hybridization    # converged V per localized site
h2.constraint_lambda # converged per-site Lagrange multiplier
```

What comes out is the heavy-fermion band structure: the conduction band hybridizes with the flat f-level at the energy set by λ_j , and the converged V_j opens a hybridization gap around it, with bands on either side that are heavy because most of their weight sits on the f-sites. The three attributes hold the converged occupation of the f-sites, one per site, the hybridization and the Lagrange multiplier.

J is Coleman’s Coqblin-Schrieffer coupling, entering the interaction as J/N with $N = 2$, not the coefficient of a bare $J\vec{S}_j \cdot \vec{s}_j$ Heisenberg-form Kondo term; the two differ by a numerical factor that the review already fixes, and this class follows its convention exactly. `filling` sets a lattice-wide chemical potential once, from the bare ($V = 0$) bands, and holds it fixed: this is a grand-canonical Hamiltonian, in which the electron count is meant to float as V and λ converge. The local $\langle n_f \rangle = 1$ constraint is enforced separately by λ_j , not by `filling`.

Note that $V = 0$ is always itself a self-consistent solution, exactly like the trivial root of the BCS gap equation, so an unseeded run (`mf=None`, the default) starts there and stays there even for a J that also supports a genuine hybridized state, and a nonzero seed (as above) is generally needed to find it. Where both solutions coexist, the hybridized state is the true (lower-energy) ground state. Avoid a `filling` that lands the chemical potential inside the flat, macroscopically degenerate f-band of the bare problem: at $V = 0$ every f-orbital sits at exactly λ , so a wide range of fillings, roughly 0.25 to 0.75 for a single conduction orbital per site, all give the same numerically ill-posed starting point, and `filling=0.15` above keeps μ inside the dispersing conduction band instead. The finite Fermi-

Dirac smearing T this loop runs at turns the continuous onset $T_K = D e^{-1/(J\rho)}$ of the zero-temperature theory into a genuine finite-temperature Kondo crossover: below a T -dependent threshold in J , thermal smearing washes out the hybridization entirely and $V = 0$ becomes the only self-consistent solution, and right at the threshold V jumps directly to a value of order one rather than growing continuously from zero.

An external Zeeman field couples exactly to both fermion species here, the conduction electron and the localized moment $\vec{S}_j = \frac{1}{2} f_j^\dagger \vec{\sigma} f_j$, already bilinear in f , so, exactly as for `SpinonHamiltonian` above, it is added as an ordinary single-particle term with `add_zeeman` or `add_exchange`, called on the `KondoLatticeHamiltonian` instance before `get_mean_field_hamiltonian`:

```
h = KondoLatticeHamiltonian(hc)
h.add_zeeman([0., 0., 0.05])
h2 = h.get_mean_field_hamiltonian(J=1.5, filling=0.15, nk=150, mf=seed)
```

`add_zeeman` applies to every site of the fused geometry, that is, to both the conduction and the f sublattice (offset in z); pass a position-dependent callable instead of a constant vector to target only one of them. The $\langle n_f \rangle = 1$ constraint (a constraint on the total occupation, not on the spin) stays exact under a field. A field competes with the Kondo singlet: the self-consistent $|V|$ shrinks as the field grows at fixed J , and a strong enough field destroys the hybridized state, which is genuine physics rather than a numerical failure, and the loop reports non-convergence (`None`) there rather than a spuriously small but nonzero V , exactly the decay toward the always-self-consistent $V = 0$ branch that a subcritical J already produces above.

See `examples/kondolattice/chain/main.py` for a runnable version, and the notebook `14_kondo_lattice.ipynb` in the folder above.

Spatially resolved density of states

Let us now look at the local density of states of a finite island, resolved in space and scanned over a whole set of energies at once, with `h.get_multildos()`. For a triangular island of the honeycomb lattice, projected onto atomic orbitals rather than onto the tight-binding sites so that the map is smooth, we write

```
from pyqula import islands
g = islands.get_geometry(name="honeycomb", n=3, nedges=3) # get an island
h = g.get_hamiltonian() # get the Hamiltonian
h.get_multildos(projection="atomic") # get the LDOS
```

The call writes one map per energy to a `MULTILDOS/` folder, with the density of states of the whole island next to them in `DOS.OUT`, and the map at a given energy is what one measures with STM at that bias. The arguments, the projections and the single-energy version are addressed in the section on the local

density of states; see `examples/readme_examples/ldos_island/main.py` for a runnable version ending in the maps.

Electronic structure folding and unfolding

We will now see how the band structure of a supercell folds into its smaller Brillouin zone, and, what matters most, how it can be unfolded back onto the Brillouin zone of the primitive cell, with `operator="unfold"`.

Building a supercell does not change the electronic structure, but it does change how it is drawn: the Brillouin zone shrinks, and the bands of the primitive cell fold into it, so that a supercell of $n \times n$ primitive cells shows n^2 times more bands, all of them replicas of the same dispersion. The inverse operation, unfolding, recovers the primitive-cell spectral weight of a supercell calculation, and this is what is needed when the supercell is there for a reason (a defect, a moire pattern, or a non-primitive choice of cell) and the result is to be compared against a measured band structure, since ARPES sees the spectral weight of the primitive cell and only a faint trace of the folded replicas when the perturbation is weak. Unfolding is implemented as a special operator, "unfold", that projects onto the Bloch states of the primitive cell; it can be passed to any of the k-resolved observables (`h.get_bands()`, `h.get_kdos_bands()`, `h.get_multi_fermi_surface()`...), and it requires the supercell to have been built keeping track of the primitive geometry

```
from pyqula import geometry
import numpy as np
g = geometry.honeycomb_lattice() # primitive geometry
n = 3
gs = g.get_supercell(n,store_primal=True) # supercell, keeping the primitive cell info
h = gs.get_hamiltonian() # Hamiltonian of the supercell
kpath = gs.get_unfolded_kpath(nk=200) # primitive k-path, in supercell coordinates
(k,e,d) = h.get_kdos_bands(operator="unfold",delta=1e-1,kpath=kpath) # unfolded weight
```

`gs.get_unfolded_kpath()` gives the k-path of the primitive cell written in the coordinates of the supercell, which is the path the unfolded bands have to be drawn along, and the section “Which k-path to unfold along” below says what it does; `d` holds the unfolded spectral weight at each $(\mathbf{k},\mathbf{e})$, so that plotting a scatter of $\mathbf{k},\mathbf{e}$ colored or sized by `d` recovers the primitive-cell band structure out of the supercell calculation: the replicas that a plain `h.get_bands()` of the supercell would show carry no weight, and the Dirac cones of the honeycomb lattice reappear where they belong. The same `operator="unfold"` can be passed to `h.get_multi_fermi_surface()` to unfold constant-energy cuts. See `examples/2d/unfolding/main.py`, `examples/1d/unfolding/main.py` and `examples/readme_examples/unfolding_FS/main.py` for runnable versions, and `jupyterter-notebooks/functionalities/single_particle_hamiltonians/08_unfolding_supercells.i` for the executed notebook.

Unfolding also works when atoms have been removed from the supercell (for example `gs = gs.remove(...)` before `gs.get_hamiltonian()`), such as a vacancy or an irregularly-shaped flake cut out of a supercell: each remaining atom is matched back to its primitive-cell replica by position, so no extra arguments are needed. The match requires the remaining atoms to sit exactly where they were in the complete supercell, so do not move them (with `gs.center()`, or a relaxation) between `gs.remove(...)` and `gs.get_hamiltonian()`. Doing so raises a `ValueError` rather than unfolding onto the wrong replica.

Unfolding also works for a general, non-diagonal or non-orthogonal supercell, built by passing a 3x3 integer matrix `M` to `get_supercell` instead of a plain `(n1,n2,...)` size (`gs.a1,gs.a2,gs.a3` become integer combinations of the primitive vectors, `gs = M @ g`). No change is needed at the unfolding call site: `get_supercell(M,...)` records, per surviving atom, which primitive replica it came from, and `operator="unfold"` reads that bookkeeping directly, both for a complete supercell and after removing atoms

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # primitive geometry
M = [[2,1,0],[0,1,0],[0,0,1]] # non-diagonal supercell matrix, det(M)=2
gs = g.get_supercell(M,store_primal=True) # supercell, keeping the primitive cell info
h = gs.get_hamiltonian() # Hamiltonian of the supercell
kpath = gs.get_unfolded_kpath(nk=200) # primitive k-path, in supercell coordinates
(k,e,d) = h.get_kdos_bands(operator="unfold",delta=1e-1,kpath=kpath) # unfolded weight
```

The projector that the unfolding operator builds is a sum of Bloch phases over the replicas of the primitive cell, and nothing in it refers to the dimensionality, so all of this works the same way for a chain, for a two-dimensional lattice and for a bulk three-dimensional crystal, with a plain size or with a 3x3 `M`

```
from pyqula import geometry
g = geometry.cubic_lattice() # three dimensional primitive geometry
gs = g.get_supercell([2,2,2],store_primal=True) # supercell, keeping the primitive cell info
h = gs.get_hamiltonian() # Hamiltonian of the supercell
kpath = gs.get_unfolded_kpath(nk=200) # primitive k-path, in supercell coordinates
(k,e,d) = h.get_bands(operator="unfold",kpath=kpath) # unfolded band structure
```

See `examples/2d/unfolding_nonorthogonal/main.py` for a runnable version with a defect in the non-diagonal supercell, and `examples/3d/unfolding/main.py` for the three-dimensional one.

A cell that is not a multiple of the primitive one along each lattice vector can also be asked for by its size alone, and the case that comes up most often is the $\sqrt{3} \times \sqrt{3}$ supercell of a triangular or a honeycomb lattice, the cell of a three-sublattice charge order

```
from pyqula import geometry
import numpy as np
g = geometry.triangular_lattice() # primitive geometry
```

```
gs = g.get_supercell(np.sqrt(3),store_primal=True) # sqrt(3) x sqrt(3) supercell
```

A non-integer size is read as a request for a cell larger by that factor in area, three primitive cells here, and `get_supercell` returns the most compact cell of that size that can be built out of integer combinations of $\vec{a}_1$ and $\vec{a}_2$, which for the triangular lattice is the cell with $|\vec{A}_1| = |\vec{A}_2| = \sqrt{3}|\vec{a}_1|$ at 60 degrees from each other, rotated so that $\vec{A}_1$ points along x . Several integer matrices of determinant three describe that same cell, `gs.supercell_matrix` says which one was taken, and passing an explicit `M` is the way to fix both the matrix and the orientation rather than leaving them to the search. Compactness is what makes the cell usable and not just correct: a cell of the right volume but with a small angle between its lattice vectors puts the first neighbors of an atom outside the cells the hopping generator searches, and the Hamiltonian comes out with hoppings missing.

Which k-path to unfold along

The unfolded band structure has to be drawn along the Brillouin zone of the primitive cell, and this is where a general supercell differs from an $n \times n$ one. The unfolding operator sends a momentum $\vec{k}$ of the supercell onto the momentum $M^{-1}\vec{k}$ of the primitive cell, so tracing the primitive Brillouin zone means feeding it $M\vec{k}_0$ for every point $\vec{k}_0$ of the primitive path. For a diagonal supercell that is the primitive path times n , which is how `examples/2d/unfolding/main.py` and the other diagonal examples write it by hand, and for a $\sqrt{3} \times \sqrt{3}$ cell it is a rotation as well as a rescaling, which no multiplication by a number can give. `gs.get_unfolded_kpath()` returns it for any supercell, diagonal or not

```
from pyqula import geometry
import numpy as np
g = geometry.triangular_lattice() # primitive geometry
gs = g.get_supercell(np.sqrt(3),store_primal=True) # sqrt(3) x sqrt(3) supercell
h = gs.get_hamiltonian() # Hamiltonian of the supercell
fons = lambda r: (np.sum((r - gs.r[0])**2)<1e-2)*0.6 # onsite in one of the three sites
h.add_onsite(fons) # three-sublattice charge order
kpath = gs.get_unfolded_kpath(nk=200) # primitive k-path, in supercell coordinates
(k,e,d) = h.get_kdos_bands(operator="unfold",delta=1e-1,kpath=kpath) # unfolded weight
```

The k-points come back in the reduced coordinates of the supercell, which is what `h.get_bands()` and `h.get_kdos_bands()` take, so they go straight into `kpath=`, and the method takes the same arguments as `g.get_kpath()`, meaning that the high-symmetry points can be named as well, `gs.get_unfolded_kpath(["G","K","M","G"],nk=400)`, and they are the points of the primitive cell and not of the supercell. Plotting `k,e` colored by `d` gives the single band of the triangular lattice along Γ -K-M-K'- Γ , with a gap opened at K by the charge order and a faint shadow band where the folded replica carries the little weight that the modulation gives it. Along the supercell k-path instead, `gs.get_kpath()`, the same calculation traces a path through the

primitive Brillouin zone that goes nowhere in particular, and this is the reason for asking for the primitive one. See `examples/2d/unfolding_sqrt3/main.py` for a runnable version.

Surface spectral functions

We will now see how to compute the surface spectral function of a semi-infinite system, meaning a system that is bulk-like far from a boundary but is cleaved along one direction, with `h.get_surface_kdos()`.

A ribbon of finite width has two edges and a discrete set of bands that depends on the width, whereas a semi-infinite system has a single edge and a continuum of bulk states behind it, which is the situation of a cleaved crystal. The way to obtain it is through the surface Green's function, computed with a renormalization (decimation) technique for the semi-infinite bulk: the layers behind the surface are folded into it one after another until adding one more changes nothing, and the imaginary part of the resulting Green's function, at each momentum along the edge and each energy, is the surface spectral function, what ARPES measures on the surface of a crystal. For a honeycomb lattice with Kane-Mele spin-orbit coupling we write

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_soc(0.05) # Kane-Mele spin-orbit coupling opens a topological gap
h.add_rashba(0.1) # break z-mirror symmetry to expose the edge states
(k,e,ds,db) = h.get_surface_kdos(delta=1e-2) # surface and bulk spectral functions
```

`ds` and `db` are, respectively, the surface and bulk spectral weight at each $(\mathbf{k}, e)$. The spin-orbit coupling opens a gap in the bulk, so `db` is empty inside it, and plotting $\mathbf{k}, e$ colored by `ds` shows the states that cross that gap: the helical edge states of the quantum spin Hall state, living at the boundary and absent from the bulk spectrum, which is what makes them topologically protected. See `examples/readme_examples/surface_2dTI/main.py` for a runnable version, and `jupyter-notebooks/functionalities/spectral_functions/02_surface_spectral_semiinfinite.ip` for the executed notebook.

The $\mathbf{k}$ -integrated counterpart is `dos.surface_dos(h, ...)`, which returns the surface and bulk densities of states at a set of energies. It takes an `operator=` and projects onto it, giving for example the spin-resolved surface DOS of a magnetized lead rather than the charge one. A momentum-dependent operator such as "valley" is refused with a `NotImplementedError` naming it, since the Green's function it is built on has already been integrated over the Brillouin zone.

This whole family, `dos.surface_dos`, `dos.dos_surface`, `dos.bulkandsurface`, `dos.surface2bulk` and the surface writers in `kdos`, reports $-\text{Im Tr } G$ without

the $1/\pi$ that `h.get_dos()` applies, so their values are π times a density of states. They agree with each other; compare them among themselves rather than against `h.get_dos()`. The same decimation of a semi-infinite lead is what the quantum transport chapter builds on.

Twisted bilayer graphene structural relaxation

Let us now see how the structural relaxation of a twisted graphene bilayer is included before its electronic structure is computed, with `GrapheneGeometry` and its `.relax()` method. Rigidly twisting two graphene layers is only an approximation: below a few degrees of twist, the real lattice relaxes so that the energetically costly AA-stacked regions shrink and triangular AB/BA (Bernal) domains grow around them, separated by solitonic domain walls (Nam and Koshino, arXiv:1706.03908). `GrapheneGeometry` wraps any graphene multilayer `Geometry` (bilayer, twisted bilayer, twisted trilayer, ...) and adds a `.relax()` method that reproduces this effect by minimizing a phenomenological energy over an in-plane relaxation displacement field. That energy has two pieces: the interlayer generalized stacking fault energy (GSFE), a closed-form periodic function of the local interlayer registry fit to graphene's AA, AB and BA stacking energies, and the intralayer linear-elastic energy, both taken from Carr, Massatt, Torrisi, Cazeaux, Luskin and Kaxiras, arXiv:1805.06972, Table 1. The minimization runs entirely in-plane: out-of-plane corrugation is not included.

`GrapheneHamiltonian` builds the actual tight-binding Hamiltonian from a (relaxed or rigid) `GrapheneGeometry`, defaulting to the same distance-decaying hoppings as `specialhamiltonian.twisted_bilayer_graphene`; since those hoppings depend on the true 3D interatomic distance, the relaxed positions feed into the electronic structure automatically

```
from pyqula import specialgeometry
from pyqula.graphenetc.geometry import GrapheneGeometry
from pyqula.graphenetc.hamiltonian import GrapheneHamiltonian

g0 = specialgeometry.twisted_bilayer(m0=15) # ~2 degree twist
g = GrapheneGeometry(g0).relax() # AA area shrinks, AB/BA domains grow
h = GrapheneHamiltonian(g)
(k,e) = h.get_bands(num_bands=20)
```

`.relax()` returns a new geometry and leaves the rigid one untouched, so the two can be compared side by side. AA is the maximum of the stacking-fault energy and AB/BA its degenerate minima, so the AA regions shrink and the AB/BA domains grow, by an amount that increases as the twist angle decreases, and the band structure computed from the relaxed positions is the one to compare against a measurement on a real sample. See `examples/2d/graphene_relax/main.py` for a runnable version comparing the rigid and relaxed lattices.

Topological insulators

Let us now look at the quantities that tell whether a gapped Hamiltonian is a topological insulator, a system with a gap in the bulk and states at the edges that no perturbation can remove as long as the gap stays open. In this chapter we first compute the invariants that count those edge states, the Chern number for a system with broken time-reversal symmetry and the Z_2 invariant for one that preserves it, then the quantum geometric tensor the Berry curvature is part of, and afterwards we resolve the Berry curvature in energy and in real space, where it becomes a local marker that needs no Brillouin zone at all. The chapter closes with the surface spectral function, where the edge states the invariant predicts are seen directly.

Topological invariants

A topological invariant is an integer computed from the occupied bands over the whole Brillouin zone, and an integer cannot change continuously: the only way to change it is to close the gap, and this is the reason the edge states it counts are protected.

Chern number

We will now see how to compute the Chern number, the invariant of a two-dimensional insulator with broken time-reversal symmetry, with `h.get_chern()`. It is defined as

$$C = \frac{1}{2\pi} \int \Omega(\mathbf{k}) d^2\mathbf{k}$$

where Ω is the Berry curvature, meaning that the Chern number is the total Berry flux of the occupied bands through the Brillouin zone in units of 2π . It is an integer, it is the Hall conductivity of the insulator in units of e^2/h , and it counts the chiral states that cross the gap at each edge, which is how one measures it. Let us take the Haldane model, a honeycomb lattice with a complex second-neighbor hopping that breaks time-reversal symmetry and opens a topological gap at the Dirac points. The Chern number can be computed with the following code

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
C = h.get_chern() # Chern number
```

The result is $C = 2$: the Hamiltonian is spinful by default and the Haldane term acts the same way on both spins, so each spin contributes one, and the same

model built with `has_spin=False` gives $C = 1$. The integral is a sum of the Berry curvature over a k-mesh of `nk` points per direction, ten by default, which is enough for a gap of this size. The closer the system is to a gap closing, the more sharply peaked the Berry curvature becomes and the finer the mesh has to be before the number settles at an integer, so a Chern number that comes out far from an integer is asking for a larger `nk`, or for the adaptive integration of the next section. An `operator=` restricts the curvature to a subspace, a spin or a valley, which is how a spin Chern number or a valley Chern number is obtained (see the Berry curvature operator section), and `h.get_berry_curvature()` returns the curvature itself as a map over the Brillouin zone, the same one that this integral sums.

See `examples/2d/chern/main.py` (a Chern insulator built from Rashba coupling and an exchange field), `examples/2d/haldane_model/main.py` and `examples/2d/valley_chern/main.py` for runnable versions, and `jupyter-notebooks/functionalities/topological_characterization/01_chern_z2.ipynb` and `02_operator_resolved_chern.ipynb` for executed notebooks with the curvature maps.

Tensor-cross-interpolation (qtci) integration By default the Brillouin-zone integral above is a plain sum over a uniform `nk` x `nk` mesh, so its cost grows as `nk`². There is a second way of evaluating it, `integration="qtci"`, which treats the integrand as a function on a binary-refined grid and learns it adaptively by quantics tensor cross interpolation, sampling where the function varies, and then integrates the resulting representation with Gauss-Kronrod quadrature. The number of evaluations then grows roughly logarithmically rather than quadratically in the effective resolution, which `nk` sets

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
h.add_haldane(0.05)
C = h.get_chern(integration="qtci",nk=20) # tensor-cross-interpolated BZ integral
```

What matters is when to reach for it. The adaptive integration is accurate for a smooth Berry curvature, a model with a large gap, where it reaches the integer to between 10^{-5} and 10^{-9} from far fewer evaluations than a dense mesh. For a sharply peaked curvature, a small gap, it is the wrong tool: the Haldane model above, whose gap is small, returns $C = 2.015$ with `nk=20`, and the error grows rather than shrinks as `nk` is raised, since refining the quadrature grid leaves fewer of its nodes near the peak. The default mesh sum has no such problem, because it counts the vortices of the Berry phase on the plaquettes of the mesh rather than integrating a field, so it is exactly quantized at every `nk`; it is the choice whenever the gap is small. The same backend can compute the density matrix in a mean-field calculation, replacing the k-mesh sum there:

```
from pyqula import geometry
```

```

g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
hscf,e = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,mf="antiferro",
nk=8,maxerror=1e-4,return_total_energy=True,integration="qtci")

```

In a metal the density matrix has a step at the Fermi surface, and a quadrature rule integrates a step slowly. The Fermi level is located on the same quadrature nodes the density matrix is integrated on, and the total energy is summed on them too, so that the charge and the energy belong to one another. On a lattice with high symmetry the nodes come in groups related by the symmetry, so at a temperature close to zero the level at the Fermi energy can be a group of several states holding a sizable fraction of an electron, and that level is then occupied only in part, so that the charge is exactly the requested filling. What remains is the quadrature error of a step, which the k-mesh has to converge away as usual: for an attractive $U = -2$ on the square lattice at `filling=0.3`, the pairing gap comes out 16% too small at `nk=8` and within 3% at `nk=16`. For a gapped system none of this applies, and qtci and the k-mesh sum agree. Note also that in two dimensions the density-matrix path ends up evaluating every node of the quadrature grid, so it is not cheaper than the k-mesh sum there.

The tensor cross interpolation itself is a pure-Python port of `TensorCrossInterpolation.jl`, bundled with pyqula, so nothing extra needs installing. Note that the density-matrix path supports 2D Hamiltonians only. Runnable versions are in `examples/2d/chern_qtci/main.py` and `examples/2d/mean_field_qtci/main.py`.

Z2 invariant

When time-reversal symmetry is preserved the Chern number vanishes, and the invariant that remains is the Z_2 index, with only two values: trivial, or a quantum spin Hall insulator with a helical pair of states at each edge. We will now see how to compute it for the Kane-Mele model, a honeycomb lattice with intrinsic spin-orbit coupling, with `topology.z2_invariant()`. It can be computed with the following code

```

from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_soc(0.05) # Add spin-orbit coupling
from pyqula import topology
z2 = topology.z2_invariant(h) # Z2 invariant

```

The result is a parity, -1 for a topological insulator and $+1$ for a trivial one: the spin-orbit coupling of the snippet gives -1 , and a honeycomb lattice gapped by a sublattice imbalance instead gives $+1$. What the invariant counts is the parity of the number of helical pairs at each edge, since an odd number of Kramers pairs crossing the gap cannot be removed by any perturbation that preserves time-reversal symmetry, while an even number can. It is evaluated from the pumping

of the Wannier centers across half of the Brillouin zone, on a mesh of `nk` momenta and `nt` pumping steps (60 each by default). `h.add_soc()` is the same term as `h.add_kane_mele()`. Note that `h.get_topological_invariant()` chooses between the two invariants for you, returning the Z_2 invariant when the Hamiltonian is time-reversal symmetric and the Chern number otherwise, and the Berry phase in one dimension.

See `examples/2d/z2_kane_mele/main.py` and `examples/2d/z2_transition/main.py` (the invariant across a transition driven by a sublattice imbalance) for runnable versions, and the `01_chern_z2.ipynb` notebook above.

Quantum geometric tensor (multiorbital/multiband)

The Berry curvature is only half of the geometry of the Bloch states, and we will now see how to compute the whole of it, the quantum geometric tensor, with `h.get_quantum_geometric_tensor()`. Its antisymmetric part is the Berry curvature, and its symmetric part is the quantum metric, a measure of the distance between neighboring Bloch states, which is the quantity that matters in a flat band, where the curvature alone says nothing about how spread out the states are. For a chosen band subspace S (e.g. the occupied bands) it is

$$Q_{ij}^{mn}(\mathbf{k}) = \sum_{l \notin S} \frac{\langle u_m | \partial_{k_i} H | u_l \rangle \langle u_l | \partial_{k_j} H | u_n \rangle}{(E_m - E_l)(E_n - E_l)}, \quad m, n \in S$$

where m and n run over the bands of the subspace and l over the bands outside it, with the quantum metric $g_{ij}^{mn} = \text{Re } Q_{ij}^{mn}$ (symmetric part) and Berry curvature $\Omega_{ij}^{mn} = -2 \text{Im } Q_{ij}^{mn}$ (antisymmetric part) recovered in the band-trace (“Abelian”) case. What about the tensor resolved per pair of bands, the “non-Abelian” one? Note that Q_{ij}^{mn} is not gauge independent: rotating the states of S among themselves, which is exactly the freedom a diagonalization has inside a degenerate multiplet, rotates the tensor too, so its individual entries are not physical, only its trace and its invariants are. For that reason `non_abelian=True` returns it in the orbital basis,

$$Q_{ij}(\mathbf{k}) = \sum_{m,n \in S} |u_m\rangle Q_{ij}^{mn} \langle u_n| = P \partial_{k_i} P \partial_{k_j} P P,$$

with P the projector on S , meaning that it depends on the subspace alone and not on the basis chosen inside it; its trace is the Abelian tensor, and the band-resolved tensor in any basis $|v_m\rangle$ of S you choose is $\langle v_m | Q_{ij} | v_n \rangle$.

What makes this form suitable for a genuinely multiorbital model is that only states *outside* S enter the energy denominators, so it stays well defined when S contains an exactly or nearly degenerate multiplet of bands, an exactly spin-degenerate pair, say, or several orbitals meeting at a high-symmetry point, which an ordinary single-band formula cannot handle. The derivative $\partial_{k_i} H$

is evaluated analytically from the hoppings, with no finite-difference error, with k in the same reduced (dimensionless, period-1) coordinates as the rest of pyqula's k -space code, not Cartesian k , so the absolute scale of the quantum metric depends on the reciprocal lattice if you convert to Cartesian coordinates yourself. `occ_idx`s defaults to the bands with $E < 0$, the same convention `h.get_chern()` uses, so it tracks `h.shift_fermi(...)`.

```
from pyqula import geometry
from pyqula import topology
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system (spinful by default)
h.add_haldane(0.2) # Add Haldane coupling
h.shift_fermi(0.3) # put the Fermi level safely mid-gap (gap is [-0.9,0.9])

Q = h.get_quantum_geometric_tensor(k=[0.1,0.2,0.],occ_idx=[0,1]) # at a k-point
g_metric = h.get_quantum_metric(k=[0.1,0.2,0.],occ_idx=[0,1]) # quantum metric only

# non-Abelian tensor of the two occupied bands, in the orbital basis
Qna = topology.quantum_geometric_tensor(h,k=[0.1,0.2,0.],occ_idx=[0,1],
    non_abelian=True)

# along a k-path, and integrated over the BZ: the integral of the Berry
# curvature part reproduces h.get_chern()
inds,gpath,omegapath = topology.quantum_geometric_tensor_path(h,occ_idx=[0,1])
C = topology.chern_from_qgt(h,nk=20,occ_idx=[0,1])
```

Q is a complex 2×2 array in two dimensions, one entry per pair of directions, and the metric is its real part; with `non_abelian=True` it is a $2 \times 2 \times 4 \times 4$ array, one 4×4 matrix in the spin-orbital basis per pair of directions, and since there is no spin-orbit coupling here it is block diagonal in spin, the trace over the spin-up orbitals being the contribution of the spin-up electrons. Along the k -path the function returns the position along the path and the metric and the curvature at each point, arrays with one 2×2 tensor per k -point, and integrating the curvature part over the Brillouin zone reproduces `h.get_chern()`, here $C = 2$ for the spin-degenerate pair of occupied bands. A degeneracy between the chosen subspace and the bands outside it is the one thing the formula cannot handle, and it raises a `ValueError` rather than dividing by a vanishing energy difference.

See `examples/2d/quantum_geometric_tensor/main.py` for a runnable version and `jupyter-notebooks/functionalities/topological_characterization/06_quantum_geometric_tens` for an executed notebook. There is also an older, unrelated Green's-function estimator of the quantum geometry trace over the whole occupied manifold (not band- or band-pair-resolved), `pyqula.topologytk.quantumgeometry.get_QG_kpath`, see `examples/2d/quantum_geometry/main.py`.

Berry curvature density in frequency space

The Chern number is a single integer for the whole set of occupied bands; we will now see how to resolve where in energy it comes from, with `topology.chern_density()`. The Berry curvature in frequency space is defined as

$$\Omega(\mathbf{k}) = \int_{-\infty}^{\epsilon_F} \Xi(\mathbf{k}, \omega) d\omega$$

where Ω is the Berry curvature of the occupied bands and $\Xi(\mathbf{k}, \omega)$ is the energy-resolved Berry curvature, meaning that Ξ says how much curvature the states at each energy carry, and the Chern number is its integral up to the Fermi energy. `topology.chern_density` integrates $\Xi(\mathbf{k}, \omega)$ over the whole Brillouin zone at a set of energies, giving the frequency-resolved Berry-curvature density and its cumulative (energy-integrated) sum

```
from pyqula import geometry
from pyqula import topology
import numpy as np
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
(es,cs,csi) = topology.chern_density(h,nk=10,es=np.linspace(-1.0,1.0,40))
```

`es` are the energies, `cs` the Berry-curvature density at each energy, and `csi` its cumulative integral. In the gap `csi` should plateau at a value related to the total Chern number of the occupied bands, and integrated over all the bands it returns to zero, since the Berry curvatures of all the bands of a Hamiltonian cancel. This estimator is numerically delicate: it needs a fine enough `nk` and a large enough `delta` to avoid spurious peaks near nearly degenerate `k`-points, and its overall sign and normalization are not guaranteed to match `h.get_chern()` (for the snippet above the plateau at zero energy sits near 4, twice the $C = 2$ of the Chern number section). Read it as a qualitative profile in frequency, and check any number it gives against `h.get_chern()`.

Optional arguments

- `nk`: number of `k`-points per direction of the mesh
- `es`: array of energies at which the density is evaluated
- `delta`: broadening of the Green's function the density is built from
- `operator`: restrict the curvature to a subspace, as for `h.get_chern()`
- `write`: if `True`, also write `CHERN_DENSITY.OUT` and `CHERN_DENSITY_INTEGRATED.OUT`

The `k`-resolved counterpart at a single energy, $\Xi(\mathbf{k}, \omega)$ over a full `k`-mesh, can be obtained with `topology.dOmega_dE_kmap(h,nk=40)`, which writes the map to `BERRY_DENSITY_KMAP.OUT`. See `examples/2d/berry_density/main.py`, `examples/2d/chern_density/main.py` and `examples/2d/berry_density_kmap/main.py`

for runnable versions, and `jupyter-notebooks/functionalities/topological_characterization/03_freq` for an executed notebook.

Berry curvature density in real-space

A Chern number needs a Brillouin zone, but the Berry curvature can also be resolved in real space, so that one sees where in the sample the topological flux is carried; this is what makes it usable on an island, a disordered sample or a supercell too large for a k-mesh. We will now see how to compute it with `topology.real_space_chern()`. The Berry curvature in real-space is defined as

$$\Omega(\mathbf{k}) = \int \Gamma(\mathbf{k}, \mathbf{r}) d^2\mathbf{r}$$

where Ω is the Berry curvature of the occupied bands and $\Gamma(\mathbf{k}, \mathbf{r})$ is the spatially-resolved Berry curvature. Note that this object is meaningful for periodic systems with very large unit cells. In pyqula, the real-space Berry curvature is obtained from a Bianco-Resta-type commutator of the position operators and the projector on the occupied states, evaluated on a large real-space (0-dimensional) supercell or island

```
from pyqula import islands
from pyqula import topology
g = islands.get_geometry(name="honeycomb", n=8, nedges=3) # a honeycomb island
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
(r,c) = topology.real_space_chern(h) # spatially-resolved Berry curvature
```

`r` are the site positions and `c` the local Berry-curvature marker at each site (the call also writes `REAL_SPACE_CHERN.OUT`, one value per site, ready to be plotted as a map over the island). The routine takes a finite Hamiltonian, and a periodic one raises a `ValueError` naming the requirement: to use it on a periodic system, build a large supercell and open it first. See `examples/0d/real_space_chern/main.py` for a runnable version and `jupyter-notebooks/functionalities/topological_characterization/04_spatially_resolved_topolog` for an executed notebook.

Chern number in real-space

The local marker of the previous section can be summed into an invariant, and we will now see how. The Chern number in real-space is defined as

$$C = \int F(\mathbf{r}) d^2\mathbf{r}$$

where C is the total Chern number of the occupied bands and $F(\mathbf{r})$ is the spatially-resolved Chern number. Note that this object is meaningful for periodic systems with very large unit cells. $F(\mathbf{r})$ is the local marker computed by `topology.real_space_chern`, and the integral above has to be read with care on a finite sample: the sum of the marker over the whole island is exactly zero, so summing over every site is not how the invariant is recovered. Instead, deep inside a large enough island, where the local environment already looks like the infinite bulk, the marker plateaus at the quantized bulk Chern number, and the edge sites carry the opposite weight that cancels it. This exact real-space cancellation is itself a manifestation of the bulk-boundary correspondence: the flux carried by the bulk is returned by the edge states

```
from pyqula import islands
from pyqula import topology
import numpy as np
g = islands.get_geometry(name="honeycomb",n=8,nedges=3) # a honeycomb island
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
(r,c) = topology.real_space_chern(h) # local marker, per site
r = np.array(r)
bulk = np.argsort(np.linalg.norm(r-r.mean(axis=0),axis=1))[:len(r)//4] # innermost sites
C = np.mean(c[bulk]) # bulk plateau value approximates the total Chern number
```

For this island the marker sums to zero to machine precision and the innermost quarter of the sites average to about 1.9, close to the $C = 2$ of the spinful Haldane model of the Chern number section, while the outermost sites are strongly negative. See [jupyter-notebooks/functionalities/topological_characterization/05_real_space_chern](#) for the same marker on an amorphous lattice, where there is no Brillouin zone to compare with.

Topological surface states

The invariant predicts states at the edge, and the direct way to see them is the spectral function of a semi-infinite system, a half-plane that is periodic along the edge and extends to infinity away from it, so that the momentum along the edge stays a good quantum number and the states localized at the boundary show up as branches crossing the bulk gap. We will now see how to compute it for the Haldane model with `kdos.surface()`

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
kdos.surface(h) # surface spectral function
```

The call writes `KDOS.OUT`, with the momentum along the edge, the energy, the surface spectral weight and the bulk spectral weight on each line, and

returns the same four arrays, flat, one entry per pair of momentum and energy, on a window of a hundred energies between -1 and 1 with a broadening `delta=0.01` by default. What you see when the surface weight is plotted against momentum and energy is the chiral edge states crossing the gap, one per unit of the Chern number, which are absent from the bulk weight: inside the gap the bulk weight is essentially zero and the surface weight is not. The same calculation is available as `h.get_surface_kdos()`, with the `energies`, `delta`, `kpath` and `operator` arguments described in the section on surface spectral functions, which also covers the quantum spin Hall case. See `examples/readme_examples/chern_surface_states/main.py` and `examples/2d/kdos_edge/main.py` for runnable versions ending in the plot.

Topological markers

The real-space Berry curvature and Chern marker of the two sections above are examples of a topological marker: a local, position-resolved quantity, computable from the ground-state projector alone, that reveals a bulk topological invariant without relying on translational symmetry or a clean Brillouin zone. This is what makes topological markers the tool for disordered systems, finite flakes and islands, or systems with spatially varying parameters, a Haldane mass that changes sign across a boundary, say, or a topological insulator with dilute vacancies, where the marker density directly shows where the invariant is carried and where it changes. See `topology.real_space_chern` above for the code that computes it, and the `05_real_space_chern_amorphous.ipynb` notebook for a system with no lattice at all.

Entanglement

Entanglement entropy and entanglement spectrum

We will now see how to compute the entanglement entropy of a region of a ground state, and its entanglement spectrum, with `h.get_entanglement_entropy()` and `h.get_entanglement_spectrum()`. The ground state of any non-interacting or mean-field Hamiltonian is a Slater determinant. For a Slater determinant the reduced density matrix of a spatial region A is the exponential of a free-fermion operator, so it is completely fixed by the one-particle correlations inside A . The whole calculation therefore reduces to diagonalizing the restricted correlation matrix $C_{ij} = \langle c_i^\dagger c_j \rangle$ ($i, j \in A$). This gives occupations $\zeta_n \in [0, 1]$, from which

$$S = - \sum_n [\zeta_n \ln \zeta_n + (1 - \zeta_n) \ln(1 - \zeta_n)], \quad \xi_n = \ln \frac{1 - \zeta_n}{\zeta_n}$$

are the entanglement entropy and the single-particle entanglement spectrum. An occupation exactly at 0 or 1 contributes nothing to the entropy, and an

occupation at $1/2$ contributes the most. This is the correlation-matrix method of Peschel: only a matrix the size of the region is ever diagonalized, never the exponentially large reduced density matrix.

The region is given as a list of site indices, a boolean mask, a callable on positions (the same convention as `sculpt`), or simply a fraction of the cells. Spin, sublattice and Nambu components are treated as extra orbitals of a site, so a region is always specified in terms of *sites*. A periodic Hamiltonian is first folded into a ring of `nsuper` unit cells, so region *A* has **two** entanglement boundaries rather than one. Its entropy is therefore twice that of a single cut, which matters when comparing with a formula written for one boundary.

The entanglement spectrum is where this becomes a topological probe. For a 2d Hamiltonian the momentum parallel to the cut remains a good quantum number, and $\xi_n(k_{\parallel})$ is the Li-Haldane entanglement spectrum. For a Chern insulator its in-gap branches flow across $\xi = 0$ and count $2|C|$ ($|C|$ chiral modes per boundary, two boundaries), mirroring the edge spectrum of the same model, while the entanglement spectrum of a trivial insulator stays gapped around $\xi = 0$. This is entirely a bulk ground-state calculation: no ribbon and no open boundary is ever constructed, and the edge states are read off from the bulk alone

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # create a honeycomb lattice
h = g.get_hamiltonian(has_spin=False) # get the Hamiltonian
h.add_haldane(0.1) # Chern insulator, C = 1
print(h.get_chern(nk=20)) # 1.0

# Li-Haldane entanglement spectrum xi_n(k_par) across the BZ
(ks,xis) = h.get_entanglement_spectrum(nsuper=10,nk=101)

# entanglement entropy, per parallel unit cell, averaged over the BZ
print(h.get_entanglement_entropy(nsuper=10,nk=20))
```

`ks` are the `nk` parallel momenta across the Brillouin zone, and `xis` has one row per momentum and one column per orbital of the region, ten here (half of a ring of ten cells, with two sites per cell). Plotting every column against `ks` shows the levels of the Chern insulator crossing $\xi = 0$; the same plot for the honeycomb lattice gapped by a sublattice imbalance shows a spectrum that never comes close to zero. The entropy is per parallel unit cell, averaged over `nk` momenta, and counts both boundaries of the region.

Optional arguments

- `region`: the region *A*, in any of the four forms above; `None` takes half of the system
- `nsuper`: number of unit cells in the ring that is cut, for a periodic Hamiltonian

- `direction`: lattice direction the cut is normal to, by default the last periodic one
- `kpar`: a single parallel momentum; `None` on a 2d Hamiltonian sweeps or averages over `nk` of them
- `nk`: number of parallel momenta
- `fermi`: occupied states are those with $E < \text{fermi}$, zero by default

Nambu/BdG Hamiltonians are handled with the full anomalous correlation matrix, whose basis doubling is divided out; `fermi` must stay at zero for them, since their chemical potential is already inside the Hamiltonian. Occupation is a hard $T = 0$ cut. A level sitting exactly at the Fermi energy makes the ground state degenerate, so the routine raises rather than silently returning the entropy of an arbitrarily chosen determinant. On a ring this happens for particular values of `nsuper` (a half-filled chain with `nsuper` divisible by four, say), and changing the ring size by one is the remedy. The normalization is pinned by the critical chain, which reproduces the $c = 1$ conformal-field-theory law $S = (c/3) \ln[(L/\pi) \sin(\pi l/L)]$ for a region of l cells in a ring of L . A gapped 2d insulator obeys the area law, and the Li-Haldane counting matches the Chern number. See `examples/1d/entanglement_entropy_chain/main.py` (the central charge from a fit of the entropy against the region size) and `examples/2d/entanglement_spectrum_haldane/main.py` (the spectrum of the Haldane model next to that of a trivial insulator) for runnable versions ending in the plots.

Response functions

Let us now see how a quantum material responds to a perturbation: a field that couples to an operator B is switched on, the change of an operator A is measured, and their ratio as a function of frequency and momentum transfer is the response function $\chi_{AB}(q, \omega)$, whose imaginary part is what a spectroscopy measures. We will build it up in three steps: the bare response of a single-particle Hamiltonian, a sum over transitions between its eigenstates; the same response dressed with the interaction of a mean-field state at the random phase approximation (RPA) level, whose poles are the collective modes, magnons and plasmons; and the Bethe-Salpeter equation, which solves the two-particle problem directly and gives the excitons of a semiconductor. The mean-field chapter feeds the last two steps.

Optical conductivity

We will start with the response to an electric field, the frequency-dependent conductivity tensor of a periodic Hamiltonian, with `h.get_optical_conductivity()`. It is computed with the Kubo-Greenwood formula, summing velocity matrix elements over a k-mesh in the Lehmann representation,

$$\sigma_{ab}(\omega) = \frac{ie^2\hbar}{N_k V_{\text{cell}}} \sum_{\mathbf{k}} \sum_{n \neq m} \frac{f_n - f_m}{E_m - E_n} \frac{v_{nm}^a v_{mn}^b}{\hbar\omega + i\eta - (E_m - E_n)}$$

where v_{nm}^a is the matrix element of the velocity along a between the bands n and m , f_n the occupation of band n and η a broadening, following the Wannier90 postw90 convention (Yates, Wang, Vanderbilt and Souza, Phys. Rev. B 75, 195121): light of frequency ω is absorbed whenever it can move an electron from an occupied to an empty band separated by $\hbar\omega$, with a weight set by the velocity matrix element. The full complex tensor is returned, so $\text{Re } \sigma_{xx}$ is the optical absorption, σ_{xy} the magneto-optical (Kerr and Faraday) response, and the $\omega \rightarrow 0$ limit of σ_{xy} the anomalous Hall conductivity, quantized to $-C e^2/h$ for a Chern insulator with Chern number C . The intraband (Drude) channel comes from the degenerate limit $(f_n - f_m)/(E_m - E_n) \rightarrow -\partial f/\partial E$ of the same sum, which also protects the formula against $0/0$ on spin-degenerate multiplets; intraband and interband switch the two channels independently.

Results are in units of $e^2/\hbar$, so one conductance quantum e^2/h is $1/(2\pi)$ of the returned value, times a^{2-d} for a d -dimensional lattice. `T` sets the temperature and `delta` the Lorentzian broadening η . The absolute normalization is fixed by the f-sum rule, $\int \text{Re } \sigma_{aa}(\omega) d\omega = \pi W_{aa}$, with W the diamagnetic weight available as `h.get_sum_rule_weight()`; the Drude weight tensor is `h.get_drude_weight()`. Let us take a Chern insulator, the honeycomb lattice with a Haldane term, and compute both its absorption and its Hall response

```
import numpy as np
from pyqula import geometry
g = geometry.honeycomb_lattice() # create a honeycomb lattice
h = g.get_hamiltonian(has_spin=False) # get the Hamiltonian
h.add_haldane(0.2) # Chern insulator
h.shift_fermi(0.3) # put the Fermi energy in the gap

(ws,s) = h.get_optical_conductivity(energies=np.linspace(0.,4.,100),
                                   nk=40,T=0.02,delta=0.05)
absorption = s[:,0,0].real # Re sigma_xx, the optical absorption

# the DC Hall response is quantized to the Chern number
(w0,s0) = h.get_optical_conductivity(energies=[0.],nk=40,T=0.01,delta=1e-3)
print("sigma_xy(0) in e^2/h:",2.*np.pi*s0[0,0,1].real) # -1
```

`ws` holds the frequencies and `s` the 3×3 tensor at each of them. What you see in the absorption is zero below the gap and an onset at it; the second call evaluates the tensor at zero frequency with a small broadening, and the off-diagonal element comes out at one conductance quantum with the sign of the Chern number. The velocity operator behind the formula is the current operator built from the full bond vectors, including the bonds inside the unit cell, so that the tensor has the symmetry of the lattice: on the honeycomb

lattice $\sigma_{xx} = \sigma_{yy}$ to machine precision, and graphene reproduces its universal absorption $\pi e^2/4h$. Superconducting (Nambu) and 3d Hamiltonians raise `NotImplementedError`. See `examples/2d/optical_conductivity/main.py` and `examples/1d/optical_conductivity_chain/main.py` for runnable versions.

Charge-charge response function

Let us now look at the simplest response function, the one of the density at site i to a potential at site j , with `h.get_chi()`. The charge-charge response function for a spinless system is computed as

$$\chi(\omega, i, j) = \sum_{n,m} f(\epsilon_n)(1 - f(\epsilon_m)) \frac{\Psi_n(i)\Psi_m(j)\Psi_m^*(i)\Psi_n^*(j)}{\epsilon_n - \epsilon_m - \omega + i\delta}$$

where $f(\epsilon)$ is the Fermi-Dirac distribution, so that only transitions from an occupied state n to an empty state m contribute, weighted by the amplitudes of both states at the two sites and resonating at the transition energy. For a periodic system the response is computed at a momentum transfer q , summing over the k-mesh

```
from pyqula import geometry
g = geometry.chain() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
(es, chis) = h.get_chi(q=[0.,0.,0.]) # get response function
```

`es` holds the frequencies and `chis` one complex number per frequency, the response traced over the sites of the unit cell, whose imaginary part is what inelastic scattering measures. Note that at $q = 0$ the response of the chain vanishes identically, since with a single band there is no state to scatter into at the same momentum; the scan below, at finite q , is where something appears.

Optional arguments - `q`: momentum transfer of the response function - `energies`: array of frequencies - `delta`: broadening - `nk`: number of k-points used in the Brillouin-zone integration

The charge-charge response can also be scanned over momentum transfer to build a $\chi(q, \omega)$ map

```
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian()
qs = np.linspace(-1.,1.,40)
chis = [h.get_chi(q=[q,0.,0.],energies=np.linspace(-3,3,100),nk=40,delta=0.1)[1] for q in qs]
```

What you see in that map is the particle-hole continuum of the band, the energies a scattering from k to $k + q$ across the Fermi level can cost. See `examples/1d/charge_response/main.py` for a runnable version.

Generic operator-operator response function

The same machinery works for any pair of operators. `h.get_chi()` computes the charge-charge response by default, but any two operators A and B can be used instead, giving the generalized response

$$\chi_{AB}(\omega, q) = \sum_{k,n,m} f(\epsilon_{k,n})(1 - f(\epsilon_{k+q,m})) \frac{\langle \Psi_{k,n} | A | \Psi_{k+q,m} \rangle \langle \Psi_{k+q,m} | B | \Psi_{k,n} \rangle}{\epsilon_{k,n} - \epsilon_{k+q,m} - \omega + i\delta}$$

where the perturbation couples through B , the measurement is of A , and the sum runs over the occupied states at k and the empty ones at $k + q$. Taking both operators to be the spin along z gives the longitudinal spin susceptibility of the chain

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian(has_spin=True)
sz = h.get_operator("sz") # any of the operators from the "Operators" section
(es, chi) = h.get_chi(A=sz, B=sz, energies=np.linspace(-2., 2., 100), nk=40, delta=0.1)
```

By default $A=B$ =identity, which recovers the charge-charge response above. Any operator from the “Operators” section (spin, valley, sublattice, location, Nambu...) can be plugged in to build the corresponding susceptibility.

RKKY response function

Let us now use that response to couple two magnetic impurities. A moment placed in a metal polarizes the conduction electrons around it, and a second moment at some distance feels that polarization: the result is an effective exchange coupling between the two, the RKKY (Ruderman-Kittel-Kasuya-Yosida) interaction, which follows from the same non-interacting response function. `rkky.rkky_map` computes it between a reference site and every other site in the system, as a function of distance

```
from pyqula import geometry
from pyqula import rkky
g = geometry.chain()
h = g.get_hamiltonian()
h.add_onsite(1.)
m = rkky.rkky_map(h, n=10, mode="LR", nk=200) # linear-response RKKY vs distance
```

Optional arguments - mode: "LR" for linear response, the same response function `get_chi` uses, or "pm" (“poor man’s”), which adds a small magnetic perturbation at each site and measures the change in total energy - n: how many neighboring cells/distances to compute - nk: number of k-points used in the Brillouin-zone integration

`m` is an array whose columns are (distance, ..., ..., RKKY energy); the RKKY energy is in the last column

```
x,e = m[:,0],m[:,3]
```

What you see is a coupling that alternates in sign with the distance and decays, with a period set by the Fermi wavevector, which is why the snippet shifts the Fermi energy with the onsite term first.

See `examples/1d/RKKY/main.py` and `examples/1d/rkky_minimal/main.py` (which compares "pm" and "LR" on the same system) for runnable versions.

Spin susceptibility and RPA

The methods above compute the bare response, the one of non-interacting electrons. We will now see how the response of an interacting system is obtained at the RPA level, with `h.get_spinchi_ladder()` and `h.get_spinchi_full()`. The idea is that the perturbation also changes the mean field the electrons feel, which moves more electrons, and summing that series gives $\chi_{RPA} = \chi(1-U\chi)^{-1}$: the bare response dressed with the Hubbard U stored on a mean-field Hamiltonian (`h.V`, see “Interactions at the mean-field level”). For a magnetically ordered state this is what carries the magnons, which the bare response does not have. Let us converge an antiferromagnetic chain and ask for its transverse spin response at a small momentum transfer

```
from pyqula import geometry
import numpy as np
g = geometry.bichain() # two sites per cell, so Neel order fits in the cell
h = g.get_hamiltonian(has_spin=True)
seed = h.copy() ; seed.add_antiferromagnetism(0.5) # symmetry-breaking seed
hmf = h.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf=seed,nk=100) # magnetic state
(es,chis) = hmf.get_spinchi_ladder(energies=np.linspace(0.,2.,100),q=[0.1,0.,0.],nk=40,delta=0.001)
```

What you see in the imaginary part of `chis` is a sharp peak at the magnon energy at that q , which is what inelastic neutron scattering measures in a magnet; without spin-orbit coupling the peak goes to zero energy at $q = 0$, the Goldstone mode of the broken spin-rotation symmetry.

- `get_spinchi_ladder` computes the transverse (S^+/S^-) response, the spin-wave-like excitations
- `get_spinchi_full` computes the full (S_x, S_y, S_z) tensor response
- `RPA=True` (default) dresses the response with the interaction; `RPA=False` returns the bare response
- `h.get_qdos_iets` scans `get_spinchi_full` over a q -path instead of a single q , giving the spin-excitation dispersion along high-symmetry directions

The "G", "K", "M" path labels need a 2D lattice; on the antiferromagnetic honeycomb lattice the dispersion map along the path is

```

g2 = geometry.honeycomb_lattice() # already two sublattices per cell
h2 = g2.get_hamiltonian(has_spin=True)
hmf2 = h2.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf="antiferro")
qdisp = hmf2.get_qdos_iets(energies=np.linspace(0.,2.,60),qpath=["G","K","M"],nq=30,nk=20,de

```

The real-space counterpart is `h.get_iets_ldos`, which computes the response at a single energy resolved in space, a map of the spin-flip signal that inelastic tunneling spectroscopy (IETS) sees with an STM; combined with `h.get_ldos` it gives the elastic and inelastic STM maps side by side.

Both snippets depend on the mean-field state actually being ordered, which is the first thing to check, with `hmf.get_vev("sz")`, before reading anything into the excitation spectrum: an RPA calculation on top of an unpolarized reference state runs perfectly happily and tells you nothing about magnons. Two things decide it. The unit cell must be able to hold the order: a one-site `geometry.chain()` cell cannot represent Neel order at all, and seeding it with `mf="antiferro"` converges to exactly zero moment, while `bichain` and `honeycomb_lattice` both have the two sublattices. And `U` must be past the ordering transition: on the honeycomb lattice at half filling `U=2` gives a moment of 0.002, meaning none, while `U=3` gives 0.44. Note also that the cost of `get_qdos_iets` is the product of its `nq`, `nk` and energy-grid sizes, so it grows quickly: the grid above takes well under a minute, while a 80x40x100 one is closer to an hour.

See `examples/1d/rpa/main.py` (RPA spin response vs `q` for an antiferromagnetic chain), `examples/2d/rpa_triangular/main.py`/`examples/2d/rpa_honeycomb/main.py` (`get_qdos_iets` dispersion along a `q`-path) and `examples/0d/rpa_island/main.py`/`examples/0d/rpa_finite` (`get_iets_ldos` real-space IETS maps) for runnable versions, and the executed notebooks `jupyter-notebooks/functionalities/interacting_mean_field_hamiltonians/09_rpa_beyond` and `10_rpa_collective_modes.ipynb` for what the spectra look like.

Running the response on a GPU All of these RPA functions, `get_spinchi_ladder`, `get_spinchi_full`, `get_qdos_iets`, `get_iets_ldos`, `get_rpa_kernel_poles`, `get_magnon_bands(method="rpa")` and the density-channel `get_densitychi_RPA`, bottom out in the same Lindhard kernel, a sum over pairs of eigenstates at every `k`-point whose cost grows as the fourth power of the number of sites in the cell. Calling `gpu.set_gpu(True)` first runs that kernel on a GPU instead, which pays off once the cell holds more than a few sites (see “Running on a GPU”). The pair basis, which the same functions take for an interaction that couples different sites, has its own kernel on the device and follows the same switch, so the route the interaction picks does not change whether the calculation runs on a GPU.

The precision of that sum is chosen with `chi_prec`. On the GPU it is `"single"` by default, since the double-precision arithmetic of a consumer card can be an order of magnitude slower than its single-precision one, and `chi_prec="double"` switches it back; the CPU takes both values too, with `"double"` as its default.

The eigenstates are obtained in double precision either way and the response comes back as double-precision numbers, so what single precision changes is only the rounding of the sum, which leaves the bare response within about 10^{-7} of the double-precision one

```
from pyqula import geometry
from pyqula import gpu
import numpy as np
gpu.set_gpu(True) # run the GPU kernels of the package on the device
g = geometry.honeycomb_lattice() # two sites per cell
h = g.get_hamiltonian() # first-neighbor hopping
hmf = h.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf="antiferro",nk=10) # Neel mean field
es = np.linspace(0.01,1.0,50) # frequencies
(es,chi) = hmf.get_spinchi_full(q=[0.2,0.,0.],nk=10,energies=es,delta=0.05,
                               chi_prec="double") # the response in double precision
```

Note that the RPA dressing amplifies that rounding close to an instability or a Goldstone mode, where $1-U\chi$ is nearly singular: at $q = 0$, next to the Goldstone mode of the Neel antiferromagnet above, the dressed response differs from the double-precision one by a few parts in 10^4 , so a calculation that needs the dressed response at a pole to more digits than that should use `chi_prec="double"`.

RPA kernel poles and magnon bands

Let us now find the collective modes directly, with `h.get_rpa_kernel_poles()` and `h.get_magnon_bands()`. The RPA-dressed response diverges wherever the kernel $1 - U\chi(q, \omega)$ becomes singular, and those poles are the collective modes of the system, spin waves (magnons) and plasmons; a kernel eigenvalue crossing zero at $\omega = 0$ instead is a Stoner or RPA instability. `h.get_rpa_kernel_poles` scans a frequency window at a fixed q and returns every such pole, for any pair of operators A, B and any interaction matrix V , defaulting to the charge channel and $q = 0$ like the other generic response functions above

```
from pyqula import geometry
import numpy as np
g = geometry.bichain() # two sites per cell, so Neel order fits in the cell
h = g.get_hamiltonian()
seed = h.copy() ; seed.add_antiferromagnetism(0.5) # symmetry-breaking seed
hmf = h.get_mean_field_hamiltonian(U=3.0,nk=100,mf=seed,filling=0.5)
N = len(g.r) # the response matrix is one entry per site
V = 3.0*np.identity(N) # charge-channel interaction, in site space
poles = hmf.get_rpa_kernel_poles(V=V,q=[0.1,0.,0.],
                                energies=np.linspace(0.,4.,200),delta=2e-2,nk=40)
```

Two things there are easy to get wrong. As above, the mean field needs a unit cell that can hold the order being sought, which is why `bichain` and not `chain` is used, and the converged state has `hmf.get_vev("sz")` equal and opposite on the two sublattices. And V must live in the same space as the response

matrix: `get_rpa_kernel_poles` defaults to the charge channel, one entry per site, so V is $N \times N$. It is not `hmf.V`, which is the mean-field interaction in spin-orbital space ($2N \times 2N$) and raises a dimension error here. For the spin channel use `get_magnon_bands` below, which builds the S_x, S_y, S_z vertex from `hmf.V` itself.

`poles` is an `(npoles, 2)` array, one row per collective mode found: the pole frequency and its residual imaginary part. The latter is signed (it is the kernel eigenvalue's actual imaginary part at the crossing, which can lie on either side of the real axis), so judge how sharp a mode is by its magnitude: a small `abs(gamma)` means a sharp mode, a large one means it is heavily damped or the crossing is numerical noise.

`h.get_magnon_bands` does this in the full S_x, S_y, S_z spin channel, taking the interaction from the converged mean field, and scans along a q-path, so that the result is the magnon dispersion of the magnetically ordered state

```
qs,ws,gammas = hmf.get_magnon_bands(nq=40,energies=np.linspace(0.01,3.,200),delta=2e-2,nk=40)
```

Different q-points can have different numbers of poles, so `qs`, `ws` and `gammas` come back as flat arrays of equal length, ready for a scatter plot. `qs` is the index of the q-point along the path, the same convention `get_bands` uses; `ws` is the pole frequency and `gammas` its residual imaginary part, and filtering on `np.abs(gammas) < threshold` keeps only the sharp branches. What you see for the antiferromagnetic chain is an acoustic branch going to zero at $q = 0$, the Goldstone mode, and an optical one above it. See `examples/1d/magnon_bands/main.py` for a runnable version.

Interactions beyond onsite

The V passed to `get_rpa_kernel_poles` is not restricted to a single onsite matrix. It can also be a real-space hopping-like dictionary `{(n1,n2,n3): matrix}`, keyed by lattice-vector offset in the same convention as `h.get_hopping_dict()`, for an interaction with support beyond the same unit cell. It is Fourier-transformed to $V(q)$ at whatever q the response is evaluated at, using the same Bloch-phase convention as the Hamiltonian's own hoppings, so that an extended interaction is dressed exactly like an extended hopping. For a nearest-neighbor V_1 on a chain this gives the expected $V(q) = 2V_1 \cos 2\pi q$, meaning that the interaction is repulsive at $q = 0$ and attractive at the zone boundary

```
import numpy as np
from pyqula import geometry
g = geometry.chain()
h = g.get_hamiltonian(has_spin=False)
N = h.intra.shape[0] # one site per cell here
I = np.identity(N)
V = {(0,0,0): 0.0*I, (1,0,0): 0.6*I, (-1,0,0): 0.6*I} # nearest-neighbor V1
```

```
poles = h.get_rpa_kernel_poles(V=V,q=[0.5,0.,0.],energies=np.linspace(0.,2.,60),
                              delta=2e-2,nk=200)
```

Note the channel: `get_rpa_kernel_poles` dresses the charge response by default, so a dictionary `V` here must be a density-density interaction in site space, of the same dimension as the response matrix `chiAB(...,mode="matrix")` returns. It is not the place to put a spin vertex.

What happens in the spin channel when the interaction couples two different sites? A vertex with one index per site carries the Hartree part of such an interaction, but not its Fock part, which acts on the electron-hole pair index (i, j) and has nowhere to go in that basis. For a neighbor-shell V_{ij} that Fock part is all there is in the spin channel, so on a ferromagnetic chain ordered by V_1 alone the site vertex vanishes and no magnon comes out at all; for an exchange J_1 it is a correction of a few percent to the dispersion at finite q . So `get_spinchi_full`, `get_spinchi_ladder`, `get_iets_ldos` and `get_magnon_bands` use the site vertex only for an interaction that couples each site to itself, a Hubbard U , where it is exact for the transverse response, and sum the ladder in the pair basis of `method="pair"` (below) for anything that couples different sites, in the same cell or in different ones. The call is the same either way, with the same `energies`, `delta`, `nk` and temperature `T` (equal to `delta` unless given); what the pair basis does not have is the Nambu basis, which is refused there rather than computed with the site vertex. With `q` left out, the response is averaged over the k-mesh after dressing each q , which is the local RPA response.

The three magnon routes

`h.get_magnon_bands` takes a `method`, and the three cover different interactions because they keep different amounts of the ladder:

interaction	"rpa"	"pair"	"tdhf"
onsite Hubbard U	yes	yes	yes
neighbour-shell density-density $V_1, V_2, \dots$	through the pair basis	yes	yes
exchange $J_1, J_2, \dots$ (isotropic or anisotropic)	through the pair basis	yes	yes
metallic reference	yes	yes	with <code>metal=True</code>
non-collinear (canted, spiral) state	yes	yes	yes
frequency-resolved $\chi(\omega)$	yes	yes	no (an eigenproblem)

The middle row splits because of which index the interaction is diagonal in. Writing $H_{int} = \frac{1}{2} \sum V_{ij} n_i n_j$, the transverse rung is $K_{(ij),(kl)} = -V_{ij} \delta_{ik} \delta_{jl}$, diagonal in the pair index (i, j) rather than in the site index. An onsite U collapses this onto the pairs with $i = j$, which is exactly what the site vertex of `method="rpa"` works with, and that is why it is exact there. An extended V_{ij} lives in the pairs with $i \neq j$, which that basis simply does not contain, and this is why `method="rpa"` hands such an interaction to the pair basis.

`method="pair"` solves the same ladder in the pair index, where V is diagonal. Only the pairs the interaction actually connects enter, so a short-ranged V costs $N(z+1)$ rather than N^2 . It keeps the frequency scan, needs no gap, and assumes nothing about the spin structure of the state. Let us take the honeycomb lattice with both an onsite U and a first-neighbor V_1 , converge its antiferromagnetic state, and ask for the magnons and the transverse response

```
import numpy as np
from pyqula import geometry
from pyqula.meanfield import VJinteraction
nk = 6 # the SCF and the magnon share this mesh
g = geometry.honeycomb_lattice()
hmf = VJinteraction(g.get_hamiltonian(), U=3.0, V1=0.5, filling=0.5,
                    mf="antiferro", nk=nk, maxerror=1e-10).hamiltonian
qs, ws, gammas = hmf.get_magnon_bands(method="pair", nq=20,
                                       energies=np.linspace(1e-3, 3., 400), delta=1e-3, nk=nk)
es, chi = hmf.get_transverse_spinchi(energies=np.linspace(0., 2., 100),
                                     q=[0.1, 0., 0.], delta=1e-2, nk=nk) # (Sx, Sy, Sz) x site chi(w)
```

The kernel has two terms, an exchange one and a Hartree one. For a state with a single spin quantization axis the Hartree term drops out of the transverse response, and keeping only the exchange term is exact. For a non-collinear state the two spin sectors mix, dropping the Hartree term breaks $SU(2)$, and the acoustic branch comes out spuriously gapped. Both terms are therefore kept, which keeps the Goldstone mode in place for a canted or spiral state as well.

The last row of the table is what `"tdhf"` is for: it gives the magnon energies as eigenvalues, with no frequency grid and no broadening, which is what the Goldstone residual is measured on. `"pair"` and `"tdhf"` agree with each other for every interaction in the table, and in a metal both reproduce the exact saturated-ferromagnet dispersion; `"rpa"` agrees with them for an onsite U , and for anything longer ranged it is `"pair"`.

What about exchange? An exchange interaction $J \vec{S}_i \cdot \vec{S}_j$ is not a density-density one: its Ising part $J S_i^z S_j^z$ is, but its transverse part $J/2(S_i^+ S_j^- + \text{h.c.})$ flips a spin on each site, and no matrix of the form $V_{ij} n_i n_j$ can write it. The way the mean field handles it is by writing $S_i^x S_j^x$ and $S_i^y S_j^y$ as the same Ising term in two rotated spin frames, and the self-consistent calculation records the three spin channels on the Hamiltonian it returns. `"pair"` and `"tdhf"` read them

and build the kernel of each channel in its own frame, which is exactly the derivative of that mean field, so the transverse part is included and the Goldstone mode sits at zero for an isotropic J as it does for U . This requires a Hamiltonian from `VJinteraction` or from `get_mean_field_hamiltonian` with $J1/J2/J3/Jr$, which are the ones that record the channels.

The site vertex would get this only partly right. It keeps the Goldstone mode for an isotropic J , but the exchange bonds also enter through a Fock term on the pair index, as V_1 does: on the antiferromagnetic honeycomb lattice with $J_1 = 3$ at $\mathbf{nk}=6$ it puts the acoustic magnon at $q = 0.1$ at 1.3296, against 1.3687 from "pair" and "tdhf", which a brute-force reference confirms. This is why "rpa" sums an exchange interaction in the pair basis.

The single-axis couplings $SzSz$, $SxSx$ and $SySy$ record their channel too, as an anisotropic exchange with the other two at zero, so all three routes take them. A global spin rotation of a converged Hamiltonian keeps an isotropic exchange exact, but it would turn an anisotropic one into an interaction with $S_i^x S_j^y$ -like cross terms that the three channels cannot hold, so `h.global_spin_rotation()` refuses it, unless the rotation is about the axis of a uniaxial exchange; rotate the Hamiltonian before the mean-field calculation instead.

Magnons from time-dependent Hartree-Fock

You can think of a magnon as the same kind of object as an exciton: a bound two-particle excitation of a mean-field state, made of electron-hole pairs whose electron and hole have opposite spin rather than the same spin. So the Bethe-Salpeter equation of the exciton sections below already describes it, and `method="tdhf"` is that equation restricted to the spin-flip pairs. Because it works in the pair index, it carries a neighbor-shell interaction properly

```
import numpy as np
from pyqula import geometry
from pyqula.meanfield import VJinteraction
nk = 6 # the SCF and the magnon share this mesh
g = geometry.honeycomb_lattice()
scf = VJinteraction(g.get_hamiltonian(),U=3.0,V1=0.5,filling=0.5,
                    mf="antiferro",nk=nk,maxerror=1e-10)
hmf = scf.hamiltonian
print(hmf.get_goldstone_residual(nk=nk)) # 2e-10: the mode is exact
qs,es = hmf.get_magnon_bands(method="tdhf",nk=nk,nq=20,n=2)
```

`qs,es` are flat 1D arrays in the same convention as the RPA `get_magnon_bands` above (`qs` is the integer index along the q -path), and `es` is complex whenever the mean-field state is unstable against some excitation. `h.get_magnon_energies(Q=...)` gives the spectrum at a single momentum.

The check that this is right is the Goldstone theorem. A state that orders magnetically without spin-orbit coupling breaks $SU(2)$ spontaneously, so a uni-

form spin rotation costs no energy and there must be a magnon at exactly zero energy at $Q = 0$. `h.get_goldstone_residual()` measures how far the calculation is from that. It comes out proportional to the tolerance the mean field was converged to and to nothing else, roughly 10^{-10} for a mean field converged to `maxerror=1e-10`, so check it before reading any dispersion. It is deliberately not “the eigenvalue nearest zero”, which converges much more slowly; a small imaginary part of that size on the acoustic branch is expected rather than a problem.

Three things matter before using it. The first is that the mean field and the magnon must use the same `nk`: a mean field converged at `nk=20` and a magnon solved at `nk=4` is not self-consistent on the magnon’s mesh, and the acoustic branch picks up a large spurious gap. The Hamiltonian does not record the mesh it was converged on, so `get_goldstone_residual` is how to catch this.

The second is that a metallic reference needs `metal=True`. By default the same number of bands is taken as occupied at every k-point, which a metal does not have; `metal=True` decides the occupied and empty sets separately at each k-point instead, which is what an itinerant magnet needs, and in particular a ferromagnet ordered by a neighbour-shell V_1 alone. It changes nothing for a gapped reference, so it is safe to leave on when unsure. Two things change once it runs. The magnon sits inside the Stoner continuum and is no longer the lowest mode, so `h.get_magnon_bands(method="tdhf", metal=True, by="weight")` picks branches by how much spin character they carry rather than by energy. And $E(q)$ is even in q only if the occupied set is symmetric under $k \rightarrow -k$, which a finite mesh need not make it: otherwise the $+q$ and $-q$ magnons genuinely differ. Choose the mesh so the occupied set is symmetric.

The third is where the interaction came from. An isotropic exchange ($J_1/J_2/J_3/J_r$) is taken with its transverse part $J/2(S_i^+ S_j^- + \text{h.c.})$, through the spin channels the self-consistent calculation recorded, as described in “The three magnon routes” above, and it has its Goldstone mode like any other interaction. Two cases raise `ValueError` instead, each with a message saying why. The first is a hand-built exchange matrix: it carries only an Ising term and no record of its spin channels, which is what an isotropic exchange that lost its transverse part would look like too, and solving that would give an ordinary-looking dispersion gapped by of order J where zero is required. The second is an anisotropic exchange (J_{1x}, J_{1y}, J_{1z} unequal, or the single axis of $S_z S_z, S_x S_x$ and $S_y S_y$), which breaks spin rotation symmetry explicitly, meaning that its magnon gap is real and there is no Goldstone mode to check; `check_su2=False` computes that spectrum, with every channel included, and it is the same for $S_z S_z, S_x S_x$ and $S_y S_y$ at the same J .

For a collinear state only the spin-flip pairs are needed, which is exact and much cheaper, and that is the default. A canted or spiral state has no such block, and the whole pair basis is used instead without your having to ask (`channel="all"` forces it). This covers genuinely non-collinear states such as the 120-degree spiral of the triangular-lattice Hubbard model, whose Goldstone

mode survives to the tolerance the mean field was converged to. On a plain onsite Hubbard U, where both this and `method="rpa"` are exact, the two agree.

See `examples/2d/magnon_bands_tdhf/main.py` for a runnable version.

Density (charge) response

The same dressing can be done in the charge channel, with `h.get_densitychi_RPA()` and `h.get_plasmon_bands()`, the density-density analogs of `get_spinchi_full` and `get_magnon_bands`: the collective modes are now the plasmons, or the charge-order modes, of a $V_1/V_2/V_3$ neighbor-shell (plus onsite U, plus a general $V_r(r)$) density-density interaction, in the same convention as `Vinteraction/VJinteraction`. Unlike the spin-channel functions, they take the interaction directly as parameters instead of reading it from `h.V`, so no mean-field convergence is needed first: they dress the bare susceptibility of whatever Hamiltonian is passed in, which can also be an already-converged one if the RPA response about that reference state is wanted

```
import numpy as np
from pyqula import geometry
g = geometry.chain()
h = g.get_hamiltonian(has_spin=True) # 1D chain at half filling
qs,ws,gammas = h.get_plasmon_bands(V1=0.6,qpath=[[0.3,0.,0.],[0.4,0.,0.],[0.5,0.,0.]],nq=3,
                                   energies=np.linspace(0.,1.,100),delta=2e-2,nk=2000)
```

`qs`, `ws` and `gammas` follow the convention of `get_magnon_bands`. What matters here is the wavevector: a 1D chain at half filling has perfect Fermi-surface nesting at $q = \pi$, which strongly enhances the static charge susceptibility there, so a repulsive V_1 drives the charge channel towards an instability at the zone boundary, the charge-channel analog of the Stoner instability of the spin channel above.

Excitons and the Bethe-Salpeter equation

Let us now go beyond a response at fixed frequency and solve the two-particle problem itself, with `h.get_bse()` and `h.get_exciton_energies()`. A mean-field band structure can only ever absorb above its gap, but the electron and the hole that absorbing a photon creates attract each other, the pair binds, and the bound state, the exciton, appears below the gap by its binding energy: the peak below the absorption edge of a semiconductor. The Bethe-Salpeter equation (BSE) diagonalizes the electron-hole pair Hamiltonian directly, which gives the exciton energies and the electron-hole amplitudes that say what each exciton is made of.

The exciton at center-of-mass momentum Q is written as a superposition of the mean field's own transitions,

$$|X\rangle_Q = \sum_{v,c,k} A_{vc}(k) c_{c,k+Q}^\dagger c_{v,k} |MF\rangle$$

where v runs over the valence bands, c over the conduction bands and k over the mesh, and the BSE is the eigenvalue problem for the amplitudes $A_{vc}(k)$, with a kernel made of a direct term (the screened electron-hole attraction, which binds) and an exchange term (which splits singlet from triplet and, on its own, reproduces the RPA). The formalism is the localized-orbital (“point-like orbitals”) BSE of the Xatu code, arXiv:2307.01572, solved here in its full non-Tamm-Dancoff form.

By default the interaction is read straight off a converged mean-field Hamiltonian (`h.V`), so the same interaction that generated the Fock self-energy inside `h` also generates the BSE kernel and nothing is double counted. This is time-dependent Hartree-Fock on top of Hartree-Fock: let us converge a gapped antiferromagnet and ask for its lowest excitons

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=True)
h.add_sublattice_imbalance(0.6)
hmf = h.get_mean_field_hamiltonian(U=1.5,filling=0.5,mf="antiferro",nk=6) # converge a mean
es = hmf.get_exciton_energies(nk=6,n=8) # the eight lowest excitons
```

A purely onsite Hubbard U is far too short ranged to bind a Wannier-Mott exciton in two dimensions, so realistic exciton calculations want a long-ranged interaction instead. `bsetk.interaction.density_interaction` builds one from the same $U/V1/V2/V3/Vr$ parameters `Vinteraction` uses, and it can be handed to any of the exciton methods through `V=`; with a Coulomb tail on a gapped honeycomb lattice the lowest exciton detaches from the band edge, and the binding energy says by how much

```
import numpy as np
from pyqula.bsetk.interaction import density_interaction
h2 = g.get_hamiltonian()
h2.add_sublattice_imbalance(1.0) # a gapped semiconductor
W = density_interaction(h2,Vr=lambda r1,r2: 0.8/np.sqrt((r1-r2).dot(r1-r2)+0.25)) # Coulomb
bse = h2.get_bse(V=W,nk=8)
print(bse.get_energies()[0], bse.get_binding_energies()[0]) # lowest exciton, and how far b
```

- `h.get_exciton_energies` returns the exciton energies, `h.get_exciton_binding_energies` how far below the lowest independent-particle transition each one lies (positive means bound), and `h.get_exciton_states` both the energies and the amplitudes $A_{vc}(k)$
- `h.get_bse` returns the full solved object, whose `pairs` attribute holds the k -mesh, the band window and the `(ik,iv,ic)` label of every pair index, so the amplitudes can be resolved in momentum or by band

- `Q=[qx,qy,qz]` gives the excitons at finite center-of-mass momentum, and `h.get_exciton_bands` scans that over a q-path to give the exciton band structure (below); `nv/nc` restrict the calculation to the `nv` highest valence and `nc` lowest conduction bands
- `kernel="full"` (default) uses both kernel terms; `"direct"` is the ladder alone (no singlet/triplet splitting), `"exchange"` is exactly the RPA, and `"none"` collapses the spectrum onto the bare transition energies
- `tda=True` applies the Tamm-Dancoff approximation, diagonalizing only the resonant block: four times smaller and Hermitian, and a good approximation at weak coupling

Since the Hamiltonian is spinful, spin is simply part of the orbital index, so singlet and triplet excitons come out of a single calculation with no separate spin channel, and the exchange term is what splits them. On a spin-rotation-invariant reference the lowest transition starts out four-fold degenerate, and the full kernel resolves it into a three-fold triplet with the singlet pushed up above it, which neither kernel term produces on its own.

One caveat on the default `V=None`: `h.V` is only the z channel of an exchange interaction, so it misses the x and y channels of an anisotropic run (J_{1x}/J_{1y} alongside J_{1z}), and after `SxSx/SySy` it is zero, the coupling being recorded in the x or y channel. In either case build the interaction explicitly and pass it as `V=` instead.

The size of the problem is $N_{pair} = n_v n_c N_k$, and the matrix is dense and $2N_{pair}$ square, so the k-mesh is the expensive knob: `max_memory` (default 2 GB) refuses a calculation that would not fit rather than letting it exhaust memory. A gapped reference state is required. A metallic filling has no well-defined electron-hole pair basis and is rejected, as are Nambu/BdG Hamiltonians, whose two-particle structure is different.

Large k-meshes

How fine a k-mesh an exciton needs is set by how tightly it is bound. A strongly bound exciton is spread out over the Brillouin zone and is already converged on a coarse mesh. A weakly bound Wannier-Mott one has an envelope $A(k)$ peaked sharply at the band edge, and only a fine mesh resolves that peak. `solver=` says how to reach one

```
import numpy as np
from pyqula import geometry
from pyqula.bsetk.interaction import density_interaction
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
h.add_sublattice_imbalance(1.0) # a gapped semiconductor
W = density_interaction(h,Vr=lambda r1,r2: 0.8/np.sqrt((r1-r2).dot(r1-r2)+0.25))
h.get_bse(V=W,nk=1024,tda=True,solver="iterative",neig=4)
h.get_bse(V=W,nk=65536,tda=True,solver="qtt",neig=1)
```

- `solver="dense"` (default): gives the whole exciton spectrum, and is the only one that works without the Tamm-Dancoff approximation. Good up to a few thousand electron-hole pairs
- `solver="iterative"`: same energies, much finer meshes, but only the `neig` lowest excitons
- `solver="qtt"`: for the very fine meshes a weakly bound exciton needs. Returns the lowest exciton only

Both "iterative" and "qtt" need `tda=True`, the Tamm-Dancoff approximation, which is accurate for the weakly coupled systems where bound excitons live. "qtt" is the specialist: it is at its best on a fine mesh over a narrow band window, which is exactly a Wannier-Mott exciton, and it is the clear winner in 1D. In 2D, use it on a primitive cell with `nv/nc` kept small; on a supercell its advantage disappears and "iterative" is the better choice. It also wants `nk` to be a power of two, and cannot take an RPA-screened interaction directly; pass one with `ScreenedInteraction.get_dict()`.

See `examples/1d/exciton_qtt/main.py` and `examples/2d/exciton_qtt/main.py` for runnable comparisons of the three solvers, the second plotting the exciton envelope $|A(k)|^2$ over the Brillouin zone, and `examples/2d/excitons_bse/main.py` for the lowest exciton of a gapped honeycomb lattice detaching from the absorption edge as the Coulomb tail is turned up.

Exciton band structure

An exciton is a two-particle state, so besides its binding energy it has a dispersion of its own: the bound electron-hole pair propagates with a center-of-mass momentum Q , and $E_X(Q)$ is the exciton band structure. It is not the difference of two band energies, since the electron-hole interaction bends it, so its curvature is the exciton's effective mass, and a flat exciton band means a strongly bound, spatially compact exciton. `h.get_exciton_bands` solves one BSE per q-point along a path and returns the result in the same flat form `get_bands` uses

```
# h2 and W as in the section above
import numpy as np
from pyqula import geometry
from pyqula.bsetk.interaction import density_interaction
g = geometry.honeycomb_lattice()
h2 = g.get_hamiltonian()
h2.add_sublattice_imbalance(1.0) # a gapped semiconductor
W = density_interaction(h2, Vr=lambda r1,r2: 0.8/np.sqrt((r1-r2).dot(r1-r2)+0.25))
# nv=nc=2 keeps both members of each spin-degenerate pair
opts = dict(V=W, nq=20, nk=8, nv=2, nc=2, n=4)
qs, es = h2.get_exciton_bands(**opts) # the four lowest excitons along the path
qs0, es0 = h2.get_exciton_bands(kernel="none", **opts) # the bare continuum
```

`qs` holds the integer index of the q-point along the path and `es` the exciton

energy, both flat 1D arrays ready for a scatter plot; `n` keeps only the `n` lowest excitons at each `q`-point and every other argument is passed straight to `get_bse`. `qpath` takes the same input as `get_bands`, a list of high-symmetry labels or of explicit `q`-vectors, and `Q` is not restricted to the `k`-mesh, since the pair basis diagonalizes at `k` and `k + Q` independently. Running it a second time with `kernel="none"` is the same call with the kernel construction skipped, and gives the bottom of the electron-hole continuum to plot the exciton band against, which is what makes the binding visible: the exciton band runs below the continuum all along the path. If the mean-field reference is unstable against some excitation at some `q`-point, `es` comes back complex there rather than silently losing the imaginary part. The cost is `nq` full dense diagonalizations, so `nv/nc` and `tda=True` are the knobs that make a long path affordable, and `parallel.set_cores` parallelizes over the path.

One trap is invisible at $Q = 0$: `nv/nc` must not cut a degenerate multiplet in half. Every band of a spinful Hamiltonian with no spin-orbit coupling and no magnetic order is two-fold degenerate. Setting `nv=1` there keeps an arbitrary state out of that degenerate pair, and the exciton energies inherit the arbitrariness: on a time-reversal-symmetric model, $E_X(Q)$ visibly stops being even in Q . The library warns when the window splits a multiplet; use an even `nv/nc` on a spin-degenerate Hamiltonian.

See `examples/2d/exciton_bands/main.py` for a runnable version: the exciton band of a gapped honeycomb lattice, plotted below the electron-hole continuum of the same model.

The screened interaction

Everything above uses the bare interaction in both kernel terms, which makes the BSE time-dependent Hartree-Fock. But an electron and a hole added to a solid do not feel the bare interaction: the other electrons rearrange around them, and what survives that rearrangement is the screened interaction. Since the mean-field step has already left the bands $\{e_n(k), C^n(k)\}$ on a `k`-mesh, the static RPA screening can be computed from them directly rather than postulated,

$$\chi_{ab}^0(q) = \frac{1}{N} \sum_k \sum_{n,m} (f_{nk} - f_{m,k+q}) \frac{\rho_a^{nm}(k, q) \rho_b^{nm*}(k, q)}{e_{nk} - e_{m,k+q}}, \quad \rho_a^{nm}(k, q) = C_a^{n,k*} C_a^{m,k+q}$$

$$\varepsilon(q) = 1 - v(q) \chi^0(q), \quad W(q) = \varepsilon^{-1}(q) v(q)$$

with a, b running over the spin-orbitals of the unit cell, in the same point-like-orbital approximation the rest of the BSE uses. `screening="rpa"` computes this and puts W in the direct (ladder) term

```
import numpy as np
from pyquala import geometry
```

```

from pyqula.bsetk.interaction import density_interaction

h = geometry.honeycomb_lattice().get_hamiltonian()
h.add_sublattice_imbalance(1.0) # a gapped semiconductor

# a BARE interaction: an onsite term plus a soft-cutoff Coulomb tail
coulomb = lambda r1,r2: 0.6/np.sqrt((r1-r2).dot(r1-r2)+0.25)
V = density_interaction(h,U=1.0,Vr=coulomb)

```

```

bare = h.get_bse(V=V,nk=8) # time-dependent Hartree-Fock
screened = h.get_bse(V=V,nk=8,screening="rpa") # GW-BSE style

```

What you see when comparing the two is a weaker binding in the screened calculation, the exciton moving closer to the band edge, since the medium partly cancels the attraction between the electron and the hole.

The exchange term keeps the bare interaction whatever **screening** is set to. This is the standard GW-BSE split rather than an oversight, since screening the exchange term too would count the same bubbles twice. One consequence is that **kernel="exchange"** is unaffected by **screening**, and a warning is raised if both are given.

A fitted Hubbard U must not be screened. A U chosen to reproduce a material is already an effective, screened interaction, and screening it again double counts and gives a spuriously weak interaction. Use screening for a genuinely bare interaction: a long-range Coulomb tail from **density_interaction(Vr=...)**, or bare $V1/V2/V3$ shells. The dangerous case is the default $V=h.V$ taken from a Hubbard calculation. This is a different question from combining an RPA W with a BSE ladder, which is the standard GW-BSE construction and is not double counting.

screening="crpa" is the constrained variant, which leaves the transitions inside the **nv/nc** BSE band window out of the polarization (arXiv:0710.4013). That is the right choice when the band window is being treated as a downfolded model to be solved exactly afterwards, and it screens strictly less than the full RPA. It needs a genuine subset of the bands: with the default **nv=nc=None** the window is the whole spectrum, nothing is left outside it to do the screening, and the call is refused rather than silently returning the bare interaction.

The screened interaction is also available on its own, as a **ScreenedInteraction**

```

W = h.get_screened_interaction(V=V,nk=8)
print(W.epsmin) # smallest dielectric eigenvalue over the mesh
Wq = W.at(W.qs[3]) # the screened interaction at one q-point
d = W.get_dict() # ... and back in real space, usable at any q

```

$W(q)$ is not short ranged, so unlike the other interactions in this guide it lives only on the mesh it was computed on, which is exactly the set of q the kernel needs. Asking for it at any other q raises rather than silently using the

nearest point. `get_dict()` gives the real-space version, which can be evaluated anywhere, inspected to see how far the screened interaction reaches, or passed to `get_mean_field_hamiltonian(V=...)` for a screened-exchange mean field. `nkW` sets a finer mesh for the screening than for the exciton, and must be an integer multiple of `nk`.

Where the dielectric matrix is built matters. Screening is a property of the charge channel: what polarizes the medium is the total density, and what the medium's induced charge acts back on is again the total density. So `channel="charge"` (the default) builds ε on site indices,

$$\varepsilon_{ij}(q) = \delta_{ij} - \sum_k v_{ik}^c(q) \chi_{kj}^c(q), \quad \chi_{ij}^c = \sum_{\sigma\sigma'} \chi_{(i\sigma)(j\sigma')}^0, \quad v_{ij}^c = \frac{1}{4} \sum_{\sigma\sigma'} v_{(i\sigma)(j\sigma')}$$

and adds the resulting correction, which is spin-independent, to the bare interaction:

$$W_{(i\sigma)(j\sigma')} = v_{(i\sigma)(j\sigma')} + [v^c \chi v^c]_{ij}, \quad \chi = \chi^c (1 - v^c \chi^c)^{-1}$$

This is the standard GW construction. One convention matters here: off site $v_{ij}^c = V_{ij}$, but on site $v_{ii}^c = U/2$ rather than U , because a Hubbard term couples only opposite spins, so only half of a site's own density acts on a given electron.

Building the dielectric matrix in the charge channel keeps spin-rotation invariance exact: the $S_i^z S_j^z$ part of W is left exactly as the bare interaction had it. That is why it is the default. The alternative, `channel="orbital"`, dresses the full spin-orbital matrix as $W = \varepsilon^{-1} v$. It screens the charge and spin channels with a single density-density kernel and therefore breaks SU(2): an exciton multiplet that should stay degenerate splits visibly in the orbital channel and stays degenerate in the charge one. Use the default. The two channels coincide exactly for a spinless Hamiltonian.

Screening does not always weaken the interaction. With no onsite term the interaction matrix has zero trace, and screening then enhances it rather than reducing it, in the same way an RPA kernel enhances a magnetic instability. A Coulomb tail with no onsite term is simply missing its largest matrix element, so include a realistic U .

Finally, if an eigenvalue of $\varepsilon(q)$ actually reaches zero, the RPA has diverged: that is a charge or spin instability of the mean field at that wavevector, the same $1 - V\chi = 0$ condition `h.get_rpa_kernel_poles` reports as a collective mode, and the call raises rather than returning a huge number.

The parameters, in short:

- `screening` picks the interaction in the direct term: `None` for the bare one, `"rpa"` for the screened W , `"crpa"` for the constrained version when the band window will be solved exactly afterwards

- `channel` picks where the dielectric matrix is built. Leave it at "charge", which keeps spin-rotation invariance exact
- `nkW` is the mesh the screening is computed on. Raise it above `nk`, by an integer factor, when the screening needs more k-points than the exciton does

See `examples/2d/screened_bse/main.py` for a runnable version.

Quantum transport

We will now see how to perform quantum transport calculations with `pyqula`, starting from the conductance of the simplest junctions between two leads and reaching the current through a voltage-biased superconducting one.

A transport calculation puts a scattering region between two semi-infinite leads, each of them a periodic Hamiltonian of the kind built in the previous chapters, and asks how much current flows through it. The leads enter through their surface Green's functions, obtained with the same decimation as in the surface spectral functions, and the quantity computed is the differential conductance dI/dV , what a two-terminal measurement gives: for normal leads it is the Landauer transmission, in units of e^2/h per open channel, and when a lead is superconducting it includes Andreev reflection, the process in which an electron arriving from the normal side is reflected as a hole while a Cooper pair enters the superconductor.

Magnetoresistance in metal-metal transport

As a specific example, here we will address how to compute the magnetoresistance of a junction between two magnetic metals, with `HT.didv()`. We build two copies of the same lead, give each one an exchange field pointing in a different direction, and compare the conductance of the parallel and antiparallel configurations

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
es = np.linspace(-.5,.5,50) # set of energies for dIdV
Gs = dict()
for name,m2 in [("parallel",[0.,0.,0.5]),("antiparallel",[0.,0.,-0.5])]:
    h1 = h.copy() ; h1.add_exchange([0.,0.,0.5]) # first lead, fixed magnetization
    h2 = h.copy() ; h2.add_exchange(m2) # second lead, parallel or antiparallel
    HT = heterostructures.create_leads_and_central(h1,h2,h1) # create the junction
    Gs[name] = [HT.didv(energy=e) for e in es] # calculate conductance
```

The magnetoresistance follows from the two conductance curves, $MR =$

$(G_P - G_{AP})/G_{AP}$ evaluated at the Fermi energy. In the parallel configuration each spin channel sees the same bands on both sides of the junction and transmits perfectly, so the conductance at zero energy is 2, one per spin; in the antiparallel one a spin-up electron of the first lead arrives at a lead whose spin-up band is shifted the opposite way, the mismatch reflects part of it, and the conductance is smaller. For the chain above the effect is a few percent, since an exchange of 0.5 is small compared with the bandwidth of 4. See `jupyter-notebooks/functionalities/quantum_transport/01_metal_metal_transport.ipynb` for the executed notebook.

Superconductor-metal transport

Here we address how transport between a superconducting lead and a metallic lead is computed, with `heterostructures.build()` and `HT.set_coupling()`. As a paradigmatic example, we will focus on the Andreev reflection regime and the tunneling regime. Below the gap an electron from the normal lead cannot enter the superconductor as a quasiparticle: it can only be reflected as a hole, leaving a Cooper pair behind, and since that process transfers a charge $2e$ it doubles the conductance of a transparent contact below the gap. At a tunneling contact Andreev reflection is suppressed, and the conductance instead follows the density of states of the superconductor, zero inside the gap and with coherence peaks at $\pm\Delta$. The loop over the transparency T goes from one limit to the other

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
h1 = h.copy() # first lead
h2 = h.copy() # second lead
h2.add_swave(.01) # the second lead is superconducting
es = np.linspace(-.03,.03,100) # set of energies for dIdV
for T in np.linspace(1e-3,1.0,6): # loop over transparencies
    HT = heterostructures.build(h1,h2) # create the junction
    HT.set_coupling(T) # set the coupling between the leads
    Gs = [HT.didv(energy=e) for e in es] # calculate conductance
```

G_s is the conductance curve at each transparency, and comparing it against its value outside the gap is the natural way to read it: at $T=1.0$ the sub-gap conductance is twice the normal one, and at the lowest transparencies it vanishes inside the gap and peaks at the gap edge. This is the spectroscopy that a normal tip on a superconductor gives, and the transparency is the knob an experiment controls through the tip-sample distance. See `examples/transport/andreev_reflection/main.py` for a runnable version, and `jupyter-notebooks/functionalities/quantum_transport/02_metal_superconductor_transport.ipynb` for the executed notebook.

Transport through an arbitrary finite region

The two examples above build the central scattering region out of copies of the leads' own unit cell. Let us now see how any finite (0d) Hamiltonian can act as the central region instead, contacted by two semi-infinite 1D chain leads attached at sites i and j , with `h.get_central_heterostructure(i,j,left=None,right=None)`

```
from pyqula import geometry

g = geometry.chain()
gc = g.get_supercell(5)
gc.dimensionality = 0 # a finite, 5-site cluster (no periodicity)
hc = gc.get_hamiltonian() # the central region, can be any 0d Hamiltonian

h_normal = g.get_hamiltonian() # normal lead
h_sc = g.get_hamiltonian(); h_sc.add_swave(0.05) # superconducting lead

ht = hc.get_central_heterostructure(0,4,left=h_normal,right=h_sc)
G = ht.didv(energy=0.02) # Andreev conductance, via the BdG scattering-matrix formula
```

It returns a plain `Heterostructure`, so `didv`, `get_dos`, `get_kappa` and the rest all work on it. The exception is `landauer`, which refuses a junction with a Nambu degree of freedom: the Landauer formula counts single-particle transmission, while a Cooper pair carries charge $2e$. Use `didv`, which applies the BdG scattering formula, in that case. `left` and `right` default to a plain spinless chain and j to the last site. At most one of `hc`, `left` and `right` may carry a pairing amplitude, so a superconducting central region between two normal leads is fine, and so is one superconducting lead, but two superconducting leads raise a `ValueError`: there is then no normal lead left to define a reflection amplitude against, and `get_dc_current`, addressed below, is the right tool instead. Only 0d central regions are supported so far. See `examples/transport/central_region_ij/main.py` for a runnable script, and `jupyter-notebooks/functionalities/quantum_transport/03_finite_central_region_two_leads.ipynb` for the executed notebook.

Landauer transmission through a real-space device

`heterostructures` builds a junction out of two periodic Hamiltonians. Let us now see the other route, with `multiterminal.Device`: the leads and the scattering region are given as *geometries*, the hoppings between them are generated from the actual interatomic distances, and the Landauer transmission follows from the leads' self-energies. It is the natural form for a disordered or irregularly-shaped conductor, where there is no unit cell to repeat.

```
import numpy as np
from pyqula import geometry
from pyqula import multiterminal
```

```

def lead_cell(x): # a one-site chain unit cell sitting at x
    g = geometry.chain() ; g.r = np.array([[x,0.,0.]]) ; g.r2xyz()
    return g

gc = geometry.chain().supercell(4) # the scattering region
xs = gc.r[:,0]
d = multiterminal.Device()
d.biterminal(left_g=lead_cell(min(xs)-1.),right_g=lead_cell(max(xs)+1.),
             central_g=gc,disorder=0.0)
print("ballistic transmission:",d.transmission(energy=0.0)[0])

```

A perfectly matched chain has one open channel and nothing to scatter off, so the transmission is 1 everywhere inside the band, to the tolerance of the lead decimation. Raising `disorder` puts random on-site energies in the central region and the transmission drops below 1 by an amount that depends on the realization drawn, meaning that a disorder-averaged result needs several draws. `d.transmission(energy=e)` returns a list with one entry per lead pair, the pair (0,1) by default; `multiterminal.landauer(d,energy,ij=[(i,j)])` is the same quantity as a plain function, with the pairs named explicitly. The same disorder physics done with the junction machinery, a chain of disordered cells between two clean leads, is in `examples/transport/disordered_transport/main.py`.

Multiple Andreev reflection and AC-Josephson current

`didv` and `landauer` above are equilibrium, zero-bias linear-response quantities. Let us now see what happens when a finite voltage is applied across a junction between two superconductors, an SNS junction, with `HT.get_dc_current()`. A finite bias makes each lead's pairing phase wind in time, giving rise to multiple Andreev reflections (MAR), in which a quasiparticle bounces between the two superconductors gaining eV at each crossing until it reaches the gap edge, and to an AC Josephson effect; the physically meaningful, measurable quantity is the time-averaged (DC) current $I_{dc}(V)$. `Heterostructure.get_dc_current(voltage)` computes it with the Floquet-Keldysh formalism of San-Jose, Cayao, Prada and Aguado, *New J. Phys.* **15**, 075019 (arXiv:1301.4408): the bias is gauged away from the (static) leads into a single, periodically time-dependent “weak link” hopping, and the resulting Floquet-space Dyson/Keldysh equations are solved to get $I_{dc}(V)$. It works for any combination of normal and superconducting leads, including the case of **two** superconducting leads, which the scattering-matrix formula behind `didv` cannot handle on its own (it has no normal lead to define a reflection amplitude against).

`didv` reaches the same formalism through a `method` argument, so a conductance can be had without differentiating I_{dc} by hand. `method="smatrix"` is the zero-temperature BdG scattering-matrix conductance, `method="keldysh"` differentiates `get_dc_current` at the bias `energy`, and the default `method="auto"`

picks Keldysh when both leads are genuinely superconducting and the scattering matrix otherwise.

The two are different physical quantities rather than two ways of getting the same one, because the bias is applied differently: "smatrix" grounds the normal lead and drops the whole bias on the other, while "keldysh" biases both leads rigidly. So on a junction with one normal lead they disagree by an $O(1)$ factor, and they keep disagreeing as the other lead's pairing is taken to zero. Let `method="auto"` choose.

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
h1 = h.copy() ; h1.add_swave(0.1) # left superconducting lead
h2 = h.copy() ; h2.add_swave(0.1) # right superconducting lead
HT = heterostructures.build(h1,h2) # create the SNS junction
HT.set_coupling(0.5) # set the normal transparency of the weak link
vs = np.linspace(0.02,1.5,40)*0.1 # bias voltages
Is = HT.get_iv_curve(vs) # MAR/AC-Josephson dc current
```

`Is` is the dc current at each bias, the curve a current-biased measurement of an SNS junction traces out. This is among the most expensive calculations in the guide, at roughly fifteen seconds per bias point for the SNS junction above, so a forty-point curve takes minutes. The reason is physical: the number of Andreev reflections that matter is the MAR order $2\Delta/eV$, which grows as the bias falls, and each one costs a Floquet sideband. The sidebands are added automatically until I_{dc} converges, but a grid reaching very close to zero bias can exhaust `nmax_max` and warn that the sidebands did not converge. That point is inaccurate; nothing has crashed. Starting the grid a little higher is the cheap fix; raising `nmax_max` the expensive one.

Only 1D leads are supported. A central region between the leads (`heterostructures.build(h1,h2,central=[hc])`, for example a quantum dot) works too, more slowly. The bias drops across the junction's rightmost bond, so the central region sits at the **left** lead's potential; detune it with `hc.shift_fermi(eps)`, which keeps it a valid BdG Hamiltonian, rather than by adding to its diagonal. See `examples/transport/floquet_keldysh_mar/main.py` for a runnable script.

A `LocalProbe` is an STM-like tip weakly coupled to one site of an infinite sample, with the tip and the sample site playing the role of the two leads. The same choice of method applies: the scattering matrix by default, and the Floquet-Keldysh MAR current when tip and sample are both superconducting, which is the situation of a superconducting tip on a superconducting sample. On a `LocalProbe` the two methods do agree when the tip is normal, since the tip is grounded there exactly as the scattering matrix grounds it.

```

from pyqula import geometry
from pyqula.transporttk.localprobe import LocalProbe

g = geometry.chain()
h = g.get_hamiltonian() ; h.shift_fermi(1.) ; h.add_swave(0.1) # SC sample
lead = geometry.chain().get_hamiltonian() ; lead.shift_fermi(1.) ; lead.add_swave(0.1) # SC
lp = LocalProbe(h,lead=lead,delta=1e-3)
lp.T = 0.3 # reference transparency
G = lp.didv(energy=0.25,nmax=4,nmax_max=12,tol=5e-2) # routed through Keldysh automatically
k = lp.get_kappa(energy=0.25,nmax=4,nmax_max=12,tol=5e-2)

```

This is considerably more expensive than the normal-probe case (each `didv` or `get_kappa` call runs several Floquet-Keldysh sideband sweeps), especially deep below the combined gap at low transparency, where the sideband sum converges slowly; see `examples/transport/decay_constant_keldysh/main.py` for a runnable script using a coarse energy grid and a modest sideband cutoff to keep the runtime reasonable, and `jupyter-notebooks/functionalities/quantum_transport/08_tunneling_con` and `09_mar_ac_josephson_sc_probe.ipynb` in the same folder for the executed notebooks on the normal and the superconducting probe.

`get_kappa` also accepts a `temp` argument for a thermally-averaged kappa (each conductance entering the power-law fit is `didv(temp=...)`'s thermal average rather than the zero-temperature value). Pass a single `energy` (returns a scalar, as above) or a whole `energies=[...]` array at once (returns an array):

```
k = lp.get_kappa(energies=[0.1,0.25,0.4],temp=0.02,nmax=4,nmax_max=12,tol=5e-2)
```

`Heterostructure.get_kappa` takes the same `temp` and `energies` arguments.

`didv`'s `energy` and `energies` arguments are mutually exclusive, the same convention `get_kappa` uses: pass a single `energy` and get a scalar back, or a whole `energies=[...]` array at once and get an array.

```

es = np.linspace(0.15,0.25,40)*0.1 # bias energies
Gs = HT.didv(energies=es, nmax_max=40) # dI/dV curve

```

Single defects in infinite systems

Let us now see how a single point defect or impurity in an otherwise infinite, periodic system is treated, with `embedding.Embedding`. A plain supercell calculation cannot do it without periodizing the defect, so that the impurity sees its own replicas; the Green's function embedding technique instead keeps the infinite pristine crystal and modifies a single cell. It takes the pristine periodic Hamiltonian `h` together with a modified cell `m` describing the defect, and gives the observables of the infinite system as perturbed by that single defect. The defect itself is written with the same site-dependent onsite energy of the first chapter

```

from pyqula import geometry
from pyqula import embedding
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # pristine, infinite Hamiltonian
hv = h.copy()
hv.add_onsite(lambda r: 1.0 if r[0]<0.01 else 0.0) # a single-site onsite defect
eb = embedding.Embedding(h,m=hv) # embed the defect in the infinite system
(x,y,d) = eb.get_ldos(energy=0.0,delta=1e-2,nsuper=200,nk=400) # LDOS around the defect

```

`get_ldos` returns the real-space positions and the LDOS profile in a window of `nsuper` unit cells around the defect, and what you see in it is how the infinite system responds to one impurity: Friedel oscillations decaying away from it in a metal, or a bound state and in-gap states induced by the impurity in a gapped system, the profile that STM measures around a single adatom or vacancy (it also writes `LDOS.OUT`; pass `write=False` to suppress that). `eb.get_dos()` gives the total DOS, `eb.multildos()` scans the LDOS over many energies (written to a `MULTILDOS/` folder), `eb.get_gf()` returns the embedded Green's function itself (with `operator=` it returns AG , the same convention `get_ldos` uses), and `eb.get_didv()` computes transport through the embedded defect. See `examples/embedding/single_impurity_1D/main.py` and `examples/embedding/honeycomb_vacancy/main.py` for runnable versions, the other scripts under `examples/embedding/` for further defect scenarios (vacancies, boundaries, Yu-Shiba-Rusinov states, self-consistent defects...), and `jupyter-notebooks/functionalities/spectral_functions/04_single_impurities_infinite.ipynb` for the executed notebook. The momentum-space picture of the same defect, the quasiparticle interference, is addressed in the Observables chapter.

Wannierization

We will now see how to build a smaller tight-binding model out of a chosen set of bands of a Hamiltonian, with `h.get_wannier_hamiltonian()`.

A band, or a group of bands, that is separated from the rest by a gap can be described by localized orbitals centered on the unit cells, the Wannier functions, whose hoppings reproduce that band subspace. Wannierization is the construction of those functions with the smallest possible spread, and what it returns is a new tight-binding Hamiltonian with as many orbitals per cell as bands were selected, that can be used like any other Hamiltonian in this guide. `h.get_wannier_hamiltonian()` Wannierizes a contiguous range of a periodic Hamiltonian's bands and returns a new, smaller multicell Hamiltonian whose real-space hoppings reproduce that band subspace on the wannierization k-mesh. By default the range is taken as a fixed subspace, Wannierized jointly as one group, and reproduced exactly; passing a `num_wann` smaller than the range instead disentangles, which is the subject of the second subsection below.

As an example, consider a staggered honeycomb lattice, where a sublattice po-

tential opens a gap and gives a genuinely dispersive valence band to Wannierize

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)
h.add_onsite([0.8,-0.8]) # sublattice potential opens a gap

# Wannierize just the lowest (valence) band: bands=[0,0] selects band
# index 0 at every k-point of a 12x12 Monkhorst-Pack wannierization mesh
hwan = h.get_wannier_hamiltonian(bands=[0,0],nk=12)

print("Number of Wannier functions:",hwan.intra.shape[0])
print("Wannier centres (Cartesian):\n",hwan.wannier_centres)
print("Wannier spreads:",hwan.wannier_spreads)
print("Total spread Omega:",hwan.wannier_spread_total)
```

One band was selected, so the Wannierized model has a single orbital per cell, and its centre and spread are the ones the minimization arrived at: the total spread Ω is what a maximally localized Wannier function has made as small as it can. The Wannierized Hamiltonian `hwan` behaves like any other `pyqula` Hamiltonian, so its bands can be compared directly against the original model's

```
(k,e) = h.get_bands(write=False)
(kw,ew) = hwan.get_bands(write=False)
```

The two coincide on the wannierization mesh, and the Wannier model interpolates smoothly between the mesh points, which is the point of doing this: a one-orbital model that carries the valence band of the two-orbital one. See `examples/wannier/get_wannier_hamiltonian/main.py` for a runnable version, and `jupyter-notebooks/functionalities/wannierization/01_maximally_localized_wannier.ipynb` and `02_exact_reproduction.ipynb` in the same folder for the executed notebooks.

Symmetry-enforced Wannierization

Let us now see how the point-group symmetry of the lattice is enforced on the Wannier functions, with `symmetries="auto"`. Passing it makes `get_wannier_hamiltonian` check, before Wannierizing, that the selected band range is a genuine union of point-group-related multiplets everywhere on the mesh, with the point group auto-detected from the geometry and the Hamiltonian. A band selection that instead slices through a symmetry-related degeneracy is rejected with a `ValueError` rather than silently returning a mis-symmetrized model. A list of explicit `symmetrytk.pointgroup.SymmetryOperation` can be passed instead of "auto" to enforce a specific subgroup.

A good illustration is kagome's flat band: it is exactly degenerate with the dispersive middle band at the K point, so no selection containing only the flat

band is a union of whole multiplets. This is the topological obstruction behind kagome's flat band having no symmetric localized Wannier function, and the check catches it instead of returning a broken model

```
from pyqula import geometry
from pyqula.symmetrytk import pointgroup

g = geometry.kagome_lattice()
h = g.get_hamiltonian(has_spin=False)

try:
    h.get_wannier_hamiltonian(bands=[2,2],nk=12,symmetries="auto")
except ValueError as e:
    print("Flat band alone correctly rejected:",str(e).splitlines()[0])

# the full 3-band manifold has no such obstruction
hwan_sym = h.get_wannier_hamiltonian(bands=[0,2],nk=12,symmetries="auto")
print("Symmetries enforced:",[c.op.name for c in hwan_sym.wannier_symmetries])
```

The first call raises, and the message names the degeneracy it found; the second returns a three-orbital model whose Wannier functions transform into each other under the operations listed in `wannier_symmetries`. See `examples/wannier/get_wannier_hamiltonian/main.py` and `examples/wannier/symmetric_wannierization/main.py` for runnable versions of these two examples, and `jupyter-notebooks/functionalities/wannierization/03_notebook.py` for the executed notebook.

Disentanglement

A fixed band range only works when the range is separated from everything else by a gap across the whole Brillouin zone. Let us now see what to do when it is not, with `num_wann`. Graphene's two p_z bands are the simplest counterexample: they touch at K, so neither of them is a smooth subspace on its own. Souza-Marzari-Vanderbilt disentanglement (Phys. Rev. B 65, 035109) handles that case by extracting `num_wann` optimally connected states out of a larger set of *offered* bands, k-point by k-point. It is switched on by passing a `num_wann` smaller than the selected range, in which case `bands=[a,b]` only says which bands are offered.

```
import numpy as np
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)

# one Wannier function out of both pz bands, with the deep part of the
# valence band frozen (kept exactly rather than optimized)
hwan = h.get_wannier_hamiltonian(bands=[0,1],num_wann=1,nk=12,
                                dis_froz_max=-1.0,cutoff=0.0)
```

```

print("Wannier functions:", hwan.wannier_num_wann)
print("window used:", hwan.wannier_disentanglement_window)

f0, fw = h.get_hk_gen(), hwan.get_hk_gen()
k = [2./12., 1./12., 0.] # a wannierization-mesh k-point
e0 = np.sort(np.linalg.eigvalsh(f0(k))) ; ew = np.linalg.eigvalsh(fw(k))
print("frozen state reproduced to:", abs(ew[0]-e0[0]))

```

What is reproduced changes with it, and this is the trade rather than a loss of accuracy: a disentangled Hamiltonian reproduces the states inside the **frozen inner window** (`dis_froz_min/dis_froz_max`) exactly at every mesh k-point, and deliberately does not reproduce the selected bands outside it. Outside the frozen window the extracted subspace is a different, smoother one, meaning that the last line above prints a number at machine precision for the frozen valence state while the conduction band is not in the model at all. Pass `cutoff=0.0` when testing that exactness, as above: the default `cutoff=1e-6` drops small real-space hoppings and with them the last few digits. An **outer window** (`dis_win_min/dis_win_max`) narrows which bands are offered at each k-point, so the number available varies across the mesh, which is the point of a window rather than a band range; every eigenvalue of the result then lies inside it. `dis_num_iter` (default 200) bounds the Ω_I minimization; not reaching convergence within it is a warning rather than an error, since the frozen window is reproduced either way.

The returned Hamiltonian carries `wannier_num_wann` and `wannier_disentanglement_window` (the four window values actually used, or `None` when not disentangling) alongside the centres and spreads.

Combinations that would silently do nothing raise instead: a `dis_*` window without a smaller `num_wann` (disentanglement is switched on by the band count exceeding `num_wann`, so the window would be accepted and ignored), `dis_froz_min` without `dis_froz_max`, or a `num_wann` outside `1..len(bands)`. Disentanglement is not implemented together with a Nambu/BdG Hamiltonian, with `symmetries=`, or with `auto_split_clusters=True`, each naming the combination. One gap in that guard is worth knowing about: `win_keywords=` is by design an unchecked passthrough merged after the check, so a window smuggled in through it can still reach the engine and be ignored; use the real `dis_froz_max=` argument, which is guarded. See [jupyter-notebooks/functionalities/wannierization/04_dimensionality_and_bdg.ipynb](#) for the executed notebook on Wannierizing in other dimensionalities and in the Nambu basis.

Chebyshev kernel polynomial (KPM) methods

Let us now see what to do when a system is too large to diagonalize. Exact diagonalization builds the full spectrum, and for a system with millions of sites that is neither possible nor needed: an observable such as the density of states is a trace, and a trace can be estimated without ever knowing the eigenstates. This is the idea of the kernel polynomial method, which expands the density of states in Chebyshev polynomials of the Hamiltonian, obtains the expansion coefficients (the moments) by repeatedly multiplying a random vector by the sparse Hamiltonian, and averages over a few random vectors, a stochastic estimate of the trace. Passing `mode="KPM"` to `h.get_dos()`, the mode listed in the density of states section, switches to this expansion, which never builds the full spectrum and scales to systems with millions of sites on a single core. A sparse Hamiltonian (`is_sparse=True`) is the natural setting, since the method only multiplies vectors by the Hamiltonian and never needs the dense matrix, although a dense one is accepted too

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
g = g.get_supercell(3000) # a big supercell
g.dimensionality = 0
h = g.get_hamiltonian(is_sparse=True, has_spin=False)
(x,y) = h.get_dos(mode="KPM",
                  energies=np.linspace(-3.0,3.0,200), # energies
                  delta=1e-4, # effective smearing (~1/npol)
                  ntries=10 # number of random vectors for the stochastic trace
                  )
```

`x` is the energy grid and `y` the density of states on it, the same quantity that exact diagonalization would give for a chain of this length. Two numbers set what you get. `delta` is the energy resolution of the expansion, and the number of polynomials is the spectral width of the Hamiltonian divided by `delta`, so a sharper density of states costs proportionally more matrix-vector products; `ntries` is the number of random vectors in the stochastic trace, and the noise in `y` goes down as more of them are averaged.

The same expansion also gives non-local correlators and Green's functions without inverting a matrix, through the lower-level `kpm` module

```
from pyqula import kpm
(x,y) = kpm.dm_ij_energy(h.intra,npol=200,i=0,j=9,ne=1000)
```

`kpm.dm_ij_energy()` returns the correlator between sites i and j resolved in energy, $\pi \sum_n \psi_n(i) \psi_n^*(j) \delta(E - E_n)$, the spectral part of the Green's function $G_{ij}(E)$: for $i = j$ it is π times the local density of states at that site, and for two different sites it integrates to zero over the energy, since the eigenstates form a complete basis, and its sign along the energy axis tells in which part of the

spectrum the two sites are correlated. `npol` is the number of polynomials and sets the resolution, as `delta` did above.

See `examples/0d/kpm_dos/main.py` and `examples/0d/kpm_correlator/main.py` for runnable versions, including a comparison of the KPM correlator against the exact Green's function calculation, and the notebooks in `jupyter-notebooks/functionalities/chebyshev_kpm_algorithms/` for the local and non-local spectral functions and for a system too large to diagonalize.

The Chebyshev moments can be computed on a GPU instead, by passing `kpm_cpugpu="GPU"` to any of the KPM functions:

```
(x,y) = h.get_dos(mode="KPM",
                  energies=np.linspace(-3.0,3.0,200),
                  delta=1e-4,ntries=10,
                  kpm_cpugpu="GPU") # Chebyshev moments on a GPU
```

When no GPU is available the moments fall back to the CPU, so the same script runs on both.

Classical spin models

Up to now every model in this guide has been quantum, a Hamiltonian of fermionic operators to diagonalize. Let us now turn to classical spins, unit vectors sitting on the sites of a lattice, with `classicalspin.SpinModel`. Each spin is parametrized by a pair of angles (θ_i, ϕ_i) , and the spins interact through a real-space tensor exchange $\vec{S}_i \cdot J_{ij} \cdot \vec{S}_j$ and an optional Zeeman field. Like the `LatticeGas` of the next chapter, the model takes its lattice and neighbor shells from a `Geometry` but is otherwise independent of the quantum `Hamiltonian`: the energy is evaluated directly from the angles, and the ground state is found by minimizing it, a local gradient-based minimization started several times from random angles, rather than by diagonalization. Only the Γ point is supported, meaning that the spin configuration repeats with the unit cell you give it, so an incommensurate texture such as a spiral needs an explicit supercell large enough to hold it.

Let us take the simplest frustrated case, a triangular lattice with an antiferromagnetic first-neighbor exchange, where the three bonds of a triangle cannot all be satisfied at once:

```
from pyqula import geometry
from pyqula import classicalspin

g = geometry.triangular_lattice() # geometrically frustrated lattice
g = g.get_supercell(3)

sm = classicalspin.SpinModel(g) # classical spin model on this geometry
```

```
sm.add_heisenberg(Jij=[1.0]) # first-neighbor antiferromagnetic exchange
sm.minimize_energy(tries=10) # multistart local minimization

mx,my,mz = sm.get_magnetization() # per-site magnetization components
```

`mx`, `my` and `mz` are the components of the unit vector on each site. What comes out is the 120-degree state: every pair of neighboring spins forms an angle of 120 degrees (their scalar product is $-1/2$ on every bond), the three spins of each triangle add up to zero, and so does the total magnetization, at an energy of $-3J$ per site. The 3×3 supercell is what makes this possible, since the pattern has three sublattices; on the primitive cell the only Γ -point configuration is the ferromagnetic one. The `tries` keyword is the number of random starting points of the minimization, and it matters for a frustrated model, which has many local minima: the lowest energy found over the restarts is the one kept.

`add_heisenberg` builds shell-based isotropic (or, via `Jm=[Jx,Jy,Jz]`, diagonally anisotropic, XXZ) couplings the same way `g.get_hamiltonian(tij=...)` does, so a list of exchanges is read as first, second, third neighbors. For other forms of coupling, `classicalspin.generating_functions(name=...)` returns ready-made two-point coupling functions: "Linear" (dipolar $1/r^3$), "RKKYTI" (RKKY on a topological-insulator surface, PRB 81 233405), "ZZ" and "XYZ" (Ising and anisotropic-diagonal), "DM" (Dzyaloshinskii-Moriya). These go into `add_tensor(fun)` (couplings within the home cell) or `add_tensor_2d(fun,ncells=...,vspiral=...)`, which also sums the periodic images and can twist the coupling tensor by a per-image angle to embed a spin-spiral wavevector. `get_local_energy()` gives the per-site energy, for a spatially resolved map of a skyrmion or a domain wall, and `classicalspintk.align.most_perp_basis()` rotates a texture into the frame where it is mostly in-plane, which is the frame to draw it in with arrows.

See `examples/classicalspin/` for runnable versions, including the 120-degree state above drawn as arrows together with its local energy (`perpendicular/`) and a ladder with a modulated exchange whose texture is then fed into a quantum Hamiltonian (`modulated_ladder/`).

Lattice gas models

We will now see how to model classical particles that either occupy a site or do not, with `latticegas.LatticeGas`. The degrees of freedom are occupations $n_i \in \{0, 1\}$, adsorbates on a surface, vacancies in a crystal, or any classical binary order parameter, and they interact through a real-space coupling $J_{ij}n_in_j$ and a site-dependent chemical potential μ_in_i , so that a positive J_{ij} is a repulsion between two occupied sites. As the spin model of the previous chapter, it takes its lattice and neighbor shells from a `Geometry` but is otherwise independent of the quantum `Hamiltonian`: the energy is evaluated directly from the occupation array, and the ground state is searched by Monte Carlo, swapping the

occupations of pairs of sites with the Metropolis rule at a given temperature, rather than by diagonalization.

Let us take a triangular lattice with a third of the sites occupied and a repulsion between first, second and third neighbors, so that the particles have to arrange themselves as far from each other as they can:

```
from pyqula import geometry
from pyqula import supercell
from pyqula import latticegas

g = geometry.triangular_lattice()
g = supercell.turn_orthorhombic(g)
g = g.get_supercell(10)
g.dimensionality = 0

lg = latticegas.LatticeGas(g,filling=1./3.) # 1/3 of the sites randomly occupied
lg.add_interaction(Jij=[1.,1.,1.]) # first, second and third neighbor repulsion
es = lg.optimize_energy(temp=0.5,ntries=1e4) # simulated annealing
```

The filling fixes the number of particles, which the swap moves conserve. `lg.den` holds the current 0/1 occupation array and `es` the energy along the anneal, which decreases and then flattens once the particles have found their arrangement; `temp` sets how often an uphill swap is accepted, and `ntries` the number of attempted swaps. What that arrangement is can be read from `get_local_energy()` and `get_local_mu()`, the per-site energy and chemical-potential contribution of the current snapshot, and from `get_correlator()` and `get_structure_factor()`, the density-density correlator in real and in reciprocal space, whose peak locates the ordering wavevector of an ordered ground state (a striped one, a honeycomb arrangement of vacancies). A single run at one temperature can get stuck in a local minimum: `anneal()` wraps `optimize_energy()` in a decreasing-temperature schedule and keeps the best configuration seen along it, and `optimize_energy_multistart()` keeps the best of several independent restarts. `optimize_grand_canonical()` switches from fixed-filling swap moves to single-site flips under `lg.mu`, letting the filling itself fluctuate, which is the way to scan a phase diagram as a function of the chemical potential, or to estimate thermodynamic quantities such as the specific heat (`get_specific_heat()` and `get_susceptibility()`) from an equilibrium trajectory at a fixed temperature. `add_tensor()` adds couplings beyond fixed neighbor shells, and `write()` and `read()` checkpoint a snapshot to and from disk.

See `examples/latticegas/` for runnable versions of the anneal, the local-energy map, the correlators, and the grand-canonical scan of the filling against the chemical potential.

Ising models

Let us now take the lattice gas one step further, to Ising spins $s_i \in \{-1, +1\}$ on a lattice, with `latticeising.LatticeIsing`. The spins interact through a real-space coupling $-J_{ij}s_i s_j$ and a site-dependent field $-b_i s_i$, the usual Ising Hamiltonian, with $J > 0$ ferromagnetic (favoring alignment). It mirrors `LatticeGas` closely, and shares its `add_tensor()`, `regroup()` and `get_specific_heat()/get_susceptibility()` helpers, but note that the sign convention of the energy is the **opposite**: in `LatticeGas` the coupling $\sum J_{ij}n_i n_j$ carries no minus sign, so a positive J there is a *repulsion*, while here a positive J_{ij} in `add_interaction()` means ferromagnetic alignment, the usual convention for Ising spins.

The simplest case is a ferromagnet on a square lattice, which is bipartite, so nothing frustrates the alignment:

```
from pyqula import geometry
from pyqula import latticeising

g = geometry.square_lattice() # bipartite, so ferromagnetic order is not frustrated
g = g.get_supercell(12)
g.dimensionality = 0

li = latticeising.LatticeIsing(g,m=0.0) # random +/-1 spins, zero net magnetization
li.add_interaction(Jij=[1.]) # first-neighbor ferromagnetic coupling
es,ms = li.anneal(temps=[3.,1.,0.3,0.1,0.03],ntries=1e4) # simulated annealing
```

`li.s` holds the current ± 1 spin array, `es` the energy along the anneal and `ms` the total magnetization $\sum_i s_i$ along it. What you see in `ms` is the phase transition: at the high temperatures at the start of the schedule the magnetization per site, `ms/li.nsites`, fluctuates around zero, and as the temperature drops below the transition it grows to $+1$ or -1 , all the spins aligned, with the sign chosen by chance. `li.optimize_energy()` runs single-spin-flip Metropolis dynamics, the usual Monte Carlo move for Ising spins, in which the total magnetization is *not* conserved (it fluctuates under `li.b`), so, mirroring `LatticeGas.optimize_grand_canonical()`, it returns `(es, ms)`, the energy and total magnetization trajectories, the latter usable directly with `latticegas.get_susceptibility()`. `li.optimize_conserved()` instead uses Kawasaki spin-exchange (swap) dynamics, which *does* conserve the total magnetization, the analog of the fixed-filling swaps of `LatticeGas.optimize_energy()`. `li.anneal()` wraps `optimize_energy()` in a decreasing-temperature schedule, and `optimize_energy_multistart()` keeps the best of several independent restarts. `get_local_energy()` and `get_local_field()` give the per-site energy and effective field of the current snapshot, and `get_correlator()` and `get_structure_factor()` are the same real- and reciprocal-space diagnostics `LatticeGas` has, locating ordered ground states (ferromagnetic, checkerboard antiferromagnetic) and their ordering

wavevector. Note that `li.pairs` lists both directions of every bond (the same convention as `LatticeGas`), so `get_energy()` is twice the usual sum over unordered bonds: the critical temperature of the ferromagnet on the square lattice sits near 2×2.269 in these units, not 2.269.

See `examples/latticeising/` for runnable versions of the anneal, a temperature scan (magnetization and specific heat), and maps of the local energy and the local field of a frustrated antiferromagnet on the triangular lattice.

Parallelism and reproducibility

Most of the calculations in this guide are sweeps: over k-points for a band structure, over energies for a density of states, over q-points for a response function, over restarts for an anneal. They run serially by default, one item after the other, and `parallel.set_cores(n)` spreads them over `n` worker processes; `parallel.set_enabled(False)` goes the other way and forces the whole package strictly serial, which is the setting to reach for when debugging.

```
from pyqula import parallel
parallel.set_cores(4)           # spread the sweeps over 4 processes
```

The number of cores changes how long a sweep takes and nothing else. Results do not depend on the core count, including for the routines that draw random numbers, the stochastic trace of the KPM and the multistart anneals of the classical models: `cores=1` and `cores=8` give identical answers. Seeding numpy in the parent process

```
import numpy as np
np.random.seed(42)
```

therefore makes a whole sweep reproducible run to run, which is what you want when comparing two calculations that differ in a single parameter.

Running on a GPU

Three of the heaviest kernels of the library have a second implementation written in jax, which runs on a GPU: the Chebyshev moments behind the KPM density of states, the Lindhard sum behind the RPA response functions, and the batched diagonalization of the Bloch Hamiltonian over a k-mesh, which band structures, densities of states and Berry curvatures all bottom out in. Which implementation runs is decided by one switch, and the package stays on the CPU until that switch is set

```
from pyqula import gpu
gpu.set_gpu(True) # every routine with a GPU path now uses the device
```

so that a script that never calls it computes what it computed before, on whichever machine it lands on. `gpu.set_gpu(False)` goes back to the CPU,

and `gpu.is_gpu_available()` says whether jax sees a device at all, which is what to pass when a script should simply take what the machine offers. Asking for a GPU where there is none is not an error: the same jax kernels then run on jax's own CPU backend and give the same numbers, with a warning saying that the speedup is not there.

How much the device buys depends on the shape of the calculation more than on its size. The Lindhard sum gains the most, since its cost grows as the fourth power of the number of sites in the cell and the whole k-mesh is contracted on the device at once. The Chebyshev moments gain little, a factor of one to three, because a sparse matrix-vector product is limited by memory bandwidth rather than by arithmetic. The batched diagonalization sits in between, and only pays from matrices of about 32 orbitals upward, so below that size it stays on the CPU even with the switch set.

One sum runs against that pattern and is worth knowing about before setting the switch. The static polarizability behind the screened interaction (`get_polarizability`, `get_screened_interaction` and any `get_bse(screening=...)`) produces only a small matrix per q-point out of a very long sum, so what it asks of the device is a great many small products rather than a few large ones. On a consumer card it is faster than the processor in single precision from about 36 orbitals upward, by around seven times, and slower than the processor in double precision at every size, which is the one place in the library where asking for double precision on a GPU costs rather than buys. Leaving `chi_prec` unset gives single precision there, which is the case that pays.

Precision is a separate choice, made per call rather than globally, because it changes the numbers and not only where they are computed. The KPM moments take `kpm_prec`, the Lindhard sum takes `chi_prec` and the batched diagonalization takes `eigh_prec`, each of them `"single"` or `"double"`. Double precision is the default everywhere except the Lindhard sum on the GPU, where single precision is the default because a consumer card can be an order of magnitude slower in double precision than in single. Single precision costs about six digits, which a density of states or a band structure does not notice, and which a quantity built from differences of nearly equal numbers does.

A handful of routines written in jax have no CPU implementation of their own, among them the mean-field solvers reached with `use_jax=True`, the graphene relaxation and the jax Keldysh current. Those follow the same switch, so they too run on the CPU by default.

Errors and unsupported inputs

Most routines only make sense for a Hamiltonian of a particular kind: a Berry curvature needs a two-dimensional one, a spin operator needs the spin degree of freedom, a superconducting quantity needs the Nambu one, and an option se-

lected by a string (`mode=`, `solver=`, `channel=`) needs one of a fixed set of strings. Those requirements are checked up front, before any calculation starts, and the exception says what was needed rather than only that something went wrong: `ValueError` for a value that cannot work (the Berry curvature of a Hamiltonian that is not two-dimensional, an unknown `mode` string, a spin operator on a spinless Hamiltonian), `NotImplementedError` for a combination that is not built yet, `TypeError` for an argument of the wrong kind. For a string-selected option the accepted values are listed in the message, so a typo is self-diagnosing:

```
ValueError: unknown mode ED2; the DOS accepts 'ED', 'KPM', 'adaptive', 'Green' and 'RG'
```

Two messages name their own fix, and you will meet them: a superconducting quantity asked of a normal Hamiltonian says to call `h.setup_nambu_spinor()` first, and a spin quantity asked of a spinless one says to call `h.turn_spinful()`.

```
from pyqula import geometry
h = geometry.honeycomb_lattice().get_hamiltonian(has_spin=False)
h.extract("mz")
# ValueError: the magnetization 'mz' needs a spinful Hamiltonian; call
# h.turn_spinful() first, or build it with g.get_hamiltonian(has_spin=True)
```

Note that the requirement is on *reading* a degree of freedom the Hamiltonian does not have. The methods that *add* a spin term, `h.add_exchange()`, `h.add_zeeman()`, `h.add_kane_mele()`, promote a spinless Hamiltonian to a spinful one rather than rejecting it, as we saw in the section on the Zeeman field. A third requirement of the same kind is the sublattice label, which several sublattice-resolved quantities need and an unlabeled cell does not have: `h.extract("CDW")` on such a cell says so rather than returning `None`.

A second family of checks exists not because the routine cannot run, but because it could run and return a number that means nothing. Those raise rather than answer:

- `h.get_average_spin_splitting()`, `h.get_spin_splitting_density()` and `h.get_spin_splitting_vs_energy()` refuse a Hamiltonian whose spin off-diagonal block does not vanish (Rashba, any spin-orbit term, non-collinear order), naming the largest off-diagonal element found
- `h.get_dos()` on a non-Hermitian Hamiltonian refuses every mode but "ED", and refuses `use_kpm=True`: the Chebyshev and adaptive expansions assume a real spectrum (see “Non-Hermitian Hamiltonians”)
- `h.get_total_energy(fermi=...)` refuses a Nambu Hamiltonian, where a nonzero `fermi` is not a rigid shift of the spectrum (the electron and hole blocks move opposite ways). Use `h.shift_fermi(-mu)` on the Hamiltonian instead
- the surface density of states refuses a momentum-dependent operator such as "valley", which its Green’s function has already integrated over
- the lead decimation behind `HT.didv()`, `h.get_dos(mode="RG")` and `Embedding.get_gf()` refuses an energy sitting exactly on a lead level with a broadening below about $1e-7$, and recommends a larger `delta`

Main functions and methods

Geometry functions and methods

g.get_hamiltonian()

Generate the Hamiltonian from a geometry.

Optional arguments

- `tij = [1.0,0,0.]`: List with 1st, 2nd, 3rd nearest-neighbor hopping, or a function of two positions returning the hopping between them, or a `specialhopping.HoppingGenerator`
- `has_spin=True`: include the spin degree of freedom
- `is_sparse=False`: store the matrices in sparse form, for large cells
- `non_hermitian=False`: build a non-Hermitian Hamiltonian, which routes the band structure, DOS, LDOS and Berry curvature to their non-Hermitian implementations (see “Non-Hermitian Hamiltonians”)
- `is_multicell=False`: store the hoppings as a multicell dictionary
- `nc=2`: neighbor cutoff used by the multicell construction

An unrecognized keyword raises **TypeError** listing the accepted ones, so a misspelled `has_spin` is not silently the default.

Returns the Hamiltonian

g.remove()

Return a copy of the geometry with sites removed. Used to carve a vacancy or an antidot out of a flake or a supercell before building its Hamiltonian (see “Valley operator” for an example).

Arguments

- `i=0`: which sites to drop, a single index into `g.r`, a list of indices, or a callable of the position, in which case every site where it returns True is removed

Returns a new geometry; `g` itself is untouched

g.get_supercell()

Generate a supercell

Arguments

- `nsuper`: size of the supercell to create, number or tuple, or a 3x3 integer matrix `M` for a general non-diagonal/non-orthogonal supercell (see “Electronic structure folding and unfolding” for how this interacts with `operator="unfold"`) (positional, or by that name)

Optional arguments

- `store_primal=False`: keep a reference to the primitive-cell geometry on the supercell, needed by `operator="unfold"` (see “Electronic structure folding and unfolding”)

A non-integer `nsuper` asks for a cell that many times larger by area, not that many primitive cells along each vector, so that `np.sqrt(3)` gives the $\sqrt{3} \times \sqrt{3}$ cell; the most compact cell of that size is returned, and this route is for 2d geometries only

Returns a new geometry

`g.get_unfolded_kpath()`

Return the k-path of the primitive cell, written in the reduced coordinates of this supercell, which is the path an unfolded band structure has to be computed along (see “Which k-path to unfold along”). Available on the Hamiltonian as `h.get_unfolded_kpath()` as well, and it needs the supercell to have been built with `store_primal=True`

Arguments

- `kpath=None`: the same as in `g.get_kpath()`, either an explicit list of k-points or a list of high-symmetry labels, which are read in the primitive cell and not in the supercell
- `g0=None`: the primitive-cell geometry, when the supercell does not carry one
- `nk`: number of k-points along the path, passed through to `g.get_kpath()`

Returns an array of k-points, one per row, ready to be passed as `kpath=` to `h.get_bands()` or `h.get_kdos_bands()`

Hamiltonian functions and methods

`h.get_bands()`

Compute band structure

Optional arguments:

- `nk = 400`: number of k-points
- `operator`: a single operator, or a list of operators, to compute expectation values for at each eigenstate
- `kpath`: an explicit k-path, either as reduced coordinates or as a list of high-symmetry labels ("G", "M", "K", "X", "Y", and in three dimensions "Z", "R", "A", "B")
- `ewindow`: a function of the energy returning whether to keep that band, used to restrict the output to an energy window (e.g. `ewindow=lambda e: abs(e)<0.5`)

- `eigmode="complex"`: non-Hermitian Hamiltonians only, which part of the complex eigenvalue is returned and written, `"complex"`, `"real"` or `"imag"` (see “Non-Hermitian Hamiltonians”). With `"complex"` the written `BANDS.OUT` carries `k`, `Re E`, `Im E` and then the operator columns

Without `kpath` the path is Γ -M for a square-like 2D lattice, Γ -K-M-K'- Γ for a triangular-like one, and Γ -X-M- Γ -R for a 3D one; the 3D path leaves the $k_3 = 0$ plane, so the band edges of a 3D crystal are reached.

Returns kpoint index and energies, plus one extra row per operator if `operator` is given

`h.get_kdos_bands()`

Compute a k-resolved spectral function (band structure dressed with a projection operator, or an unfolded spectral function) along a k-path.

Optional arguments:

- `kpath`: k-point path (auto-generated if not given)
- `operator=None`: operator used to weight the spectral function, e.g. `"unfold"` (see “Electronic structure folding and unfolding”)
- `energies`, `delta`, `nk`: frequency range, broadening, k-point density
- `mode="ED"`: `"ED"` or `"KPM"` (equivalently `use_kpm=True`)
- `frand=None`: generator of the random vectors the KPM stochastic trace draws. Only the KPM path uses them, so passing it without `mode="KPM"` raises `ValueError`

Returns k-path fraction, energy and spectral weight

`h.get_dos()`

Compute the density of states, normalized so that its integral over the energies is the number of states per unit cell.

Optional arguments:

- `energies`: array with frequencies of the DOS
- `delta=None`: broadening of the DOS. Left as `None` it is chosen from the k-mesh, $5/nk$ (so 0.05 at the default `nk=100`), which keeps the curve smooth as the mesh is refined
- `mode="ED"`: `"ED"`, `"KPM"`, `"adaptive"`, `"Green"` or `"RG"` (see the “Density of states” section). Anything else raises, listing those five
- `nk=100`: k-points per direction, for `"ED"` and `"KPM"`; `"adaptive"` uses it only as a subdivision limit of its error-controlled quadrature, and

"Green"/"RG" forward it to the self-energy's own k-sum (see the "Density of states" section)

- `operator=None`: operator the DOS is projected onto (a name, a matrix or an `Operator`)

For a non-Hermitian Hamiltonian only `mode="ED"` exists, and the extra `eigmode` argument chooses which part of the complex eigenvalue the broadening is centred on. See "Non-Hermitian Hamiltonians".

Return energies and DOS

`h.get_gap()`

Return the indirect gap, i.e. the smallest energy difference between an empty and an occupied state anywhere in the Brillouin zone (the two need not sit at the same k-point). Obtained by numerically minimizing over k rather than by scanning a fixed mesh, so a gap that closes at an incommensurate k-point is not missed.

Optional arguments:

- `ntries=1`: repeat the minimization this many times and keep the smallest result. The search is deterministic, so every repetition returns the same number and raising this only costs time

Returns a single number, the gap. Zero (up to numerical noise) for a metal or a Dirac semimetal

`h.get_bandwidth()`

Return the bottom and top of the spectrum, (`emin,emax`); note that this is the pair of band edges, not their difference. Uses the same k-space optimization as `h.get_gap()`, so the edges are the true extrema over the Brillouin zone rather than the extrema of a k-mesh sample

`h.get_filling()`

Return the fraction of states below zero energy, i.e. the filling measured with the Fermi energy at $E = 0$. Half filling gives 0.5. Use `h.set_filling(nu)` to shift the onsite energy so that a target filling is realized, and this method to check the result.

For a Hamiltonian with the electron-hole (Nambu) degree of freedom the spectrum is particle-hole symmetric, so exactly half of it lies below zero whatever the density is; there the filling is instead the weight that the occupied BdG states put on the electron components, normalized by the two electron states per site. At zero pairing this reproduces the normal-state filling exactly.

Optional arguments:

- nk: k-point density used to sample the spectrum

h.get_total_energy()

Return the total energy, i.e. the sum of the occupied single-particle eigenvalues. For a mean-field Hamiltonian this is the band energy only, `h.get_mean_field_hamiltonian(...,return_total_energy=True)` returns the interacting total energy including the double-counting correction instead

Optional arguments:

- nk=10: k-point density of the Brillouin-zone sum
- fermi=0.0: energy below which states are counted as occupied. Not accepted for a Nambu Hamiltonian (raises `ValueError`): a nonzero `fermi` is not a rigid shift of a BdG spectrum, since the electron and hole blocks shift by $-\mu$ and $+\mu$. Shift the Hamiltonian instead, with `h.shift_fermi(-mu)`, and leave `fermi=0`
- mode="mesh": k-space sampling; `use_kpm=True` switches to a Chebyshev estimate for large systems

For a Nambu/BdG Hamiltonian this returns the *electronic* energy, i.e. the energy of the physical electrons rather than of the doubled Nambu spectrum, so at zero pairing it agrees with the normal-state answer for the same model. `nbands=` is not implemented there (raises `NotImplementedError`). Note that this is still the band energy: the mean-field total energy of a paired state, with the double counting of both the normal and the pairing mean field removed, is the one returned with `return_total_energy=True`

h.get_density_matrix()

Return the full density matrix of the occupied states, as a dense matrix in the same basis as `h.intra`. See “Interactions at the mean-field level” for the k-resolved, hopping-resolved version the self-consistent loops use.

Its index convention is $dm_{ij} = \sum_{occ} \psi_i^* \psi_j$, the *transpose* of the usual $\rho_{ij} = \sum_{occ} \psi_i \psi_j^*$. So an expectation value is `np.trace(dm.T@A)`, not `np.trace(dm@A)`: the two agree for any real operator (the density, σ_x , σ_z , a projector) and differ by a sign for a purely imaginary one (σ_y , the valley operator, any current operator $i[H, r]$). Use `h.get_vev(operator)` rather than contracting the matrix yourself

h.get_ipr()

Return the inverse participation ratio of every eigenstate, as `(energies,ipr)`. A delocalized state in a system of N sites gives $IPR \sim 1/N$ and a state localized on one site gives $IPR \sim 1$, so this is the usual diagnostic for Anderson localization or for in-gap bound states. Finite (0d) systems only: it raises `NotImplementedError` for a periodic Hamiltonian; for those use the IPR operator instead (see “Inverse participation ratio operator”)

`h.get_vev()` / `h.get_single_vev()` / `h.get_several_vev()`

Ground-state expectation values of operators, evaluated by summing over the occupied states.

- `h.get_vev(operator=...)` returns one real number **per site**: the site-resolved expectation value of `operator` (any name accepted by `h.get_operator`, e.g. "sz"), or the site occupation if `operator` is omitted. This is what produces a magnetization or charge-density map
- `h.get_single_vev(A)` returns the single number $\langle A \rangle$ for one operator A, summed over the whole system
- `h.get_several_vev([A,B,...])` does the same for a list of operators in one pass, sharing the diagonalization

An operator that acts with a different matrix at every k-point, "unfold" above all, is applied inside the sum over the Brillouin zone rather than contracted with the k-summed density matrix, so `h.get_vev(operator="unfold")` gives the unfolded weight site by site

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # primitive geometry
gs = g.get_supercell(2,store_primal=True) # supercell, keeping the primitive cell
h = gs.get_hamiltonian(has_spin=False)
h.add_onsite(lambda r: 100.0 if sum((r-gs.r[0])**2)<1e-2 else 0.0) # a vacancy
d = h.get_vev(operator="unfold") # unfolded weight on each site
```

Optional arguments:

- `nk=30`: k-point density of the Brillouin-zone sum

`h.get_magnetization()`

Site-resolved magnetization, as an (`nsites,3`) array. Two different quantities go by this name, and the `mode` argument picks between them:

- `mode="vev"` (the default) returns the physical moment: the per-site expectation value ($\langle S_x \rangle, \langle S_y \rangle, \langle S_z \rangle$) over the occupied states, i.e. `get_vev("sx"/"sy"/"sz")`. Being a Brillouin-zone integral it needs a k-mesh. Pass `nk`, or rely on the mesh a self-consistent Hamiltonian remembers from its own loop. On a metal at $T = 0$ the moment is quantized in steps of $2/n_k$, so use a fine enough mesh (or `mode="field"`) when comparing weakly polarized states
- `mode="field"` reads the magnetic *term written in the Hamiltonian* instead, i.e. the coefficients of $\sigma_{x,y,z}$ on each site. After a self-consistent calculation this is the mean-field exchange field: the natural order parameter of the loop, proportional, not equal, to the moment, and continuous where the moment is quantized. On a Hamiltonian whose field you put in by hand it hands that field straight back

Note the sign: `add_zeeman/add_exchange` add $+\vec{h} \cdot \vec{\sigma}$, so the occupied states polarize *against* $\vec{h}$ and the moment comes out antiparallel to the field you applied.

```
from pyqula import geometry
h = geometry.chain().get_hamiltonian()
h.add_exchange([0.,0.,0.5])
h.get_magnetization(nk=40)           # the moment: [0,0,-0.15], against the field
h.get_magnetization(mode="field")    # [0,0,0.5], the field you put in
```

Optional arguments:

- `mode="vev"`: "vev" or "field", as above
- any further keyword (e.g. `nk`) is forwarded to `get_vev` in "vev" mode

h.get_topological_invariant()

Return a topological invariant of the occupied bands: the Berry (Zak) phase in one dimension, the Z_2 invariant in two dimensions when the Hamiltonian is time-reversal symmetric, and the Chern number otherwise. A 0d Hamiltonian has no Brillouin zone and 3D is not implemented, so both raise; for a 3D system take the Chern number of a 2D slice, or `topology.berry_phase(h,kpath=...)` along a chosen path.

h.add_soc()

Add Kane-Mele intrinsic spin-orbit coupling

Arguments:

- `t`: value of the SOC (positional, or by that name)

h.add_zeeman()

Add a Zeeman field to the Hamiltonian

Arguments:

- `zeeman`: value of the Zeeman field, as a number (assumes $[0,0,B_z]$), array or callable function (positional, or by that name)

h.add_rashba()

Add Rashba spin-orbit coupling

Arguments:

- `c`: value of the Rashba SOC (positional, or by that name)

h.add__onsite()

Add a local onsite energy

Arguments:

- **fermi**: value of the onsite energy, as a number, an array with one entry per site, or a callable of the position (positional, or by that name)

h.add__sublattice_imbalance()

Add a staggered onsite energy, $+m$ on one sublattice and $-m$ on the other: the mass term that opens a gap in a honeycomb lattice and turns it into a boron-nitride-like semiconductor

Arguments:

- **mass**: the imbalance, as a number or as a callable of the position

The term is written with the geometry's sublattice index, so it needs one. A geometry with no sublattice (a chain, a triangular or square lattice) and a geometry with more than two (kagome, pyrochlore, whose index runs 0, 1, 2, ... rather than ± 1) both raise a **ValueError**. On a bipartite lattice you can label one yourself with **g** = **g.get_supercell(2)** followed by **g.get_sublattice()**; otherwise write the profile explicitly with **h.add__onsite(f)**.

h.add__antiferromagnetism()

Add a staggered exchange field: a Neel pattern on two sublattices, and the 120-degree frustrated pattern on more than two

Arguments:

- **mass**: the exchange field, as a number or as a callable of the position

Like **add__sublattice_imbalance** this needs a sublattice and raises a **ValueError** when the geometry has none. Unlike it, more than two sublattices are supported, through the frustrated pattern

h.add__haldane()

Add a Haldane term: a complex second-neighbor hopping whose sign is set by the chirality of the two-step path, which opens a gap and makes a honeycomb lattice a Chern insulator (see “Chern number”).

Arguments

- **t**: amplitude of the second-neighbor hopping, a number or a callable of the position

Needs a geometry with a sublattice. Breaks time-reversal symmetry

h.add_modified_haldane() / h.add_antihaldane()

Same second-neighbor complex hopping, but with the sign also flipped between the two sublattices, so the two valleys acquire opposite masses and the total Chern number is zero, a valley-Hall rather than a Chern insulator. **add_antihaldane** is a second name for the same method.

Arguments

- **t**: amplitude of the second-neighbor hopping

h.add_kane_mele()

Add a Kane-Mele spin-orbit term: the Haldane hopping with opposite sign for the two spin channels, so time-reversal symmetry is preserved and the model is a quantum spin Hall insulator instead of a Chern insulator. Turns the Hamiltonian spinful if it is not already.

Arguments

- **t**: amplitude of the spin-orbit second-neighbor hopping

Its spin Chern number is what **topology.spin_chern** computes

h.add_anti_kane_mele()

The sublattice-staggered counterpart of **add_kane_mele**, in the same way **add_modified_haldane** is the counterpart of **add_haldane**.

Arguments

- **t**: amplitude of the second-neighbor hopping

h.add_kekule() / h.add_chiral_kekule()

Add a Kekule bond modulation: a period-tripling pattern on the first-neighbor bonds of a honeycomb lattice, which folds the two Dirac points onto Γ and gaps them. **add_kekule(t)** takes the modulation amplitude. **add_chiral_kekule(t1=...,t2=...)** adds the bond-direction-aware chiral version, whose two amplitudes are the two independent complex components of the modulation, optionally on an explicit **registry** of retained hexagon centers rather than the default one

h.add_valley_exchange()

Add a valley-space exchange term $\vec{v} \cdot (\tau_x, \tau_y, \tau_z)$, the valley-pseudospin analogue of **add_exchange** for real spin (see “In-plane valley operators”).

Arguments

- **v** = (vx,vy,vz): the valley field

Built from the same in-plane valley operators, so for a periodic Hamiltonian it needs a Kekule-commensurate cell, a multiple-of-3 supercell of the primitive honeycomb cell, and raises `ValueError` otherwise. A finite (0d) flake needs no such commensurability

h.add_peierls() / h.add_orbital_magnetic_field()

Add an out-of-plane orbital magnetic field as a Peierls phase on every hopping. `add_orbital_magnetic_field` is a second name for the same method.

Arguments

- `mag_field`: the field, in units of the flux quantum per unit area of the lattice; the Peierls phase on a bond is $2\pi B y dx$ in the Landau gauge. A periodic (commensurate) calculation needs the flux through the unit cell to be a rational multiple of the flux quantum, so build the supercell first and pick `mag_field` to match it

Optional arguments

- `gauge="Landau"`: "Landau" or "symmetric"

Refuses a Hamiltonian that already carries a pairing amplitude (`NotImplementedError`): a Cooper pair has charge $2e$, so the anomalous term has no single Peierls phase, and an orbital field in a superconductor means vortices. Add the field to the normal-state Hamiltonian first, then `h.turn_nambu()/h.add_swave()`

h.add_inplane_bfield()

Add an in-plane magnetic field, as a Peierls phase built from the out-of-plane coordinate. Meaningful for a multilayer or a system with finite thickness, where an in-plane field still threads flux between the layers.

Optional arguments

- `b=0.0`: field strength
- `phi=0.0`: in-plane direction of the field, in units of π

Only implemented up to two dimensions

h.add_strain()

Modify the hoppings according to a strain field, which is how a pseudo-magnetic field is produced in graphene.

Arguments

- `sr`: a callable of the position returning the local strain

Optional arguments

- `mode="scalar"`: `"scalar"` rescales every hopping by `sr(r)` alone; `"directional"` also uses the bond direction, for a genuinely anisotropic strain

Turns the Hamiltonian multicell. Not implemented for a spinless Nambu Hilbert space

`h.add_crystal_field()`

Add a crystal field: an onsite potential built from the local atomic environment, so that sites with fewer or more distant neighbors (an edge, a vacancy, the two inequivalent stackings of a bilayer) sit at different energies.

Arguments

- `v`: strength of the crystal field. The built-in potential is normalized and has its average removed, so `v` sets the spread of the onsite energies rather than their offset

Optional arguments

- `rcut=6.0`: distance cutoff of the neighbor sum

`h.generate_spin_spiral()`

Rotate the Hamiltonian into a spin-spiral ansatz, i.e. impose a magnetic texture whose quantization axis winds with a wavevector q . Used both as a starting point for a spiral mean-field calculation and to scan the energy of the spiral as a function of q (see “Spin-spin exchange interactions”).

Optional arguments

- `vector=[0.,0.,1.]`: axis the spins rotate about
- `qspiral=[1.,0.,0.]`: the spiral wavevector
- `fractional=True`: read `qspiral` in fractional (reduced) coordinates. With `False` only the inter-cell hoppings are rotated

Needs a spinful Hamiltonian

`h.add_pairing()`

Add a general superconducting pairing term, given by its symmetry channel, turning the Hamiltonian into its Nambu (BdG) form if it is not already (see “Spin-triplet d-vector and non-unitary superconductivity”).

Optional arguments

- `delta=0.0`: pairing amplitude, a number or a callable of the position
- `mode="swave"`: the pairing symmetry: `"swave"`, `"extended_swave"`, `"triplet"`, `"pwave"`, `"chiral_pwave"`, `"dx2y2"`, `"dxy"`, `"nodal_dwave"`, `"chiral_dwave"`, `"nodal_fwave"`, `"chiral_fwave"`, `"chiral_gwave"`,

"haldane", "antihaldane", and several others. A callable returning the 2x2 pairing matrix for a pair of positions is also accepted. An unknown name raises `ValueError` listing every accepted one

- `d=[0.,0.,1.]`: the d-vector, for the triplet channels

h.setup_nambu_spinor()

Put the Hamiltonian into its Nambu (electron-hole doubled) form, with zero pairing. This is the call the error messages of the superconducting routines point at: a superconducting quantity asked of a normal Hamiltonian needs the electron-hole degree of freedom to exist first. Equivalent to `h.add_swave(0.0)`. Modifies in place

h.turn_nambu()

Add the electron-hole degree of freedom without adding any pairing. Lower level than `setup_nambu_spinor()`, which is the one to call normally. Modifies in place

h.turn_spinful()

Add the spin degree of freedom to a spinless Hamiltonian, doubling every matrix with an identity in spin space. This is the call the error messages of the spin-resolved routines point at. Modifies in place, and does nothing if the Hamiltonian is already spinful

h.remove_spin()

The inverse: keep one spin block and discard the other, returning to a spinless Hamiltonian.

Optional arguments

- `channel="up"`: which block to keep, "up" or "dn"

This drops the spin off-diagonal block, so it is only meaningful when spin is a good quantum number. With spin-orbit coupling or non-collinear order the result is a different model, not a projection of this one. The spin-splitting routines built on it check that explicitly (see "Errors and unsupported inputs")

h.turn_multicell()

Convert the Hamiltonian to multicell form, i.e. store the inter-cell hoppings as a dictionary keyed by lattice vector instead of the single `inter` matrix of the nearest-neighbor-cell form. Several routines require it, and their error messages say so; hoppings beyond the first neighboring cell need it. Modifies in place. `h.get_no_multicell()` returns a non-multicell copy where that is possible

h.get_hk_gen()

Return the Bloch generator: a function of the reduced k-vector returning the Bloch matrix $H(k)$ of the Hamiltonian. This is the object every k-space routine in the library is built on, and the way to evaluate a model at a chosen k-point directly (`h.get_hk_gen()([0.3,0.,0.])`). pyqula's convention is the periodic one, $H(k) = \sum_R t(R)e^{2\pi i k \cdot R}$ with k in reduced coordinates and no intra-cell atomic positions in the phases

h.get_gf()

Return the bulk Green's function of the Hamiltonian at one energy, as a matrix in the same basis as `h.intra`, obtained by integrating over the Brillouin zone.

Optional arguments

- `energy=0.0`: energy at which it is evaluated
- `delta=1e-5`: imaginary part (analytic continuation)
- `mode="adaptive"`: how the Brillouin-zone integral is done, "adaptive" (error-controlled), "full" (a fixed `nk` mesh) or "renormalization"
- `gtype="bulk"`: "bulk" or "surface"

— $\text{Im Tr } G/\pi$ is the density of states, which is what `h.get_dos(mode="Green")` computes from it

h.get_hopping_dict()

Return the real-space hoppings as a dictionary `{(n1,n2,n3): matrix}`, keyed by the lattice vector connecting the two cells, with (0,0,0) the intracell block. This is the representation the response functions accept for an interaction with support beyond one cell (see "Interactions beyond onsite"), and the natural way to inspect or modify a Hamiltonian's hoppings directly. `h.get_multihopping()` returns the same thing wrapped in a `multihopping.MultiHopping`, which supports addition and scalar multiplication, and `h.set_multihopping()` writes one back

h.get_rkky()

Compute the RKKY interaction between two magnetic impurities mediated by the electrons of `h`, i.e. the effective exchange coupling as a function of their separation.

Optional arguments

- `mode="pm"`: "pm" computes it explicitly ("poor man's"), by adding two local exchange fields to the Hamiltonian and taking the energy difference between their parallel and antiparallel alignment; "LR" computes it from linear response instead, which is much faster for a map over many separations. An unknown mode raises `ValueError` listing both

- for `mode="pm"`: `ri`, `rj` the positions of the two impurities (both required), `nk=10`, `dj=1e-1` the strength of the probe exchange fields
- for `mode="LR"`: `R=[0,0,0]` the lattice vector between the two cells, `ii=0`, `jj=0` the sites inside them, `nk=100`, `delta` the analytic continuation (default `1/nk`)

`rkky.rkky_map(h,n=...)` sweeps either mode over a range of separations. On a bipartite lattice at half filling the sign is ferromagnetic between sites of the same sublattice and antiferromagnetic between opposite ones

`h.get_ldos()`

Compute the local density of states.

Optional arguments:

- `e`: energy of the LDOS (`energy` is accepted as an alias, since that is how the Green's function, embedding and transport routines spell it)
- `delta=0.001`: broadening of the LDOS, which must be positive
- `operator=None`: operator the LDOS is projected onto (a name, a matrix or an `Operator`); see the LDOS section above for how the two modes weight it
- `mode="arpack"`: `"arpack"` (diagonalization on a k-mesh) or `"green"` (Green's function, 2D only)
- `projection="TB"`: `"TB"`, `"TBRS"` (real-space interpolated) or `"atomic"`
- `eigmode="complex"`: non-Hermitian Hamiltonians only, whether `e` is read on the real or the imaginary axis of the complex spectrum (see "Non-Hermitian Hamiltonians"). Only `mode="diagonalization"` is implemented there

Return `x`, position, `y` position and LDOS

`h.get_multildos()`

Compute the LDOS at many energies, writing one file per energy to a `MULTILDOS/` folder, together with a `MULTILDOS/DOS.OUT` holding the total DOS on the same energies and a `DOSMAP.OUT` in the working directory. The maps and that DOS carry the same normalization as `h.get_ldos()` and `h.get_dos()`, so they can be read against each other directly.

Optional arguments:

- `energies=linspace(-1,1,100)`: energies to compute
- `delta`, `nk`: broadening and k-point density, as in `h.get_ldos()`
- `operator=None`: operator the LDOS is projected onto, weighting each eigenstate by $\langle \Psi | A | \Psi \rangle$, the same convention `h.get_ldos(mode="arpack")`

uses. The older spelling `op=` is still accepted; passing both raises `TypeError`

- `projection="TB"`: "TB" or "atomic". Anything else raises `ValueError` listing the two, and "atomic" together with `operator=` raises `NotImplementedError` naming "TB" as the projection that supports it

An unrecognized keyword raises `TypeError`.

`h.get_chi()`

Compute a non-interacting operator-operator response function (charge-charge by default).

Optional arguments:

- `q=None`: momentum transfer. Left as `None` the response is *averaged over the whole q-mesh* rather than evaluated at `q=0`; pass `q=[0.,0.,0.]` explicitly for the uniform response
- `A=None, B=None`: operators defining the response (default: identity, i.e. charge-charge)
- `energies, delta, nk`: frequency range, broadening, k-mesh density

Returns energies and the response function

`h.get_spinchi_ladder()`

Compute the transverse (S^+/S^-) spin susceptibility, RPA-dressed by default using the Hubbard U of a mean-field Hamiltonian.

Optional arguments:

- `q=None, energies, delta, nk`: as above, `None` again meaning the q-average
- `RPA=True`: dress with the random-phase approximation; `False` for the bare response. An interaction that couples different sites is dressed in the pair basis of `h.get_transverse_spinchi()` instead of with the site vertex, with the same arguments (see “Interactions beyond onsite”)
- `T=None`: temperature of the occupations, equal to `delta` when not given
- `chi_prec=None`: precision of the Lindhard sum, "single" or "double"; unset, it is "single" on the GPU and "double" on the CPU. Also accepted by `get_spinchi_full`, `get_qdos_iets`, `get_iets_ldos`, `get_rpa_kernel_poles` and `get_magnon_bands(method="rpa")`, and by `get_transverse_spinchi` and `get_magnon_bands(method="pair")`, which sum the response in the pair basis; `mode="trace"/"diagonal"` are double precision only, as is the pair basis on the CPU, and raise for "single". Where the response is computed is the package-wide switch

`gpu.set_gpu` (see “Running on a GPU”), and `mode="trace"/"diagonal"` and `imode="adaptive"` are not available on the GPU and raise

`h.get_rpa_kernel_poles()`

Compute the poles of the generic RPA kernel $1 - V(q)\chi(q, \omega)$: the frequencies of the collective modes/instabilities of the interacting response.

Optional arguments:

- `V=None` (required): the interaction; a `ValueError` is raised if not given. Either a plain matrix (q-independent, onsite-only) or a real-space hopping dict/`MultiHopping` `{(n1,n2,n3): matrix}` for an interaction with support beyond the onsite cell, Fourier-transformed to $V(q)$ at this call’s `q` (see “Interactions beyond onsite”)
- `A=None`, `B=None`, `q=None`, `energies`, `delta`, `nk`: as in `get_chi`, `None` again meaning the q-average

Returns an `(npoles,2)` array, one row per collective mode found, sorted by frequency: the pole frequency and its signed residual imaginary part (filter on its magnitude to keep only the sharp modes).

`h.get_magnon_bands()`

Compute the magnon bands of a magnetic mean-field state, scanned along a q-path. Three methods, with different domains of validity:

Optional arguments:

- `method="rpa" / "pair" / "tdhf"`: which ladder to sum. See “The three magnon routes” above for the coverage table; in short, “rpa” is the site basis for an onsite `U` and the pair basis for anything that couples different sites, “pair” keeps the interaction’s pair index (any density-density interaction, onsite or not, and exchange through the spin channels the SCF records, metals included, frequency-resolved), “tdhf” solves the electron-hole pair eigenproblem (the same interactions, no frequency grid).
- `"rpa"` takes the poles of the full spin RPA kernel (the same S_x, S_y, S_z channel as `get_spinchi_full/get_iets_ldos`), with the interaction taken from the mean field: the site vertex for an onsite `h.V`, and the pair-basis ladder of `"pair"`, with these same defaults, for a density-density or exchange interaction that couples different sites. It works for metals as well as insulators, and needs a frequency grid. `"tdhf"` solves the time-dependent Hartree-Fock problem in the spin-flip electron-hole pair basis: it handles a neighbor-shell density-density interaction and an exchange one whose spin channels the SCF recorded, has an exact Goldstone mode, needs no frequency grid, and requires a gapped reference converged on the same `nk` (or `metal=True`)

- `qpath=None, nq=20`: the q-path (default path of the geometry) and number of q-points
- `energies, delta, nk`: as above (`method="rpa"` only)
- `nk, n, channel, V`: for `method="tdhf"`, the k-mesh (which must match the SCF's), how many branches to keep per q-point, whether to restrict to the spin-flip block ("`auto`", "`spinflip`", "`all`"), and an interaction overriding `h.V`

Returns `(qs,ws,gammas)` for `method="rpa"`: three flat 1D arrays of equal length, `qs` the integer q-point index along the path, `ws` the pole frequency, `gammas` its residual imaginary part. `method="tdhf"` returns `(qs,es)`, with `es` the (complex) magnon energy.

`h.get_transverse_spinchi()`

Return the spin response computed in the basis of the interaction's *pair* index rather than of sites, which is what lets it carry a neighbor-shell density-density interaction, the one the site-basis RPA maps to exactly zero, and an exchange interaction with its transverse part. Needs no gapped reference and no global spin quantization axis, so it covers metals and non-collinear states alike, and returns a frequency-resolved, spin- and site-resolved $\chi(\omega)$.

Optional arguments:

- `W=None`: the interaction, defaulting to the one the mean field was converged with
- `q=None, energies, delta, nk`: as in `get_chi`, `None` again meaning the q-average
- `component=None`: a pair of spin indices (`a,b`) to return only that spin block instead of the full tensor
- `chi_prec=None`: precision of the Lindhard sum in the pair basis, as in `get_chi`; the CPU kernel of this route is double precision only and raises for "`single`"

Returns `(energies,chi)` with `chi` one $3N \times 3N$ tensor per frequency, in the same $(S_x, S_y, S_z) \times \text{site}$ layout `get_spinchi_full` uses.

`h.get_magnon_energies()`

Return the magnon energies at a single center-of-mass momentum `Q`, from the spin-flip channel of the Bethe-Salpeter equation. Same arguments as `get_magnon_bands(method="tdhf")` with `Q=[qx,qy,qz]` in place of the q-path. A sizable imaginary part on an energy means the mean-field reference is unstable against that excitation.

h.get__goldstone_residual()

Return how far a magnetic mean field is from having a zero-energy magnon at $Q = 0$, as the Goldstone theorem requires of any magnetic state without spin-orbit coupling: $\|Mv\|/\|v\|$ with M the time-dependent Hartree-Fock matrix and v the uniform spin-rotation generator in the pair basis. It is proportional to the SCF tolerance the mean field was converged to and to nothing else, so it is the check to run before trusting a magnon dispersion. In particular it is what catches a mean field converged on a different **nk** than the magnon is being solved on.

Optional arguments:

- **nk=10, V=None, channel="auto"**: as in **get_magnon_energies**
- **relative=True**: divide by the largest transition energy of the pair basis, so the number is comparable across models with different bandwidths

h.get__densitychi_RPA()

Compute the density (charge) RPA response function for a **V1/V2/V3**-neighbor-shell (+ onsite **U**, + general **Vr(r)**) density-density interaction, same convention as **Vinteraction/VJinteraction**. Unlike **get_spinchi_full**, the interaction is taken directly as parameters, not read from **h.V**. No mean-field convergence is needed first.

Optional arguments:

- **V1=0.0, V2=0.0, V3=0.0, U=0.0, Vr=None**: the density-density interaction, built the same way as **Vinteraction/VJinteraction**'s
- **q=None, energies, delta, nk**: as in **get_chi**, **None** again meaning the q-average

h.get__plasmon_bands()

Compute the plasmon/charge-order bands: the poles of the density RPA kernel for a **V1/V2/V3/U/Vr** neighbor-shell density-density interaction, scanned along a q-path, the charge-channel analog of **get_magnon_bands**.

Optional arguments:

- **V1=0.0, V2=0.0, V3=0.0, U=0.0, Vr=None**: as in **get_densitychi_RPA**
- **qpath=None, nq=20, energies, delta, nk**: as in **get_magnon_bands**

Returns (**qs,ws,gammas**), same convention as **get_magnon_bands**.

h.get__bse()

Solve the Bethe-Salpeter equation (excitons) on top of this mean-field Hamiltonian, and return the solved **BSE** object.

Optional arguments:

- `V=None`: the electron-hole interaction. **None** reads it from `h.V`, the interaction the mean field was converged with (so the BSE kernel and the Fock self-energy inside `h` come from the same interaction). Otherwise a real-space dictionary `{(n1,n2,n3): matrix}`, or a plain matrix for an onsite-only interaction, `bsetk.interaction.density_interaction` builds one from `U/V1/V2/V3/Vr`
- `Q=None`: center-of-mass momentum of the exciton, defaulting to the zone center
- `nk=10`: k-points per direction of the mesh the electron-hole pairs are built on
- `nv=None, nc=None`: restrict to the `nv` highest valence and `nc` lowest conduction bands; **None** takes all of them
- `kernel="full"`: which kernel terms to include, **"full"**, **"direct"** (ladder only), **"exchange"** (exactly the RPA) or **"none"** (bare transitions)
- `tda=False`: apply the Tamm-Dancoff approximation, diagonalizing only the resonant block
- `max_memory=2.0`: refuse, rather than attempt, a calculation whose dense matrix would need more than this many GB
- `screening=None`: the interaction of the *direct* (ladder) term. **None** keeps the bare one, i.e. time-dependent Hartree-Fock; **"rpa"** replaces it by the static RPA screened interaction $W = \varepsilon^{-1}v$ built from this Hamiltonian's own bands; **"crpa"** does the same with the transitions inside the `nv/nc` window left out of the polarization; a **ScreenedInteraction** reuses a pre-computed one. The exchange term always keeps the bare interaction. Do not screen a fitted Hubbard `U`, see the section above
- `nkW=None`: k-mesh for the screening, defaulting to `nk`. Must be an integer multiple of it
- `channel="charge"`: where the dielectric matrix is built. **"charge"** builds it on site indices (the standard GW construction, spin-rotation invariant); **"orbital"** dresses the full spin-orbital matrix as $\varepsilon^{-1}v$, which does not preserve `SU(2)`. The two coincide for a spinless Hamiltonian
- `solver="dense"`: how the eigenproblem is solved. **"dense"** builds the matrix and diagonalizes it, and is the only route to the full non-Tamm-Dancoff spectrum. **"iterative"** applies the exactly factorized kernel matrix-free and runs a preconditioned block LOBPCG, removing the memory wall. **"qtt"** compresses the kernel into a quantics matrix product operator and solves it by DMRG, with a cost growing like $\log N_k$. The last two need `tda=True`; see "Large k-meshes" above

- `neig=None`: how many excitons `solver="iterative"` returns; `"dense"` returns all of them and ignores it, and `"qtt"` accepts only `neig=1`. `None` means 4 for `"iterative"` and 1 for `"qtt"`
- `gauge="auto"`: smooth the arbitrary phase left on each Bloch eigenvector. `"auto"` turns it on (as `"projection"`) only for `solver="qtt"`, which cannot work without it; `"phase"`, `"projection"` or `None` apply to every solver. It changes no energy, being a unitary on the pair index

`solver="qtt"` additionally takes `tolerance` (default `1e-6`), `maxbonddim` and `maxdim/nsweep/cutoff`, which trade accuracy against cost, plus `coarse_nk` (the submesh the band window is read from) and `unfolding`.

The returned object exposes `energies`, `amplitudes` (the resonant amplitudes $A_{vc}(k)$), `amplitudesY` (the antiresonant ones, zero under `tda`), `pairs` (the k-mesh, band window and `(ik,iv,ic)` label of every pair index), and the methods `get_energies(n)`, `get_binding_energies(n)` and `get_lowest_transition()`. The last is the lowest independent-particle transition, the energy the binding energies are measured from.

`h.get_exciton_energies()`

Return the exciton energies from the Bethe-Salpeter equation, sorted. Takes the same arguments as `get_bse`, plus `n=None` to keep only the `n` lowest.

`h.get_exciton_binding_energies()`

Return the exciton binding energies, i.e. how far below the lowest independent-particle transition each exciton lies; positive means bound. Same arguments as `get_exciton_energies`.

`h.get_exciton_states()`

Return `(energies,amplitudes)` of the excitons, the amplitudes being the electron-hole amplitudes $A_{vc}(k)$ of each one, indexed by the flattened pair index whose `(ik,iv,ic)` meaning is in the BSE object's `pairs.labels`. Same arguments as `get_exciton_energies`.

`h.get_exciton_bands()`

Return the exciton band structure $E_X(Q)$: one Bethe-Salpeter solve per q-point along a path.

Optional arguments:

- `qpath=None, nq=20`: the q-path, same input as `get_bands` (a list of high-symmetry labels, a list of explicit q-vectors, or `None` for the default path with `nq` points)
- `n=None`: keep only the `n` lowest excitons at each q-point

- `V`, `nk`, `nv`, `nc`, `kernel`, `tda`, `max_memory`, `screening`, `nkW`: as in `get_bse`, passed through unchanged

Returns `(qs, es)`, flat 1D arrays of equal length: `qs` the integer index of the q-point along the path (same convention as `get_bands`) and `es` the exciton energy, complex at any q-point where the mean-field reference is unstable against the excitation. Note that `nv/nc` must not split a degenerate multiplet (a spinful Hamiltonian with no spin-orbit coupling or magnetic order needs an even `nv/nc`); a warning is issued if they do.

`h.get_screened_interaction()`

Return the static RPA screened interaction $W(q) = \epsilon^{-1}(q)v(q)$ built from this Hamiltonian's own bands, as a `ScreenedInteraction`.

Optional arguments:

- `V=None`: the *bare* interaction to screen, same forms as `get_bse`'s `V`. `None` reads `h.V`, but note that a fitted Hubbard `U` is already an effective screened interaction and should not be screened again
- `nk=10`: k-mesh for both the Brillouin zone sum and the q-grid `W` is tabulated on
- `screening="rpa"`: `"rpa"` (all transitions polarize) or `"crpa"` (those inside `exclude` do not)
- `exclude=None`: (`vbands, cbands`) to leave out of the polarization, required by `"crpa"`
- `channel="charge"`: `"charge"` (standard GW, on site indices, spin-rotation invariant) or `"orbital"` (the full spin-orbital matrix, which breaks $SU(2)$)
- `chi_prec=None`: precision of the Brillouin zone sum that builds χ_0 , `"single"` or `"double"`; unset, it is `"single"` on the GPU and `"double"` on the CPU, whose kernel is double precision only and raises for `"single"`. Where that sum runs is the package-wide switch `gpu.set_gpu` (see "Running on a GPU"). This is the one sum in the library that a consumer card computes *more slowly* in double precision than the processor does, so on such a card leave it unset

The returned object exposes `qs`, `Wq`, `chi0`, `bare`, `epsmin` (the smallest dielectric eigenvalue found over the mesh), `.at(q)` for the value at a mesh q-point and `.get_dict()` for the inverse Fourier transform back to a real-space interaction. Raises if an eigenvalue of $\epsilon(q)$ reaches zero, which is a charge or spin instability of the mean field at that wavevector.

`h.get_polarizability()`

Return `(qs, chi0)`, the static polarizability of this Hamiltonian on its k-mesh, in the spin-orbital basis; `chi0` has shape `(nq, norb, norb)`. Takes

`nk`, `exclude`, `chi_prec` and the precomputed-eigenstate arguments of `get_screened_interaction`.

h.get_fermi_surface()

Compute the spectral weight on a 2D k-mesh at a single energy.

Optional arguments:

- `e=0.0`: energy of the cut
- `nk=50`: k-points per direction
- `delta`: broadening
- `operator=None`: project/weight by an operator (e.g. `"sz"`, `"valley"`, `"unfold"`)

Returns `kx`, `ky` and the Fermi-surface weight

h.get_multi_fermi_surface()

Compute the Fermi surface at many energies, writing one file per energy to a `MULTIFERMISURFACE/` folder.

Optional arguments:

- `energies=[0.0]`: energies to compute
- `nk`, `delta`, `operator`: as in `get_fermi_surface`

h.get_surface_kdos()

Compute the surface and bulk spectral function of a semi-infinite system, from the surface Green's function (renormalization/decimation technique).

Optional arguments:

- `kpath`: k-point path (auto-generated if not given)
- `energies`, `delta`: frequency range, broadening

Returns `k`, energy, surface spectral weight and bulk spectral weight; also writes `KDOS.OUT`

h.get_qpi()

Compute the quasiparticle-interference map (2D systems only). Writes output to disk (default `MULTIQPI/` folder plus `DOS.OUT`) rather than returning arrays.

Optional arguments:

- `energies`, `nk`, `delta`: as above

- mode="response": "pm" ("poor man's", autoconvolves the actual k-resolved spectral weight, the physical QPI of a real scatterer) or "response" (cheaper Lindhard-like joint-DOS convolution of the clean bands)
- nunfold=1: unfold the QPI of a defect embedded in an `nunfoldxnunfold` supercell back onto the primitive Brillouin zone

`h.get_qpi_impurity()`

Compute quasiparticle interference by placing real-space impurities in a supercell, computing the real-space LDOS by partial diagonalization, and Fourier transforming it directly (2D systems only). Returns `(r,ldos_r,q,qpi_q)`.

Optional arguments:

- `nsuper=10`: supercell size (scalar or `(n1,n2)`)
- `impurities=[]`: list of dicts, each `{"position":"index": ..., "onsite": v}` or `{"position":"index": ..., "vacancy": True}`
- `energies=0.0`, `delta`, `nk`: as above
- `num_waves=20`: starting number of eigenstates computed nearest the requested energies. It grows automatically until the window reaches `margin` (default 5.0) times `delta` past every requested energy, without stopping inside a degenerate manifold, so too small a value costs time, not correctness
- `write=True`, `output_folder="QPI_IMPURITY"`: also write the MULTIQPI-style disk output

`h.get_spin_splitting_density()`

Compute the energy-resolved spin splitting of a collinear magnet as a smooth weighted density: every band pair contributes its squared splitting, broadened, at the mean energy of the pair. Returns `(energies,values)`.

Optional arguments:

- `nk=20`: number of k-points per direction
- `energies`: energies at which the density is evaluated (default 400 points spanning -3 to 3)
- `delta=1e-2`: broadening
- `tol=1e-7`: largest spin off-diagonal element of the Bloch Hamiltonian tolerated, the same guard `get_spin_splitting_vs_energy` applies; above it this raises rather than answering

`h.get_average_spin_splitting()`

Return a single number: the spin splitting of a typical band, averaged over bands as well as over the Brillouin zone. Being an average rather than a sum it is intensive, so the same crystal described in a larger cell gives the same answer.

Optional arguments:

- `nk=20`: number of k-points per direction
- `tol=1e-7`: collinearity tolerance, as above

Use `h.get_spin_splitting_vs_energy()` instead when what is wanted is the largest splitting anywhere in the zone rather than the typical one.

`h.get_spin_splitting_vs_energy()`

Compute the energy-resolved **maximum** spin splitting over the Brillouin zone, so that the largest value returned bounds the spin splitting anywhere in the zone. Bands are paired by index within each spin channel and binned at the mean energy of the pair. Returns `(energies, values)`, the same convention as `get_spin_splitting_density`, so the two are directly comparable.

Optional arguments:

- `nk=100`: linear mesh density (nk^d points in d dimensions)
- `energies=None`: explicit bin centers; otherwise `nbins` points spanning `emin` to `emax`
- `nbins=400`, `emin=None`, `emax=None`: used when `energies` is not given; the default range is that of the mean energies actually found, so no state falls outside the window
- `tol=1e-7`: largest spin off-diagonal element of the Bloch Hamiltonian tolerated. Above it this raises rather than answering, since the spin-resolved splitting is not meaningful with spin-orbit coupling or non-collinear order

Empty bins are 0.0 (not NaN), and states outside an explicitly requested window are dropped rather than clamped onto the end bins. Diagonalization is dense throughout, since a sparse solver would return only the eigenvalues nearest $E=0$ and could miss the peak.

`h.get_chern()`

Return Chern number of the Hamiltonian.

Optional arguments: - `nk`: number of kpoints, 10 for the default `integration="grid"` and 20 for `integration="qtci"` - `integration="grid"`: how the Brillouin-zone integral is evaluated. “grid” (default) sums the Berry curvature over a uniform $nk \times nk$ mesh; “qtci” integrates it by quantics tensor cross interpolation plus Gauss-Kronrod quadrature, sampling adaptively instead of uniformly; accurate for a smooth curvature (a large gap) and unreliable for a sharply peaked one, where the mesh sum stays exactly quantized. See “Tensor-cross-interpolation (qtci) integration” - `operator=None`: a name, a matrix or an `Operator`, as for `h.get_berry_curvature()`. The operator-projected invariants (`topology.spin_chern`, `topology.operator_berry`) work on sparse Hamiltonians too, which is what a moire or supercell model is

`h.get__berry__curvature()`

Return the Berry curvature of the occupied bands as a map over the Brillouin zone, (`kx,ky,berry`). Three flat arrays, so it goes straight into a `plt.scatter(kx,ky,c=berry)` or, after reshaping to (`nk,nk`), into a `contourf`. This is the same curvature that `h.get_chern()` integrates.

Optional arguments:

- `nk=100`: linear k-point density of the map (the map has `nk*nk` points)
- `reciprocal=True`: return `kx,ky` in Cartesian reciprocal coordinates, which is what you want to plot a hexagonal Brillouin zone undistorted. Pass `reciprocal=False` for fractional coordinates instead, in which the curvature integrates to the Chern number directly: with `nsuper=1` the map covers $[-1,1)$ along both fractional directions, i.e. four Brillouin zones, so `np.sum(berry)*(2/nk)**2/(2*np.pi)` comes out at four times `h.get_chern()`
- `mode="Wilson"`: how the curvature is evaluated. "Wilson" uses the Fukui-Hatsugai-Suzuki plaquette construction; "Green" uses the Green's-function Kubo formula and is selected automatically when `operator` is given
- `operator=None`: restrict the curvature to a subspace, e.g. "valley" for a valley-resolved curvature (see "Berry curvature operator"). A name, a matrix or an `Operator` are all accepted, the same spellings `h.get_bands(operator=...)` takes; the same holds for `h.get_chern()`, `topology.chern_density` and `topology.chern_qtci`
- `nsuper=1`: extend the map over this many Brillouin zones
- `kpath`: compute along a k-path instead of over a 2D grid
- `delta=0.001`: broadening used by the Green's-function mode

`h.get__quantum__geometric__tensor()`

Return the (multiband/multiorbital) quantum geometric tensor at a single k-point, see "Quantum geometric tensor (multiorbital/multiband)".

Optional arguments: - `k=[0.,0.,0.]`: k-point - `occ_idxs=None`: band indices of the chosen subspace (default: the bands with $E < 0$, the same Fermi-level convention `h.get_chern()` uses, so this tracks `h.shift_fermi(...)`) - `non_abelian=False`: if True, return the full tensor in the orbital basis, $\sum_{m,n \in S} |u_m\rangle Q^{mn} \langle u_n|$, an $n \times n$ matrix per pair of directions that does not depend on the basis chosen inside a degenerate subspace, instead of its trace over the subspace - `degeneracy_tol=1e-8`: energy tolerance used to detect a degeneracy between the chosen subspace and its complement (raises `ValueError`)

h.get_quantum_metric()

Same arguments as **h.get_quantum_geometric_tensor()**, but returns only the quantum metric (symmetric part of the tensor).

h.get_dvector_non_unitarity()

Non-unitarity vector $\vec{q} = i(\vec{d} \times \vec{d}^*)$ of the spin-triplet d-vector of a BdG (Nambu) Hamiltonian, resolved per site (see “Spin-triplet d-vector and non-unitary superconductivity”). It is real, vanishes for a unitary state, and otherwise is the spin moment of the Cooper pairs, parallel to the magnetization in a ferromagnetic spin-triplet superconductor, and along $+z$ for a pure $\Delta_{\uparrow\uparrow}$ pairing.

Optional arguments

- nk = 10: k-points per periodic direction

Returns an array of shape (number of sites, 3).

h.write_non_unitarity()

Write $\vec{q}$ as a real-space map in **NON_UNITARITY_MAP.OUT**, with columns (x, y, z, q_x, q_y, q_z) .

Optional arguments

- nrep = 2: number of replicas written per direction
- nk = 10: k-points per periodic direction

h.get_average_dvector()

k-averaged squared components $(|d_x|^2, |d_y|^2, |d_z|^2)$ of the spin-triplet d-vector of a BdG (Nambu) Hamiltonian.

Optional arguments

- nk = 10: k-points per periodic direction
- spatial_sum = True: average over sites, returning a single 3-component vector
- non_unitarity = False: average the squared components of $\vec{q}$ instead of those of $\vec{d}$. This is a magnitude only and carries no sign, so use it to ask whether the state is non-unitary rather than in which direction; use **h.get_dvector_non_unitarity()** for the signed vector.

h.get_superfluid_weight()

Superfluid weight tensor D_s^{ab} of a BdG (Nambu) Hamiltonian of dimensionality 1, 2 or 3 (see “Superfluid weight and BKT temperature”).

Optional arguments:

- nk=20: k-points per periodic direction

- $T=0.0$: temperature (see the caveat above for gapless normal states at $T=0$)
- `mode="kubo"`: "kubo" for the analytic multiband formula, "finite_difference" for the assumption-free numerical derivative of the grand potential
- `gauge="atomic"`: "atomic" twists with the full bond vector (the physical Peierls substitution), "lattice" with the lattice vector alone (the cell-gauge convention)
- `decompose=False`: if True, return a dict with "total", "conventional", "geometric", "delta" and "gauge" instead of a bare tensor

Returns a Cartesian (`dim,dim`) array, or a dictionary if `decompose=True`. `decompose=True` raises `ValueError` when the decomposition's assumptions do not hold.

`h.get__bkt__temperature()`

Berezinskii-Kosterlitz-Thouless temperature of a 2d BdG Hamiltonian, from the self-consistent Nelson-Kosterlitz criterion $T_{\text{BKT}} = (\pi/8)D_s(T_{\text{BKT}})$ at frozen $|\Delta|$. Arguments `nk=20`, `tmax=None`, `tol=1e-6`, `maxite=60`, plus `gauge`. Returns a float.

`h.get__nonlinear__drude__conductivity()`

Compute the l -th order nonlinear Drude conductivity $\sigma^{\{x^{l1} y^{l2} ; b\}}$ of a collinear magnet, the quantity whose lowest nonvanishing order measures the X-wave index of an altermagnet (`p:0`, `d:1`, `f:2`, `g:3`, `i:5`). Requires spin to be a good quantum number.

Optional arguments

- `field = "x"`: one character per power of the electric field, e.g. "yyyyy" for the fifth-order response to E_y
- `current = "x"`: Cartesian direction of the measured current
- `channel = "spin"`: "spin", "charge", "up" or "dn"
- `nk = 100`: k-points per periodic direction
- `T = 0.01`: temperature of the Fermi occupations
- `mu = 0.0`: chemical potential
- `tau = 1.0`: relaxation time; the l -th order response scales as τ^l
- `omega = 0.0`: frequency, entering as $1/(i\omega + 1/\tau)^l$
- `degeneracy_tol = 1e-8`: multiorbital cells only, the relative band spacing below which bands are grouped into a degenerate multiplet

Returns a complex scalar

`h.get__nonlinear__drude__components()`

Every component of the l -th order nonlinear Drude conductivity, as a dict $\{x^{l1} y^{l2}; b\}: \text{value}\}$. Shares one k-mesh across the components, so it is much cheaper than the equivalent individual calls.

h.get__nonlinear__drude__orders()

X-wave selection-rule sweep: for each order $l = 0..lmax$, the largest $|\sigma^{x^{l1} y^{l2} ; b}|$ over that order's components. The lowest l with a nonzero entry is the wave index readout.

Optional arguments

- $lmax = 6$: highest order to sweep
- plus every optional argument of `h.get__nonlinear__drude__conductivity()`

Returns a list of floats

h.get__optical__conductivity()

Frequency-dependent conductivity tensor $\sigma_{ab}(\omega)$ in the Kubo-Greenwood formalism (see “Optical conductivity”).

Optional arguments:

- `energies=None`: frequencies at which to evaluate σ
- `nk=20`: k-points per periodic direction
- `T=None`: temperature (defaults to `delta`)
- `delta=0.1`: Lorentzian broadening η
- `intraband=True, interband=True`: switch the two channels independently
- `component=None`: e.g. “xy” to return that component alone
- `degeneracy_tol=1e-6`: relative tolerance for treating a pair as degenerate

Returns `(energies,sigma)` with `sigma` of shape `(nw,3,3)` and complex, or `(nw,)` if `component` is given. Units of $e^2/\hbar$ (so e^2/h is $1/(2\pi)$ of it). Raises `NotImplementedError` for 3d and Nambu Hamiltonians.

h.get__drude__weight()

Drude (intraband) weight tensor. Arguments `nk=20, T=0.05, degeneracy_tol=1e-6`. Returns a real `(3,3)` array.

h.get__sum__rule__weight()

Diamagnetic weight tensor W entering the optical f-sum rule $\int \text{Re } \sigma_{aa}(\omega) d\omega = \pi W_{aa}$. Arguments `nk=20, T=0.05`. Returns a real symmetric `(3,3)` array.

h.get__entanglement__entropy()

Entanglement entropy of a real-space region, from the eigenvalues of the region-restricted one-particle correlation matrix (see “Entanglement entropy and entanglement spectrum”).

Optional arguments:

- `region=None`: the region, as a list of site indices, a boolean mask, a callable on positions (the `sculpt` convention) or a float fraction of the cells; `None` takes half the system
- `nsuper=10`: unit cells stacked into the ring that is cut (periodic Hamiltonians)
- `direction=None`: lattice direction normal to the cut, defaults to the last periodic one
- `kpar=None`: momentum parallel to the cut; `None` on a 2d Hamiltonian averages over an `nk` mesh
- `nk=20`: k-points used for that average
- `fermi=0.0`: occupied states are those with $E < \text{fermi}$ (must stay 0 for BdG)

Returns a float. A level exactly at the Fermi energy raises rather than returning the entropy of an arbitrary determinant.

`h.get_entanglement_spectrum()`

Single-particle entanglement Hamiltonian eigenvalues $\xi_n = \ln[(1 - \zeta_n)/\zeta_n]$ of a real-space region. Same arguments as `h.get_entanglement_entropy()` (with `nk=41` by default).

Returns a sorted array of ξ_n for a 0d/1d Hamiltonian or a single `kpar`; for a 2d Hamiltonian with `kpar=None` it returns `(ks,xis)` with `xis` of shape `(nk,nA)`, the Li-Haldane entanglement spectrum across the BZ.

`h.get_wannier_hamiltonian()`

Wannierize a range of bands and return the resulting real-space Hamiltonian.

Arguments:

- `bands = [a,b]`: first and last band to Wannierize (0-indexed, both ends inclusive)

Optional arguments:

- `nk=12`: k-points per periodic direction for the wannierization mesh
- `cutoff=1e-6`: real-space hoppings smaller than this are dropped (the intra-cell block is always kept). Set it to `0.0` to keep the reproduction exact to machine precision
- `symmetries=None`: `"auto"` to auto-detect and enforce the point group, or an explicit list of `symmetrytk.pointgroup.SymmetryOperation`
- `num_wann=None`: how many Wannier functions to extract. `None` means the whole selected range, Wannierized as a fixed subspace and reproduced exactly. A smaller value switches on Souza-Marzari-Vanderbilt disentanglement, after which only the frozen window is reproduced (see “Disentanglement”). Outside `1..len(bands)` raises `ValueError`
- `dis_win_min, dis_win_max=None`: outer energy window, i.e. which bands are offered to the extraction at each k-point. Every eigenvalue of

the result lies inside it

- `dis_froz_min`, `dis_froz_max=None`: frozen inner window, whose states are reproduced exactly at every mesh k-point. `dis_froz_max` alone is enough; `dis_froz_min` alone raises `ValueError`
- `dis_num_iter=200`: maximum iterations of the Ω_I minimization; not converging within them is a warning, not an error

Any `dis_*` argument given without a smaller `num_wann` raises `ValueError`. Disentanglement together with a Nambu/BdG Hamiltonian, with `symmetries=`, or with `auto_split_clusters=True` raises `NotImplementedError`.

Returns a new, smaller Hamiltonian; `.wannier_centres`, `.wannier_spreads` and `.wannier_spread_total` hold the Wannier-function geometry, and `.wannier_num_wann/.wannier_disentanglement_window` record what was extracted and through which window (the latter `None` when not disentangling)

h.get_szsz_mean_field_hamiltonian()

Self-consistent Hartree-Fock mean field for a $J_z \sum S_i^z S_j^z$ spin-spin exchange interaction (see “Spin-spin exchange interactions”). $J > 0$ is antiferromagnetic, $J < 0$ ferromagnetic.

Optional arguments:

- `J1, J2, J3 = 0.`: first/second/third-neighbor J_z couplings
- `Jr=None`: general distance-dependent coupling function, as `Vr` for `get_mean_field_hamiltonian`
- `filling, mf, nk, maxerror, mix, constrains`: as in `get_mean_field_hamiltonian`
- `return_total_energy=False`: also return the total energy

Also works on BdG (Nambu) Hamiltonians, decoupling both the normal and anomalous (pairing) channels.

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

h.get_sxsx_mean_field_hamiltonian() / h.get_sysy_mean_field_hamiltonian()

Same as `get_szsz_mean_field_hamiltonian()`, for a $S_i^x S_j^x / S_i^y S_j^y$ interaction instead, implemented by rotating the problem so that x (or y) becomes the computational z axis, solving there, and rotating the converged Hamiltonian back. Also works on BdG Hamiltonians.

h.get_exchange_mean_field_hamiltonian()

Self-consistent anisotropic exchange mean field, combining $J_x S_i^x S_j^x + J_y S_i^y S_j^y + J_z S_i^z S_j^z$ in a single SCF loop.

Optional arguments:

- `Jx1, Jx2, Jx3, Jy1, Jy2, Jy3, Jz1, Jz2, Jz3 = 0.`: first/second/third-neighbor couplings for each axis

- Jxr, Jyr, Jzr=None: general distance-dependent couplings, one per axis
- mf, filling, nk, maxerror, mix, constrains: as above (only `integration="ed"` and the plain-mixing solver are supported)

Also works on BdG Hamiltonians, with the same full normal-plus-anomalous decoupling as `get_combined_mean_field_hamiltonian`'s density-density channels. Exchange can itself induce superconducting pairing (e.g. an antiferromagnetic isotropic J alone, seeded with a random guess, can spontaneously decouple into a purely superconducting state).

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

`h.get_combined_mean_field_hamiltonian()`

Self-consistent mean field combining density-density interactions (onsite U , $V_1/V_2/V_3/V_r$ neighbor-shell) with spin-spin exchange in a single SCF loop.

Optional arguments:

- U , V_1 , V_2 , V_3 , V_r : as in `get_mean_field_hamiltonian`
- J_1 , J_2 , $J_3 = 0.$: isotropic Heisenberg exchange for the first/second/third-neighbor shells (same shell convention as $V_1/V_2/V_3$)
- J_r =None: general distance-dependent isotropic exchange function, as V_r
- J_{1x} , J_{1y} , $J_{1z} = 0.$: optional anisotropic correction added to J_1 on the first-neighbor shell only (e.g. the effective first-neighbor J_z coupling is J_1+J_{1z}); second/third neighbors stay purely isotropic
- mf, filling, nk, maxerror, mix, constrains: as above (only the plain-mixing solver is supported)
- integration="ed": compute the density matrix at each SCF iteration by exact diagonalization. "kpm" uses a per-k Chebyshev-moment (Kernel Polynomial Method) expansion instead, never diagonalizing the Bloch Hamiltonian $H(k)$, for systems too large to diagonalize or to hold as a dense matrix (a big 0D flake, say); normal-state (non-BdG) Hamiltonians only. With "kpm", `scale=None` sets the KPM energy rescaling (estimated automatically if not given), `npol` the number of Chebyshev moments, `ne` the number of energies sampled in the occupied window and `cores` the number of parallel workers across k-points; all four are unused for "ed". Also reachable through `h.get_mean_field_hamiltonian(integration="kpm", ...)`.
- `use_jax=True`, `solver=...`: solve the same SCF fixed point $x = f(x)$ (x the mean-field parameters, f one SCF iteration) with a nonlinear solver instead of plain mixing. "error_gradient" is the most robust of these on a generic Hamiltonian, "newton" the default; as local methods they can still stall short of `maxerror`, in which case the call returns `None` like any other non-converged SCF, so always check for that. Restricted to

a normal-state (non-BdG) Hamiltonian, dense exact diagonalization only (no `integration="kpm"`), and no `constrains`:

```
from pyqula import geometry
h = geometry.chain().get_hamiltonian()
hmf = h.get_combined_mean_field_hamiltonian(U=4.0, J1=-0.5, filling=0.5,
      use_jax=True, solver="newton") # JAX-derivative-based SCF solver
```

On a BdG Hamiltonian every channel keeps the full normal+anomalous (pairing) treatment: $U/V_1/V_2/V_3/V_r$ as in `get_mean_field_hamiltonian`, and the exchange (J) channels identically, so exchange alone can induce superconducting pairing rather than only magnetism (see “Spin-spin exchange interactions” above).

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

`filling` also accepts a per-site array (length `len(h.geometry.r)`, same 0-to-1 fraction of the two states per site as the scalar case) instead of a single lattice-averaged value, enforcing $\langle n_i \rangle = \text{filling}[i]$ at every site independently through a per-site Lagrange multiplier co-converged with the mean field in the same SCF loop. The converged per-site potentials are left on the `fermi` attribute of the returned Hamiltonian, and a `None` return means that either the mean field or the per-site constraint did not converge to within `maxerror`. Only supported for a normal-state (non-BdG), `integration="ed"` Hamiltonian with `mu=None` (the default). This is the mechanism `SpinonHamiltonian` (see “Abrikosov-pseudofermion (spinon) mean field for Heisenberg models”) builds on to enforce exactly one auxiliary fermion per site.

SpinonHamiltonian(g)

Abrikosov-pseudofermion (RVB) mean-field Hamiltonian for a spin- $\frac{1}{2}$ Heisenberg model on geometry `g`. See “Abrikosov-pseudofermion (spinon) mean field for Heisenberg models” above. `from pyqula.spinon import SpinonHamiltonian`; built with zero bare hopping, couplings supplied through `get_mean_field_hamiltonian`’s usual `J1/J2/J3/Jr/J1x/ J1y/J1z` kwargs.

- `h.get_mean_field_hamiltonian(J1=...,nk=...,...)`: same SCF kwargs as `get_combined_mean_field_hamiltonian` above, except `filling` cannot be passed (always exactly one fermion/site, enforced site-by-site). Returns the converged Hamiltonian (or `None` if the SCF did not converge), with two extra diagnostic attributes:
 - `h2.local_occupation`: converged $\langle n_i \rangle$ per site (electron-count convention, 0 to 2, target is exactly 1.0)
 - `h2.constraint_lambda`: converged per-site Lagrange multiplier (local chemical potential)

KondoLatticeHamiltonian(hc)

Abrikosov-pseudofermion (Read-Newns) mean-field Hamiltonian for the Kondo lattice / periodic Anderson model built from a conduction-electron Hamiltonian `hc`. See “Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice” above. `from pyqula.kondolattice import KondoLatticeHamiltonian`; fuses a second, zero-bare-hopping f-sublattice onto `hc`’s geometry, with the Kondo coupling supplied through `get_mean_field_hamiltonian`’s `J` kwarg.

- `h.get_mean_field_hamiltonian(J=...,filling=...,mf=(V,lam),nk=...,...)`: self-consistently solves for the hybridization and the local constraint’s Lagrange multiplier. Returns the converged Hamiltonian (or `None` if the SCF did not converge), with three extra diagnostic attributes:
 - `h2.local_occupation`: converged $\langle n_f \rangle$ per localized site (target is exactly 1.0)
 - `h2.hybridization`: converged V_j per localized site
 - `h2.constraint_lambda`: converged per-site Lagrange multiplier

GrapheneGeometry(g)

`Geometry` subclass wrapping a graphene multilayer geometry `g` (bilayer, twisted bilayer, twisted trilayer, ...), adding a `.relax()` method, see “Twisted bilayer graphene structural relaxation” above. `from pyqula.graphenetc.geometry import GrapheneGeometry`; `g` must have `has_sublattice=True` (raises `ValueError` otherwise).

- `g.relax(nrep=1,maxiter=500,verbose=False,layer_pairs=None,gsfe_coeffs=...,elastic_coeff=...)`: minimizes the GSFE (interlayer) + elastic (intralayer) energy over an in-plane displacement field and returns a new, relaxed `GrapheneGeometry`. `layer_pairs` selects which (lower,upper) pairs of layers (ordered by `z`) get an interlayer GSFE term, defaulting to all adjacent pairs; `gsfe_coeffs/elastic_coeffs` override the built-in graphene constants.

GrapheneHamiltonian(geometry)

Hamiltonian built from a graphene multilayer `geometry` (typically a `GrapheneGeometry`, relaxed or not), see “Twisted bilayer graphene structural relaxation” above. `from pyqula.graphenetc.hamiltonian import GrapheneHamiltonian`; defaults to the distance-decaying hoppings of `specialhopping.twisted_matrix` (`ti=0.12,lambi=8.0,lamb=12.0,d1=3.0`, matching `specialhamiltonian.twisted_bilayer_graphene`’s defaults) rather than the generic first-neighbor default, so relaxed (in-plane displaced) positions feed into the electronic structure automatically. Pass `mgenerator=...` to use a different hopping generator instead.

h.get_central_heterostructure()

Build a two-terminal **Heterostructure** using **h** (a finite, 0d Hamiltonian) as the central scattering region, contacted by two semi-infinite 1D chain leads attached at sites **i**/**j** (see “Transport through an arbitrary finite region” above).

Optional arguments:

- **i=0, j=None**: 0-indexed sites **h** is contacted at; **j** defaults to the last site
- **left=None, right=None**: lead Hamiltonians; default to a plain spinless **geometry.chain()**. Give one of them (or **h** itself) nonzero pairing (**add_swave**) for a normal-superconductor junction. At most one of **{h, left, right}** may carry pairing

Returns a **Heterostructure**, so **didv**, **get_dos**, **get_kappa**, etc. all apply unmodified; **landauer** is the exception, raising **NotImplementedError** once any of the three carries pairing (use **didv** there). Only 0d central regions are supported so far (**h.dimensionality>0** raises **NotImplementedError**).

Heterostructure functions and methods

HT.get_dc_current()

Compute the time-averaged (DC) current through a two-terminal junction at a given bias, using the Floquet-Keldysh formalism (see “Multiple Andreev reflection and AC-Josephson current”). Works for any combination of normal/superconducting leads built with **heterostructures.build(h1,h2)**, with or without an explicit **central=** Hamiltonian (the latter is solved by a general dense Floquet inversion, and assumes the bias drops across the junction’s rightmost bond).

Arguments:

- **voltage**: bias voltage

Optional arguments:

- **nmax=6, nmax_max=40**: initial/maximum number of Floquet sidebands (increased adaptively until convergence; a warning is issued if **nmax_max** is reached before **tol** is satisfied)
- **tol=1e-3**: relative convergence tolerance used to decide when to stop increasing the sideband count
- **temperature=0.**: lead temperature
- **delta=None**: broadening (defaults to **HT.delta**); should be much smaller than the smallest relevant gap for small-gap superconductors

Returns the DC current

HT.get_iv_curve()

Convenience wrapper: `get_dc_current` evaluated over an array of voltages.

Arguments:

- `voltages`: array of bias voltages

Returns an array of DC currents

HT.didv_curve() / lp.didv_curve()

Convenience wrapper: `didv` evaluated over an array of energies, in parallel, the array-native equivalent of `[ht.didv(energy=e) for e in es]`. Also reachable as `didv(energies=...)` (mutually exclusive with `didv`'s scalar `energy=...`, same convention as `get_kappa`). If `use_aaa=True` and the sweep resolves to the Floquet-Keldysh method, one AAA self-energy interpolant is built and shared across the whole sweep.

Arguments:

- `energies`: array of bias energies
- any keyword argument accepted by `didv` (`method`, `delta`, `use_aaa`, `nmax_max`, `temp`, ...)

Returns an array of dI/dV values

`T` means different things on the two classes. On a `Heterostructure`, `T` is the temperature. On a `LocalProbe` it is the probe *transparency*, the same knob as `LocalProbe(...,T=...)`, `set_coupling` and `get_kappa(T=...)`, and the temperature there is `temp` (or its alias `temperature`). Both `lp.didv(T=...)` and `lp.didv_curve(...,T=...)` honour it for that call alone, leaving the probe itself untouched.

`delta=` passed to any of `didv`, `didv_curve` or `get_smatrix` likewise applies to that call, self-energies included, taking precedence over the junction's own attribute. For a `LocalProbe` it sets both the probe broadening and the sample's bulk broadening, exactly as the constructor argument does. `HT.with_delta(delta)`, `lp.with_delta(delta)` and `lp.with_coupling(T)` return a shallow copy with that one knob rebound, for when the same setting is wanted across several calls.

HT.get_kappa()

Compute the superconducting/normal conductance power-law-ratio “kappa” diagnostic (also available as `LocalProbe.get_kappa()`, see “Multiple Andreev reflection and AC-Josephson current”).

Optional arguments:

- `energy=0.0`: energy at which to evaluate kappa (returns a scalar)

- `energies`: array of energies to evaluate at once instead (returns an array); mutually exclusive with `energy`
- `temp=0.`: temperature. The default is the zero-temperature result; a nonzero value thermally averages each conductance entering the power-law fit
- `T=1e-2`: reference coupling the power-law exponent is extracted around

Returns `kappa` (a scalar, or an array matching `energies`)

At `temp=0.` (the default), `kappa` is $d(\log G)/d(\log T)$: how steeply the conductance scales with the probe-sample coupling. For a `LocalProbe` whose probe lead is not itself superconducting it is an exact derivative, obtained with `jax.grad`; otherwise it is estimated from the conductance at two nearby couplings (0.9T and 1.1T). Which of the two applies is decided automatically, and the two agree to within the finite-difference bias of the secant. See `examples/transport/localprobe_kappa_1D` for a runnable version.

SpinModel functions and methods

`sm.add_heisenberg()`

Add shell-based exchange couplings to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- `Jij=None`: list of shell couplings, e.g. `[J1,J2,J3]` for first/second/third neighbor exchange; passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too (`None` is that constructor's own default, first-neighbor hopping)
- `Jm=[1.,1.,1.]`: diagonal (J_x, J_y, J_z) weights, applied on top of `Jij`; use unequal values for XXZ/Ising-like anisotropy

Modifies `sm` in place, no return value

`sm.add_field()`

Add a uniform Zeeman field to every site's `sm.b`. Repeated calls accumulate (not overwrite).

Arguments:

- `v`: 3-vector field, e.g. `[0.,0.,1.]`

`sm.add_tensor()` / `sm.add_tensor_2d()`

Add a general pairwise tensor coupling J_{ij} from a function `fun(r1,r2) -> 3x3 matrix` (see `generating_functions` below), rather than the fixed-form couplings `add_heisenberg` builds. `add_tensor` only considers pairs within the home cell; `add_tensor_2d` additionally sums periodic images `ia*a1 + ja*a2`

for `ia,ja` in `[-ncells,ncells]`, and can rotate the coupling tensor of each image about z by `vspiral[0]*ia + vspiral[1]*ja` (in units of π), a way to embed a spin-spiral wavevector directly into the exchange tensor.

Arguments:

- `fun`: coupling function, e.g. one returned by `generating_functions`

Optional arguments (`add_tensor_2d` only):

- `ncells=1`: number of periodic images to sum on each side, along each lattice vector
- `vspiral=[0.,0.]`: per-image in-plane rotation angle coefficients, see above

Modifies `sm` in place, no return value

`classicalspin.generating_functions()`

Factory returning a `fun(r1,r2) -> 3x3 matrix` two-point coupling function for a standard exchange form, for use with `add_tensor/add_tensor_2d`.

Optional arguments:

- `name="Heisenberg"`: one of "Heisenberg" (isotropic, cutoff `fc(distance)`), "Linear" (dipolar $1/r^3$ tensor), "RKKYTI" (RKKY on a topological-insulator surface, PRB 81 233405), "ZZ" (Ising, $S_i^z S_j^z$ only), "XYZ" (diagonal, anisotropic weights `v`), "DM" (Dzyaloshinskii-Moriya, with `v` the intermediate-ion/mirror direction, or a function of the bond vector)
- `J=1.0`: overall coupling strength
- `v=[0.,0.,1.]`: form-dependent vector (or vector-valued function of the bond vector), see above
- `fc=None`: distance-dependent cutoff/envelope, e.g. restricting "Heisenberg"/"ZZ" to first neighbors ($0.9 < d < 1.1$, the default)
- `fdiff=lambda x,y: x-y`: how the bond vector is computed from `(r1,r2)`; override for e.g. a minimum-image convention
- `fr=None`: extra rotation matrix `fr(r1,r2)` applied on top of the base coupling (defaults to the identity)

Returns the coupling function

`sm.minimize_energy()`

Multistart local minimization of the classical energy over every spin's (θ, ϕ) angles, by local gradient-based minimization of the energy (or, if `calle` is given, gradient-free Powell). Only the ground state at Γ is found. There are no twisted boundary conditions, so an incommensurate (e.g. spiral) texture needs an explicit supercell that fits it.

Optional arguments:

- `theta0=None, phi0=None`: initial angles; each try re-randomizes them in $[0, \pi]/[0, 2\pi]$ when left as `None` (the default), which is what makes `tries` explore different basins; passing explicit arrays starts every try from the same point (the optimizer itself is deterministic), so `tries>1` is only useful with the default
- `tries=10`: number of independent minimizations; the lowest-energy one is kept
- `calle=None`: optional extra function `calle(sm) -> float` added to the energy during minimization, e.g. a penalty favoring a particular texture

Updates `sm.theta`, `sm.phi` and `sm.magnetization` in place to the best try found, and returns `(theta, phi)`

`sm.get_energy() / sm.energy()`

Evaluate the total energy $\sum_i \vec{b}_i \cdot \vec{S}_i + \sum_{ij} \vec{S}_i \cdot J_{ij} \cdot \vec{S}_j$ of the current `sm.theta/sm.phi` snapshot. Returns a scalar.

`sm.get_local_energy()`

Per-site breakdown of the current snapshot's energy: each site's own field term plus half of every exchange term it takes part in (as with `LatticeGas.get_local_energy()`, each bond's energy is split evenly between its two endpoints), so the values sum exactly to `sm.get_energy()`.

Returns an array over sites

`sm.get_magnetization()`

Convert the current `sm.theta/sm.phi` angles to Cartesian components.

Returns `(mx, my, mz)`, arrays over sites

`align.most_perpendicular_vector() / align.most_perp_basis()`

(`classicalspintk.align`) Given a set of vectors (typically a `SpinModel`'s magnetization), find the direction most nearly perpendicular to all of them, and use it to build a rotated basis in which that direction is the new z axis, i.e. the vectors end up mostly in the new xy plane. Useful for plotting a magnetization texture (e.g. a skyrmion or spiral) as an in-plane quiver plot when it is not already aligned with a coordinate axis; see `examples/classicalspin/perpendicular/main.py`.

Returns a perpendicular unit vector, or (for `most_perp_basis`) the input vectors expressed in the rotated basis

LatticeGas functions and methods

`lg.add_interaction()`

Add a coupling shell to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- `Jij`: list of shell couplings, e.g. `[J1,J2,J3]` for first/second/third neighbor $J_{ij}n_i n_j$ repulsion (or attraction, for negative values); passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too

Modifies `lg` in place, no return value

`lg.set_filling()`

Reset the occupation array `lg.den` to a new random configuration with a given filling fraction, discarding the current snapshot (e.g. to re-seed a fresh anneal, see `examples/latticegas/optimize/main.py`).

Arguments:

- `filling`: fraction of sites occupied (rounded to the nearest integer site count)

`lg.get_energy()`

Evaluate the total energy $\sum_i \mu_i n_i + \sum_{ij} J_{ij} n_i n_j$ of the current occupation snapshot `lg.den`. Returns a scalar.

`lg.optimize_energy()`

Anneal `lg.den` towards a low-energy configuration with a Metropolis discrete-swap optimizer: at each step, 1-3 random occupied/empty site pairs are swapped (preserving the total filling) and accepted unconditionally if the energy does not increase, or with probability $e^{-\Delta E/T}$ otherwise. Only the energy change of each swap is computed, not the full energy, so the cost of a step is set by how many neighbors a site has rather than by the size of the system.

Optional arguments:

- `temp=0.1`: Metropolis temperature (higher accepts more uphill moves; anneal by calling this repeatedly with decreasing `temp`, or see `lg.anneal()` below). `temp=0` runs zero-temperature (greedy) dynamics, where an uphill move is never accepted
- `ntries=1e5`: number of swap attempts
- `resync_every=1000`: how often (in swap attempts) to recompute the energy from scratch, bounding floating-point drift in the incremental tracking

- `patience=None`: if set, stop early once this many attempts have passed without a new best energy being found (the returned array is truncated to what actually ran)
- `checkpoint_at=None`: an int or iterable of ints; captures a copy of `lg.den` after that many attempts (1-indexed) into `lg.checkpoints` (a dict `step` -> `den` snapshot), independent of the final configuration, e.g. to inspect or animate how the configuration evolves partway through a run

Overwrites `lg.den` with the final configuration and returns the array of energies recorded at each attempt (whether or not it was accepted)

`lg.anneal()`

Simulated annealing over a decreasing temperature schedule, running `lg.optimize_energy()` once per temperature in `temps`. The best configuration seen anywhere in the schedule is kept, since a high-temperature step can wander back up in energy before it ends.

Optional arguments:

- `temps=None`: sequence of temperatures, high to low; defaults to a 10-step geometric schedule from 2.0 down to 0.05 (any entry, including 0, is passed straight through to `lg.optimize_energy()`)
- `ntries=1e4`: number of swap attempts per temperature
- `checkpoint_at=None`: an int or iterable of ints; like `lg.optimize_energy()`'s `checkpoint_at`, but numbered continuously across the whole schedule (e.g. step 120 is the 20th attempt of the 3rd temperature stage if `ntries=100`), so a snapshot can be recovered after any number of annealing steps, not just the final/best configuration
- any other keyword accepted by `lg.optimize_energy()` (e.g. `patience`, `resync_every`), applied at every temperature

Overwrites `lg.den` with the best configuration found and returns the concatenated energy trajectory across all temperatures

`lg.optimize_energy_multistart()`

Run `nstart` independent anneals from independent random seeds at the current filling, and keep the lowest-energy result, which reduces the risk of a single anneal settling into a metastable configuration. Each restart is a full `lg.optimize_energy()` run; the restarts run in parallel once `parallel.set_cores()` has been called, and serially by default (see “Parallelism and reproducibility”).

Optional arguments:

- `nstart=10`: number of independent restarts
- any other keyword accepted by `lg.optimize_energy()` (e.g. `temp`, `ntries`, `patience`), applied identically to every restart

Overwrites `lg.den` with the best configuration found and returns its energy (a scalar)

lg.optimize_grand_canonical()

Grand-canonical Metropolis sampling/annealing: instead of swapping pairs at fixed filling, single sites are flipped (occupied $\leftrightarrow$ empty) and accepted/rejected the usual Metropolis way, so the total filling fluctuates under `lg.mu` rather than being conserved. This is the standard lattice-gas MC move set, useful for scanning a phase diagram vs. chemical potential, or for equilibrium sampling at one fixed temperature (see `latticegas.get_specific_heat()/get_susceptibility()` below). Unlike `lg.optimize_energy()`, it does not need both occupied and empty sites to start from: `lg.den` can start uniformly empty or full.

Optional arguments: same as `lg.optimize_energy()` (`temp`, `ntries`, `resync_every`; no `patience`), except that `temp` defaults to 1.0 here rather than 0.1. Grand-canonical sampling is usually wanted at a temperature, not as an anneal

Overwrites `lg.den` with the final configuration and returns (`es`, `ns`): the energy trajectory and the filling (occupied-site count) trajectory, both arrays of length `ntries`

latticegas.get_specific_heat() / latticegas.get_susceptibility()

Thermodynamic quantities from a trajectory sampled at one fixed temperature, as produced by `lg.optimize_energy()` or `lg.optimize_grand_canonical()` run at constant `temp` rather than annealed. `get_specific_heat(es, temp, burn=0.2)` gives $C = \text{Var}(E)/T^2$ from an energy trajectory, and `get_susceptibility(ns, temp, burn=0.2)` gives $dN/d\mu = \text{Var}(N)/T$ from a filling trajectory. The second is only meaningful in the grand canonical case, where the filling is free to fluctuate. `burn` is the leading fraction of the trajectory discarded as equilibration.

lg.add_tensor()

Add a custom coupling $J_{ij} = \text{fun}(r_i, r_j)$ between every pair of sites, for interactions beyond `add_interaction()`'s fixed neighbor shells, e.g. a screened or dipolar $1/r^n$ form. Scalar analog of `classicalspin.SpinModel.add_tensor` (which returns a 3x3 tensor per pair); self-pairs are skipped, and pairs where `fun` evaluates to (near) zero are dropped.

Modifies `lg` in place, no return value

lg.regroup()

Merge duplicate interaction-pair entries accumulated from repeated `add_interaction()/add_tensor()` calls (e.g. overlapping neighbor shells added twice), summing their couplings. A performance cleanup only; it does not change `lg.get_energy()`.

Modifies `lg` in place, no return value

lg.write() / lg.read()

Save/load the current occupation snapshot `lg.den` to/from a text file, the same checkpoint pattern as `classicalspin.SpinModel.write()/load_magnetism().write()` forces `nrep=1` (no periodic replication) by default so `read()` round-trips exactly regardless of `lg.geometry.dimensionality`; `read()` raises `ValueError` if the file's site count does not match `lg.nsites`.

Optional arguments:

- `name="DENSITY.OUT"`: file path

lg.get_local_energy() / lg.get_local_mu()

Per-site breakdown of the current snapshot's energy. `get_local_energy()` returns each site's own contribution $\mu_i n_i + \sum_j J_{ij} n_i n_j$, summed over its interaction neighbors j (`lg.get_energy()` itself counts every bond twice, once from each endpoint, so the values here sum exactly to `lg.get_energy()`, not to half of it); `get_local_mu()` instead evaluates that same expression with site `i` forced occupied, i.e. the energy cost/gain of occupying site `i` given its neighbors' current state.

Optional arguments:

- `normalize=False`: divide each site's value by the total coupling weight of its own interaction terms

Returns an array over sites

lg.get_correlator()

Neighbor-shell density-density correlator of the current snapshot `lg.den`, useful for detecting ordered ground states (e.g. after `optimize_energy`). `n` sets how many neighbor shells to return, `normalized` whether to divide out the mean occupation.

Returns (`distances`, `correlators`), arrays of matching length

lg.get_structure_factor()

Reciprocal-space structure factor $S(q) = |\sum_i (n_i - \bar{n}) e^{-iq \cdot r_i}|^2 / N$ of the current snapshot `lg.den`, evaluated directly on the real-space site positions, the

reciprocal-space companion to `lg.get_correlator()`: where the neighbor-shell correlator tells you the ordering length scale, $S(q)$ tells you the ordering wavevector. Subtracting the mean occupation makes $S(q = 0) = 0$ identically, so a peak elsewhere in q is what signals order.

Optional arguments:

- `qpath=None`: explicit array of q vectors to evaluate; if omitted, a default square grid of `nq`×`nq` points spanning $\pm$ `qmax` is used
- `nq=60`: grid resolution when `qpath` is not given
- `qmax=None`: half-width of the default grid; defaults to $2\pi/d$ set by the nearest-neighbor spacing d

Returns (`qpath`, `sq`): the array of q vectors evaluated and the matching array of $S(q)$ values

LatticeIsing functions and methods

`li.add_interaction()`

Add a coupling shell to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- `Jij`: list of shell couplings, e.g. [`J1`,`J2`,`J3`] for first/second/third neighbor $-J_{ij}s_i s_j$ exchange; positive is ferromagnetic (opposite sign convention from `LatticeGas.add_interaction()`). Passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too

Modifies `li` in place, no return value

`li.add_field()`

Add an external (Zeeman-like) field to `li.b`.

Arguments:

- `h`: a scalar (applied uniformly to every site) or a per-site array

Modifies `li` in place, no return value

`li.set_magnetization()`

Reset the spin array `li.s` to a new random ± 1 configuration with a given average magnetization, discarding the current snapshot.

Arguments:

- `m=0.0`: target average magnetization in $[-1, 1]$ (rounded to the nearest integer up-spin count)

li.get_energy()

Evaluate the total energy $-\sum_i b_i s_i - \sum_{ij} J_{ij} s_i s_j$ of the current spin snapshot `li.s`. Returns a scalar. Since `li.pairs` lists both directions of every bond, this is twice the usual sum-over-unordered-bonds convention.

li.get_magnetization()

Return `mean(s)`, the average magnetization per site of `li.s` (a scalar in $[-1, 1]$).

li.optimize_energy()

Single-spin-flip Metropolis dynamics, the standard Ising Monte Carlo move set: at each step, one random site is flipped and accepted unconditionally if the energy does not increase, or with probability $e^{-\Delta E/T}$ otherwise. Magnetization is *not* conserved (it fluctuates under `li.b`), the spin analog of `LatticeGas.optimize_grand_canonical()`.

Optional arguments:

- `temp=1.0`: Metropolis temperature; `temp=0` runs zero-temperature (greedy) dynamics
- `ntries=1e5`: number of flip attempts
- `resync_every=1000`: how often (in flip attempts) to recompute the energy from scratch, bounding floating-point drift in the incremental tracking
- `checkpoint_at=None`: an int or iterable of ints; captures a copy of `li.s` after that many attempts (1-indexed) into `li.checkpoints` (a dict `step` -> `s` snapshot)

No `patience` option: unlike `optimize_conserved()`, this trajectory is meant to be fed to `latticegas.get_specific_heat()/get_susceptibility()`, and early truncation would silently bias those variance estimates.

Overwrites `li.s` with the final configuration and returns `(es, ms)`: the energy trajectory and the total-magnetization ($\sum_i s_i$) trajectory, both arrays of length `ntries`

li.optimize_conserved()

Kawasaki spin-exchange dynamics: at each step, one up spin and one down spin are picked at random and swapped, which conserves the total magnetization, the spin analog of `LatticeGas.optimize_energy()` (swap-based, fixed filling). Raises `ValueError` if `li.s` does not have both `+1` and `-1` present (e.g. after `set_magnetization(1.0)`).

Optional arguments: same as `li.optimize_energy()`, except that `temp` defaults to `0.1` here rather than `1.0`, plus:

- `patience=None`: if set, stop early once this many attempts have passed without a new best energy being found (the returned array is truncated to what actually ran)

Overwrites `li.s` with the final configuration and returns the array of energies recorded at each attempt

`li.anneal()`

Simulated annealing over a decreasing temperature schedule: calls `li.optimize_energy()` once per temperature in `temps`, keeping the best (lowest-energy) configuration seen across the whole schedule.

Optional arguments:

- `temps=None`: sequence of temperatures, high to low; defaults to a 10-step geometric schedule from 2.0 down to 0.05
- `ntries=1e4`: number of flip attempts per temperature
- `checkpoint_at=None`: like `li.optimize_energy()`'s `checkpoint_at`, but numbered continuously across the whole schedule
- any other keyword accepted by `li.optimize_energy()`, applied at every temperature

Overwrites `li.s` with the best configuration found and returns `(es, ms)`, the concatenated energy and magnetization trajectories across all temperatures

`li.optimize_energy_multistart()`

Run `nstart` independent flip-based anneals from independent random seeds (same initial magnetization as the current `li.s`) and keep the lowest-energy result. Each restart is a full `li.optimize_energy()` run, in parallel or serially as for `lg.optimize_energy_multistart()`.

Optional arguments:

- `nstart=10`: number of independent restarts
- any other keyword accepted by `li.optimize_energy()` (e.g. `temp`, `ntries`), applied identically to every restart

Overwrites `li.s` with the best configuration found and returns its energy (a scalar)

`li.get_local_energy()` / `li.get_local_field()`

Per-site breakdown of the current snapshot. `get_local_energy()` returns each site's own contribution to `li.get_energy()` (values sum exactly to `li.get_energy()`, mirroring `LatticeGas.get_local_energy()`'s $j/2$ correction for the double-counted bonds). `get_local_field()` returns the effective field $h_i^{\text{eff}} = b_i + 2 \sum_k J_{ik} s_k$ seen by each site, defined so that flipping s_i costs exactly $2s_i h_i^{\text{eff}}$, the spin analog of `LatticeGas.get_local_mu()`.

Returns an array over sites

li.get_correlator() / **li.get_structure_factor()**

Spin-spin correlator and reciprocal-space structure factor of the current snapshot `li.s`. See `LatticeGas.get_correlator()/get_structure_factor()` for the argument reference.

li.add_tensor() / **li.regroup()**

Same as `LatticeGas.add_tensor()/regroup()`.

Modifies `li` in place, no return value

li.write() / **li.read()**

Save/load the current spin snapshot `li.s` to/from a text file. See `LatticeGas.write()/read()`. `read()` rounds to $\{-1, +1\}$ (sign, treating exact 0 as +1) to absorb the text round-trip's floating point noise, and raises `ValueError` if the file's site count does not match `li.nsites`.

Optional arguments:

- `name="SPIN.OUT"`: file path